



**Universidad
Internacional
de Valencia**

Predicción de demanda eléctrica a corto plazo mediante señales de bajo coste y Gradient Boosting

Titulación:
Máster en Inteligencia
Artificial
Curso académico
2025 – 2026

Alumno/a: Ibáñez Puertas,
Jorge
D.N.I: 48744941Y

Director/a de TFM: Mónica
Díaz Hernández

Convocatoria:
Primera

*Una jarra
se llena gota a gota.*

Gautama Buda

Agradecimientos

Agradezco a mis padres Isabel y Miguel Ángel, a mi pareja, Viktoriya, ejemplo de lucha, a mi amiga María, y, en especial, a mi hermano Miguel.

Finalmente, quiero agradecer a mis profesores de matemáticas su apoyo y cariño durante tantos años: Víctor Jiménez, Rafael Pérez y Alberto del Valle; y a mi tutora Mónica Díaz Hernández, por su gran labor dirigiendo de este proyecto.

Índice general

Índice de figuras	IV
Índice de tablas	VI
Resumen	1
Abstract	3
1. Introducción	5
1.1. Motivación y contexto	5
1.2. Alcance y limitaciones	7
1.3. Estructura del documento	8
2. Objetivos	9
3. Estado del Arte y Marco Teórico	11
3.1. Series temporales y predicción a corto plazo (Short Term Load Forecasting - STLF)	11
3.1.1. La demanda eléctrica como una serie temporal: sus componentes	11
3.1.2. Evaluación de la precisión predictiva	12
3.1.3. Métodos estadísticos clásicos y su transición al aprendizaje automático	12
3.2. LightGBM: árboles de decisión de regresión	14
3.3. Gradient Boosted Decision Trees	14
3.3.1. Formulación general	14
3.3.2. Regularización	14
3.4. LightGBM: arquitectura y optimizaciones	15
3.4.1. GOSS <i>Gradient-based One-Side Sampling</i>	15
3.4.2. EFB <i>Exclusive Feature Bundling</i>	15
3.4.3. Crecimiento por hojas (<i>leaf-wise</i>)	15
3.4.4. Manejo nativo de valores faltantes	16
3.4.5. Tratamiento de variables categóricas	16

3.4.6.	El ecosistema del gradient boosting: Random Forest, XGBoost, LightGBM y CatBoost	16
3.5.	Ingeniería de características	18
3.5.1.	Codificación cíclica de variables periódicas	18
3.5.2.	Retardos temporales (<i>lags</i>)	18
3.5.3.	Temperatura y efecto de climatización	19
3.5.4.	Detección robusta de valores atípicos: MAD insesgado	19
3.6.	Predicción autorregresiva a ciegas	20
3.7.	Revisión de la literatura y posicionamiento metodológico	20
3.7.1.	Gradient boosting: el paradigma tabular dominante para STLTF	21
3.7.2.	Aprendizaje profundo: arquitecturas y comparación crítica	22
3.7.3.	Modelos fundacionales de series temporales	23
3.7.4.	Estrategias de predicción multi-horizonte	24
3.7.5.	Posicionamiento y contribución del trabajo	24
4.	Desarrollo del Proyecto y Resultados	25
4.1.	Metodología	25
4.1.1.	Visión general del pipeline	25
4.1.2.	Ingeniería de características	27
4.1.3.	Estrategia de validación	27
4.1.4.	Configuración del modelo LightGBM	31
4.1.5.	Optimización de hiperparámetros	33
4.2.	Planteamiento del Problema	34
4.2.1.	Modelo de referencia (baseline): naive estacional	34
4.3.	Desarrollo del Proyecto	35
4.3.1.	Demanda eléctrica alemana SMARD	35
4.3.2.	Datos meteorológicos Open-Meteo/ERA5	42
4.3.3.	Festivos nacionales alemanes	46
4.3.4.	Análisis exploratorio de datos	48
4.3.5.	Descripción de los conjuntos de datos anexos	51
4.4.	Resultados	53
4.4.1.	Métricas globales: MAE, RMSE y MAPE sobre la ventana de validación en vivo	53
4.4.2.	Análisis por horizonte de predicción	56
4.4.3.	Validación operativa en vivo (julio de 2026)	58
4.4.4.	Análisis por franja horaria y día de semana	62
4.4.5.	Análisis por estación del año	63
4.4.6.	Importancia de variables (<i>feature importance</i>)	64
4.4.7.	Análisis de casos extremos	66
4.4.8.	Interpretación de los resultados	67
4.4.9.	Suficiencia de las señales de bajo coste	68

4.4.10. Comportamiento autorregresivo y papel de las variables exógenas . . .	70
4.4.11. Diferencias metodológicas con la previsión oficial de SMARD	70
4.4.12. Síntesis: resultados frente a objetivos	71
4.4.13. Limitaciones	72
5. Conclusiones y Líneas de Trabajo Futuro	79
5.1. Conclusiones	79
5.2. Líneas de trabajo futuro	80
A. Conjuntos de datos anexos	86
B. Código fuente	87
C. Hiperparámetros finales del modelo	88
Bibliografía	91

Índice de figuras

4.1. Diagrama de flujo del pipeline completo de predicción de la demanda eléctrica.	26
4.2. Curvas de aprendizaje (RMSE de entrenamiento y de validación, $\sqrt{\text{MSE}}$) del ciclo de reentrenamiento del 12 de julio de 2026 que generó el modelo actualmente desplegado, extraídas de <code>logs/cron/_entrenamiento.log</code> . La parada temprana detiene el proceso en la iteración 1.565 tras 50 iteraciones sin mejora del error de validación, y conserva el árbol óptimo de la iteración 1.515 como modelo exportado.	33
4.3. Respuesta del <i>endpoint</i> de índice (<code>index_hour.json</code>) de la API de SMARD. La clave <code>timestamps</code> contiene 597 bloques semanales (posiciones 0–596), cada uno identificado por su marca de tiempo de inicio en milisegundos UTC.	36
4.4. Últimas entradas del índice de SMARD (posiciones 584–596). El bloque más reciente (1780264800000) corresponde al domingo 31 de mayo de 2026 a las 22:00 UTC, es decir, al lunes 1 de junio a las 00:00 en hora civil de Berlín (CEST, UTC+2): los bloques semanales del índice están anclados a la medianoche local alemana, no a la UTC, en coherencia con la conversión de husos descrita en el texto. La marca confirma que el historial disponible supera los 11 años.	37
4.5. Ejemplo de respuesta del <i>endpoint</i> de datos de SMARD para el indicador 410 (consumo real). Cada entrada de <code>series</code> contiene el par [<code>timestamp_ms</code> , <code>potencia_MW</code>]. Los valores <code>null</code> corresponden a horas futuras aún no publicadas.	38
4.6. Error devuelto por la API Open-Meteo al superar el límite diario de peticiones gratuitas.	45
4.7. Evolución del MAE, el RMSE (izquierda, en MW) y el MAPE (derecha, en %) en función del horizonte de predicción h (escala logarítmica), a partir de los mismos datos de la Tabla 4.7. El crecimiento es marcadamente sub-lineal: el grueso del deterioro se concentra entre $h = 1$ y $h = 24$, seguido de una meseta entre $h = 24$ y $h = 72$, con la ligera no monotonía entre $h = 48$ y $h = 72$ visible en ambos paneles.	58

4.8. Demanda real frente a predicción sobre la ventana de validación en vivo (4–12 de julio de 2026), a los dos horizontes que enmarcan el régimen de <i>nowcasting</i> y el de <i>day-ahead</i> . A $h = 1$ la predicción se superpone casi por completo a la demanda real; a $h = 24$ se aprecia un desfase visible en los picos y valles, consistente con la asimetría de MAPE entre ambos horizontes (1,50 % frente a 3,45 %, Sección 4.4.2).	59
4.9. Captura del panel de control en producción (<code>panel_control.py</code> , 12 de julio de 2026, 21:42 hora de Berlín): demanda real, predicción del modelo a $h = 1$, previsión oficial de SMARD y trayectoria autorregresiva proyectada hacia el 13–15 de julio. El tramo violeta es una proyección hacia el futuro respecto al instante de la captura y no tiene todavía demanda real con la que compararse; las métricas del encabezado ($N = 156/168$, MAE, RMSE, Pearson R) corresponden a la ventana móvil de siete días del panel en ese instante, no a la ventana congelada de las Tablas 4.6 y 4.7.	61
4.10. Representación gráfica de la Tabla 4.8: ganancia relativa (<i>gain</i>) de cada variable en el <i>ensemble</i> LightGBM desplegado, ordenada de mayor a menor. La escala lineal deja patente la asimetría de la jerarquía: el retardo inmediato domina con claridad, el retardo de 24 h es el único otro predictor con una contribución de dos dígitos, y el resto forma una cola larga de aportaciones modestas.	65

Índice de tablas

4.1. Resumen completo de características del modelo.	28
4.2. Espacio de búsqueda de hiperparámetros.	33
4.3. Factor de corrección $MAD \rightarrow \sigma$ calibrado por simulación, según el número de muestras estacionales disponibles.	40
4.4. Ciudades seleccionadas para la extracción de temperatura horaria, por orden de población.	42
4.5. Estadísticos descriptivos de la demanda eléctrica horaria (MW), calculados sobre las 30.569 observaciones del período descriptivo 1 de enero de 2023–30 de junio de 2026.	48
4.6. Métricas de error sobre la ventana de validación operativa en vivo (4 de julio de 2026, 00:00 UTC – 12 de julio de 2026, 15:00 UTC; 208 marcas horarias con demanda real). Todas las cifras se calculan exclusivamente sobre el anexo <code>exportacion_demanda_para_metricas.xlsx</code> . Las predicciones del modelo propio son prospectivas: quedaron registradas antes de existir la demanda real correspondiente. [†] Calculada sobre las 208 marcas completas; en la comparación uno a uno con el modelo propio a $h = 24$, restringida a las 175 marcas comunes, el MAPE de SMARD es del 4,03 % (véase el texto). *El <i>naïve</i> semanal solo dispone de retardo dentro de la ventana para las últimas 40 marcas (horas de fin de semana), un subconjunto que lo favorece sistemáticamente; esta restricción proviene de que el anexo congelado solo incluye los retardos $t-1$ y $t-24h$ que emplea el modelo, no $t-168h$, y no de una carencia real de datos (Sección 4.4.13); véase el texto.	56

4.7. Evolución del MAE, RMSE y MAPE en función del horizonte de predicción h , calculada exclusivamente sobre el registro prospectivo en vivo de la ventana de validación (anexo <code>exportacion_demanda_para_metricas.xlsx</code> ; 4–12 de julio de 2026). El tamaño muestral n decrece con el horizonte porque una predicción a h horas solo puede existir para marcas posteriores en h horas al arranque efectivo de escritura de esa columna (ligeramente anterior, según los registros de la base de datos, a la medianoche del 4 de julio tomada como referencia general de la ventana; véase el texto), además de por los huecos puntuales de la Sección 4.4.3 (y, para $h = 168$, dos huecos adicionales no incluidos en esa lista). Todos los valores son medidos en condiciones operativas reales, no estimados ni retrospectivos.	57
4.8. Importancia de cada variable predictora, medida como ganancia relativa (<i>gain</i>) acumulada en el <i>ensemble</i> LightGBM, extraída directamente del modelo actualmente desplegado (<code>modelos/LightGBM_model.txt</code> , 1.515 árboles). Las 18 variables suman el 100 % de la ganancia total.	64
4.9. Comparación con resultados representativos de STLF con gradient boosting en sistemas nacionales. h = horizonte de predicción. Los valores de la literatura son los reportados o inferidos en la revisión de cada fuente. La fila a $h = 1$ del presente trabajo corresponde a un esquema de <i>nowcasting</i> (el modelo dispone de la demanda real en $t - 1$, Sección 4.4.2); no se ha verificado si las cifras de la literatura a ese mismo horizonte nominal comparten ese régimen de información, por lo que esa fila concreta debe compararse con cautela adicional.	69
4.10. Correspondencia entre los objetivos del trabajo y los resultados que los demuestran.	72
C.1. Hiperparámetros finales del modelo LightGBM, transcritos directamente de <code>conf/train.conf</code>	88

Resumen

Este trabajo de fin de máster aborda un problema clásico de *Short Term Load Forecasting*. En concreto, se centra en realizar predicciones a corto plazo de la demanda eléctrica en el mercado alemán, en horizontes de horas a días. Para cumplir este objetivo se aplica una filosofía *Open Source*, basada en tratar con datos públicos, gratuitos y de fácil obtención. La demanda eléctrica se obtiene a partir de una API de SMARD, de la agencia federal Bundesnetzagentur. Se usarán también los datos climáticos de las diez ciudades alemanas de mayor población mediante la API Open-Meteo/ERA5; y datos de días no laborables alemanes a nivel nacional. Con dichas señales se lleva a cabo el entrenamiento de un modelo *Gradient Boosting* con predicción a un paso, capaz de encadenar sus propias previsiones horarias hasta una previsión semanal. Su apoyo principal es un proceso tipo *cron* en Linux, que descarga datos cada hora en una base de datos local y fuerza un reentrenamiento semanal. La evaluación del sistema usa exclusivamente parte de ese registro: 208 observaciones horarias entre el 4 y el 12 de julio de 2026. En el horizonte diario el modelo comete un error medio del 3,45 %, mientras que la previsión oficial de SMARD alcanza un 4,03 % sobre las 175 marcas comunes a ambos. Ahora bien, esto se trata con cautela: es una ventaja aparente, pero se obtiene con una única semana de verano. A una hora, el error baja a un MAPE del 1,50 % y un MAE de unos 717 MW; aunque cabe decir que aquí el modelo juega con ventaja, es un caso de *nowcasting* y no es comparable con la previsión diaria. Todo lo observado en el proyecto sugiere que, efectivamente, se pueden utilizar señales de fácil obtención para alcanzar predicciones útiles en el nivel operativo.

Este trabajo es un desarrollo original y no parte de ningún proyecto anterior del tutor.

Palabras clave: predicción de demanda eléctrica, Short-Term Load Forecasting, LightGBM, gradient boosting, aprendizaje automático.

Abstract

This Master's thesis addresses a classic *Short Term Load Forecasting* problem. Specifically, it focuses on producing short-term forecasts of electricity demand in the German market, over horizons ranging from hours to days. To achieve this goal, an *Open Source* philosophy is applied, based on working with public, free, and easily obtainable data. Electricity demand is retrieved from a SMARD API, provided by the federal agency Bundesnetzagentur. Weather data for the ten most populous German cities are also used, obtained through the Open-Meteo/ERA5 API, along with data on non-working days in Germany at the national level. With these signals, a *Gradient Boosting* model is trained for one-step-ahead prediction, capable of chaining its own hourly forecasts up to a weekly forecast. Its main support is a *cron*-type process on Linux, which downloads data every hour into a local database and triggers a weekly retraining. The evaluation of the system relies exclusively on part of that record: 208 hourly observations between July 4 and July 12, 2026. At the daily horizon, the model achieves a mean error of 3.45 %, whereas the official SMARD forecast reaches 4.03 % over the 175 timestamps common to both. However, this result is treated with caution: it is an apparent advantage, but it is obtained from a single summer week. At the one-hour horizon, the error drops to a MAPE of 1.50 % and an MAE of around 717 MW; although it should be noted that here the model has an advantage, as this is a *nowcasting* case and is not comparable with the daily forecast. Everything observed in the project suggests that easily obtainable signals can indeed be used to achieve useful predictions at the operational level.

This work is an original development and does not stem from any previous project of the tutor.

Keywords: electricity demand forecasting, Short-Term Load Forecasting, LightGBM, gradient boosting, machine learning.

Introducción

1

1.1 Motivación y contexto

La red eléctrica tiene la misma limitación en todo el mundo, y no tiene solución sencilla: a día de hoy no tenemos los medios tecnológicos para poder almacenar energía de forma económica. Esto es lo que define el dilema del operador de red: producción y demanda será mejor que coincidan todo momento. Siempre. Una vez aparece un equilibrio con capacidad de perpetuarse en el tiempo, la frecuencia de la red se desvía, lo que activa desconexiones en cadena de la red; y en caso extremo en el que no se llegue a controlar, un apagón generalizado ([Energuide, 2026](#)). Un ejemplo reciente: durante la pandemia de COVID-19 la demanda cayó de forma imprevista y el sistema quedó con un exceso de generación, que hubo que corregir deteniendo la actividad de varias centrales ([Aziz y Than Oo, 2020](#)).

La frecuencia de la red funciona como un termómetro en tiempo real del equilibrio entre generación y demanda; en Europa oscila en torno a los 50 Hz ([Fraunhofer Institute for Solar Energy Systems ISE, 2026](#)). [Kundur \(1994\)](#) lo formaliza así:

- Si la generación supera a la demanda, hay un exceso de generación: los rotores de las turbinas se aceleran y la frecuencia *sube* por encima de los 50 Hz ($f > 50$ Hz).
- Si la demanda supera a la generación, hay un exceso de demanda: los rotores sufren un frenado electromagnético, los generadores ceden energía cinética y la frecuencia *baja* por debajo de los 50 Hz ($f < 50$ Hz).

Este comportamiento queda descrito por la ecuación de oscilación del rotor (*swing equation*) que introduce [Kundur \(1994\)](#):

$$\frac{2H}{\omega_s} \frac{d^2\delta}{dt^2} = P_m - P_e \quad (1.1)$$

donde H es la constante de inercia, ω_s la velocidad angular síncrona, δ el ángulo del rotor, P_m la potencia mecánica de entrada y P_e la potencia eléctrica de salida. En la práctica, la variación de frecuencia es la señal que indica a las centrales si deben inyectar más o menos potencia.

Ahora bien, si la frecuencia se desvía lo suficiente y sobrepasa un umbral establecido por los controladores de seguridad, los relés de protección desconectan generación para evitar

daños en las centrales; algo parecido, al salto de los fusibles en una vivienda, pero mucho más grave porque es a escala nacional. Este proceso puede propagarse hasta colapsar el sistema completo (Kundur, 1994) si no se controla.

Todo lo anterior demuestra la importancia de predecir correctamente la demanda eléctrica. Es claramente un elemento crítico, y por ello todo estado en el mundo tiene un organismo regulador llamado operador del sistema con el único objetivo de satisfacer dos misiones a la vez: vigilar la frecuencia de la red, y vigilar la red de alta tensión. Si no fuera por su labor, la red eléctrica resultaría inoperativa. En Europa destacan organismos como *Red Eléctrica de España* (REE) o, en el caso alemán, los cuatro TSO (*Transmission System Operators*) que publican sus datos en la plataforma [SMARD](#), fuente fundamental de este trabajo.

En la literatura, esta tarea de predicción se conoce como *Short-Term Load Forecasting* (STLF) (Gross y Galiana, 1987; Ullah et al., 2024), y consiste en pronosticar hora a hora las cargas demandadas. Los horizontes pueden ir desde una hora hasta un mes completo, aunque lo habitual es trabajar entre una hora y una semana.

Una previsión de demanda también minimiza el recurso a las reservas de regulación de la red. Estas actúan como colchón de seguridad pero resultan muy costosas (Gross y Galiana, 1987). Una buena previsión facilita también la integración de energías renovables como la solar y la eólica, cuya producción es variable y difícil de anticipar, lo que exige tanto previsión como flexibilidad (Ullah et al., 2024). Este aspecto es de especial interés en la Unión Europea y, en particular, en España, donde la llamada transición ecológica figura entre los objetivos declarados de Red Eléctrica de España (Red Eléctrica de España, 2026).

No es un problema nuevo ni moderno. Como recogen Ullah et al. (2024) y Gross y Galiana (1987), la predicción de demanda lleva décadas estudiándose con métodos estadísticos. Lo que sí ha cambiado es el contexto actual. La irrupción de las renovables, de los coches eléctricos... son factores que añaden complejidad y no linealidad al sistema (Haben et al., 2023). Este nuevo marco de hoy justifica la entrada en juego del aprendizaje automático en el problema de consumo-predicción. Y por suerte, se puede resolver con señales fáciles. Muchos gobiernos con el desarrollo de internet, han apostado por los datos abiertos y el Open Source, lo que ha mejorado el acceso del público a información de calidad gratuita. Plataformas como SMARD ([Strommarktdaten](#), Bundesnetzagentur, Alemania) ofrecen series temporales de consumo real y de previsión oficial, con resolución horaria y de forma totalmente gratuita. En el caso Alemán, la accesibilidad a los datos es total, lo que motiva la elección de este país para llevar a cabo el proyecto. Luego además está la temperatura, que también influye en la demanda. Servicios meteorológicos como [Open-Meteo](#) publican, sin coste alguno, series temporales de temperatura por localidades. En concreto para este proyecto se seleccionan las diez ciudades alemanas más pobladas y se obtiene la serie de temperaturas de cada una de ellas.

Con la combinación de estos datos, a la que se pueden añadir variables de calendario (por ejemplo, días festivos), se pretende construir un predictor de demanda competitivo a partir únicamente de esas *señales fácilmente observables*: temperatura, calendario, y valores de demanda pasados (un enfoque autorregresivo). Esto lleva a este proyecto a la difusa

frontera entre estadística y aprendizaje automático, que es justo donde vive el forecasting moderno. Como se ha expuesto, aunque es cierto que se lleva realizando décadas un enfoque estadístico a este problema, el aprendizaje automático tiende a resolverlo mejor sobre datos heterogéneos ([Grinsztajn et al., 2022](#)). Lo que sigue es un proyecto de machine learning aplicado al forecasting eléctrico, tratar de ver como unas pocas señales sencillas son suficientes para obtener predicciones solventes.

1.2 Alcance y limitaciones

Se ha elegido para el desarrollo del proyecto datos de Alemania, cuyo consumo eléctrico bruto anual supera los 500 TWh ([Federal Ministry for Economic Affairs and Climate Action \(BMWK\), 2025](#)). La base de datos de producción de este trabajo consiste en un historial horario consolidado desde el 29 de junio de 2016 (88.111 registros en el volcado de julio de 2026, con cobertura horaria prácticamente completa; Sección 4.3.5), pero su uso en esta memoria está deliberadamente acotado: el entrenamiento del modelo emplea una ventana móvil de los últimos 730 días (unos 24 meses), y la evaluación se realiza exclusivamente sobre una ventana de validación operativa en vivo, del 4 al 12 de julio de 2026, en la que cada predicción quedó registrada en la base de datos antes de que existiera la demanda real correspondiente, a siete horizontes de entre 1 hora y 1 semana. Se trata de una decisión de alcance del trabajo, no de una limitación de disponibilidad de datos: el histórico anterior a esa ventana móvil de entrenamiento (desde el 29 de junio de 2016 hasta mediados de 2024, aproximadamente) permanece en la base de datos y queda disponible como línea de trabajo futura (Sección 5.2), pero no se emplea ni en el entrenamiento ni en la evaluación aquí descritos. Esto se debe a un problema encontrado durante el desarrollo, que ha obligado a realizar un nuevo entrenamiento y nuevas predicciones que no se esperaba.

Este trabajo no trata ni la predicción de precios de mercado, la asignación de unidades de generación (*unit commitment*), la predicción de generación renovable, ni factores sociales, económicos o estratégicos (por ejemplo, conflictos bélicos o pandemias). El objeto de estudio es estrictamente la demanda agregada del sistema, que incorpora toda esa información de forma difusa. Tampoco se comparan modelos de aprendizaje profundo ni arquitecturas recurrentes. La elección de gradient boosting responde al equilibrio documentado en la literatura, entre capacidad predictiva, interpretabilidad y eficiencia computacional para este tipo de problema tabular ([Chen y Guestrin, 2016](#)). La generalización a otros países o a periodos fuera de la muestra queda como línea de trabajo futura. Por último en cada ventana de reentrenamiento se aplica una partición estricta cronológica, doce meses de entrenamiento y doce de validación para detectar sobreajuste. El esquema inicial, de doce y tres meses, se amplió tras detectar un sesgo estacional (ver Sección 4.1.3).

1.3 Estructura del documento

Esta memoria se organiza en los siguientes capítulos. El Capítulo 2 recoge los objetivos del proyecto y el Capítulo 3 desarrolla el estado del arte, desde las series temporales y sus métricas hasta el gradient boosting, LightGBM y la predicción autorregresiva. La metodología, los datos y sus fuentes, la configuración y los resultados (con un análisis por horizonte y la comparación con SMARD) ocupan el Capítulo 4. Cierra el Capítulo 5 con las conclusiones y las principales líneas de trabajo futuro.

Objetivos

2

El objetivo general de este proyecto es diseñar, implementar y evaluar un sistema de inteligencia artificial basado en LightGBM para la predicción horaria de la demanda eléctrica a corto plazo en el mercado alemán. El modelo se alimenta únicamente de señales públicas y gratuitas. Interesa comprobar, por tanto, hasta qué punto pueden obtenerse predicciones competitivas sin recurrir a fuentes de datos costosas o de acceso restringido.

Este objetivo general se concreta en los siguientes objetivos específicos:

1. Diseñar y validar un pipeline de predicción de demanda eléctrica horaria que alcance resultados competitivos empleando exclusivamente señales meteorológicas, de calendario y autorregresivas, obtenidas de los datos históricos públicos de SMARD para Alemania. El pipeline se apoya en un modelo LightGBM configurado para regresión sobre series temporales, con codificación cíclica de las variables periódicas y una gestión explícita de los retardos temporales.
2. Evaluar si el conjunto de señales de bajo coste empleado basta para alcanzar un error absoluto medio competitivo. Estas señales son tres: la temperatura horaria de las principales ciudades del país, las variables de calendario (hora del día, día de la semana y festivos nacionales) y los retardos temporales ($t-1$ y $t-24$ h). Conviene precisar que este trabajo no aísla la contribución individual de cada familia de variables mediante ablación, por lo que la conclusión de suficiencia se predica del conjunto y no de cada señal por separado.
3. Analizar el comportamiento del modelo en modo autorregresivo puro: estudiar cómo se propaga el error cuando el modelo emplea sus propias predicciones pasadas como entrada para horizontes superiores a una hora, sin corrección alguna con valores reales.

Estado del Arte y Marco Teórico

3

3.1 Series temporales y predicción a corto plazo (Short Term Load Forecasting - STLF)

[Stock y Watson \(2020\)](#) define una serie temporal como una secuencia de observaciones de una variable aleatoria ordenadas por un índice temporal t . La notación habitual para estas sucesiones es $\{y_t\}_{t=1}^T$, donde t es el instante de observación y T el total de observaciones con que se cuenta. En el problema de la demanda eléctrica, y_t es la energía consumida durante el intervalo $[t, t + \Delta t)$. Además en este trabajo se opta por una resolución horaria $\Delta t = 1$ h. El *forecasting* a corto plazo consiste en estimar valores futuros $\hat{y}_{T+h}$ a partir de información disponible en el instante T ([Haben et al., 2023](#)). La demanda en un instante dado y_t se puede explicar combinando la demanda del instante inmediatamente interior (su inercia histórica) con la influencia de factores externos que pueden anticiparse como la temperatura o el ciclo de calendario ([Hedengren y APMonitor Development Team, 2024](#)). En esencia, un modelo autorregresivo con variables exógenas (ARX):

$$\hat{y}_{T+h} = f(y_T, y_{T-1}, \dots, y_{T-p}, \mathbf{x}_{T+h}), \quad (3.1)$$

donde p es el orden de retardo considerado, h el horizonte de predicción (número de pasos hacia adelante) y $\mathbf{x}_{T+h}$ el vector de variables exógenas conocidas en $T + h$ (meteorología, calendario, etc.). Si nos restringimos al caso lineal, esta misma idea se escribe como:

$$\hat{y}_{T+h} = c + \sum_{i=0}^p a_i y_{T-i} + \mathbf{b}^\top \mathbf{x}_{T+h} + \varepsilon_{T+h}, \quad (3.2)$$

con $c, a_i, \mathbf{b}$ como coeficientes a ajustar y ε_{T+h} como término de error.

3.1.1 La demanda eléctrica como una serie temporal: sus componentes

La ecuación definida anteriormente [3.1](#) es cierto que sirve como punto de partida teórico, pero la demanda eléctrica real, *realmente* va mucho más allá: sus variaciones no se explican solo con relaciones lineales con el pasado. La demanda tiene ritmo y ciclos. Varios, de hecho: el consumo obedece a ciclos diarios, semanales y anuales muy marcados (vacaciones, fines de semana, festivos, horario laboral...), es decir tiene estacionalidad ([Haben et al., 2023](#)). Una

mejor aproximación sería pues:

$$y_t = S_t^{(\text{diario})} + S_t^{(\text{semanal})} + S_t^{(\text{anual})} + c(T_t) + \varepsilon_t, \quad (3.3)$$

donde los términos $S_t^{(\cdot)}$ recogen las estacionalidades periódicas, $c(T_t)$ es una función no lineal de la temperatura T_t y ε_t el residuo estadístico. Las variables cíclicas de calendario aproximan los términos $S_t^{(\cdot)}$, las temperaturas urbanas dan cuenta de $c(T_t)$ y los retardos (valores pasados de demanda) se encargan de la autocorrelación que queda en el residuo ε_t .

3.1.2 Evaluación de la precisión predictiva

Las métricas que [Hosamo y Mazzetto \(2025\)](#) recogen como estándar para evaluar modelos de STLF son las siguientes:

MAE Error Absoluto Medio

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (3.4)$$

Se comporta bien frente a *outliers* y como se expresa en la misma unidad que la demanda (MW), resulta fácil de interpretar en términos operativos.

RMSE Raíz del Error Cuadrático Medio

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (3.5)$$

Castiga con más dureza los errores grandes. Equivale a minimizar la norma L_2 .

MAPE Error Porcentual Absoluto Medio

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{y_i} \quad (3.6)$$

Al expresar el error en porcentaje relativo, permite comparar sistemas con niveles de demanda muy distintos. Su inconveniente es que no está definida cuando $y_i = 0$.

3.1.3 Métodos estadísticos clásicos y su transición al aprendizaje automático

Entre los años ochenta y noventa, la predicción de carga a corto plazo fue territorio casi exclusivo de los métodos estadísticos paramétricos, apreciados por su interpretabilidad y por lo poco que exigían en términos de cómputo ([Hong y Fan, 2016](#); [Hyndman y Athanasopoulos, 2021](#)). Cabe distinguir tres grandes familias.

Suavizado exponencial. La idea de fondo es sencilla: ponderar las observaciones pasadas con pesos que decaen geométricamente conforme nos alejamos en el tiempo. Su extensión clásica a series con tendencia y estacionalidad es el método de Holt-Winters ([Holt,](#)

2004; Winters, 1960), que separa la serie en nivel ℓ_t , tendencia b_t y componente estacional s_t . Winters, por cierto, no pensaba en kilovatios cuando lo formuló en 1960, sino en previsión de ventas; la electricidad heredó el método después. En su versión aditiva:

$$\begin{aligned}\hat{y}_{t+h} &= \ell_t + h b_t + s_{t+h-m(k+1)}, \\ \ell_t &= \alpha (y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}), \\ b_t &= \beta (\ell_t - \ell_{t-1}) + (1 - \beta) b_{t-1}, \\ s_t &= \gamma (y_t - \ell_{t-1} - b_{t-1}) + (1 - \gamma) s_{t-m},\end{aligned}\tag{3.7}$$

donde m es el período estacional, $k = \lfloor (h - 1)/m \rfloor$ y $\alpha, \beta, \gamma \in (0, 1)$ son los parámetros de suavizado. Años más tarde, Hyndman et al. (2008) formalizaron toda esta familia bajo el marco de espacio de estados ETS (*Error, Trend, Seasonality*), que hizo posible seleccionar el modelo de forma automática y estimar intervalos de predicción.

Modelos ARIMA y la metodología de Box-Jenkins. Los modelos autorregresivos integrados de media móvil (ARIMA), introducidos por Box y Jenkins (1970), combinan componentes autorregresivos (AR), de media móvil (MA) y de diferenciación (I) para hacer frente a la no estacionariedad. La variante SARIMA añade términos estacionales y SARIMAX incorpora además regresores exógenos (habitualmente la temperatura y variables de calendario), de modo que puede verse como el análogo lineal directo de la formulación ARX de la Ecuación (3.1). Ahora bien, en el contexto de la demanda eléctrica horaria SARIMA tropieza con un obstáculo importante: le cuesta modelar varias estacionalidades a la vez (diaria, semanal y anual). Para sortearlo se han propuesto extensiones como TBATS o la introducción de términos de Fourier como regresores (De Livera et al., 2011; Hyndman y Athanasopoulos, 2021).

Modelos de regresión. El tercer pilar histórico del STLF es la regresión lineal múltiple con variables meteorológicas y de calendario (Hong y Fan, 2016; Ramanathan et al., 1997). Estos modelos hacen explícita la relación entre demanda, temperatura y tipo de día, y son el antecedente conceptual más directo del enfoque basado en características que se adopta aquí: las mismas señales que alimentarían una regresión clásica (retardos, temperatura, calendario) son las que se le proporcionan al modelo de gradient boosting.

¿Por qué se produjo entonces la transición hacia el aprendizaje automático? Porque estos métodos comparten limitaciones de fondo: asumen relaciones esencialmente lineales, obligan a especificar a mano la estructura estacional y las transformaciones de las variables, y capturan mal la respuesta no lineal de la demanda a la temperatura (la curva en «U» que se describe en la Sección 3.5.3). A estas carencias se sumó la creciente disponibilidad de datos y de capacidad de cómputo, que allanó el camino a métodos no paramétricos como los *ensembles* de árboles (Hastie et al., 2009; Hong y Fan, 2016).

3.2 LightGBM: árboles de decisión de regresión

El punto de partida es el árbol de decisión de regresión, tal y como lo presenta [Hastie et al. \(2009\)](#): un modelo que particiona recursivamente el espacio de características $\mathcal{X} \subseteq \mathbb{R}^d$ en regiones rectangulares $R_1, \dots, R_J$ y asigna a cada una un valor constante γ_j :

$$T(\mathbf{x}; \Theta) = \sum_{j=1}^J \gamma_j \cdot \mathbf{1}[\mathbf{x} \in R_j], \quad (3.8)$$

donde $\Theta = \{(R_j, \gamma_j)\}_{j=1}^J$ son los parámetros aprendidos y $\mathbf{1}[\mathbf{x} \in R_j]$ una función indicatriz. Cada partición se decide buscando el par (característica, umbral) que minimiza la impureza del nodo, que en regresión suele medirse mediante el error cuadrático medio. Un árbol aislado plantea un dilema: si es poco profundo, presenta baja varianza pero alto sesgo; si es profundo, ocurre lo contrario. Los métodos de *ensemble* resuelven esta tensión combinando muchos árboles para lograr un estimador con mejor equilibrio entre sesgo y varianza ([Hastie et al., 2009](#)).

3.3 Gradient Boosted Decision Trees

3.3.1 Formulación general

El gradient boosting, propuesto por [Friedman \(2001\)](#), construye el modelo de forma aditiva e iterativa: en cada iteración m se incorpora un árbol T_m que se ajusta a los *pseudo-residuos* del modelo acumulado hasta la iteración anterior:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot T_m(\mathbf{x}), \quad (3.9)$$

donde $\eta \in (0, 1]$ es la tasa de aprendizaje (*learning rate*), que modula la contribución de cada árbol y funciona de facto como un regularizador implícito. Los pseudo-residuos de la iteración m son las derivadas negativas de la función de pérdida $\mathcal{L}$ respecto a las predicciones del modelo anterior:

$$r_{im} = - \left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}. \quad (3.10)$$

En el caso particular de la pérdida cuadrática $\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2/2$, los pseudo-residuos coinciden con los residuos ordinarios: $r_{im} = y_i - F_{m-1}(\mathbf{x}_i)$. El algoritmo se detiene ([Chen y Guestrin, 2016](#); [Ke et al., 2017](#)) bien al alcanzar el número máximo de iteraciones M (hiperparámetro `num_iterations`), bien cuando la pérdida de validación deja de mejorar (*early stopping*).

3.3.2 Regularización

El gradient boosting dispone de varios mecanismos de regularización que se complementan entre sí. El más básico es la propia tasa de aprendizaje η , que actúa como factor de

encogimiento (*shrinkage*): con valores pequeños, del orden de 0,01–0,05, el modelo necesita más iteraciones para converger, pero a cambio generaliza mejor, porque ningún árbol individual llega a pesar demasiado. A esto se añaden dos formas de submuestreo. La de datos (*bagging fraction*) entrena cada árbol sobre una fracción aleatoria $p \in (0, 1]$ de las instancias, con el efecto de reducir la covarianza entre árboles; la de características (*feature fraction*) limita las columnas que cada árbol puede ver, e impide así que el modelo se apoye en exceso en un único predictor. Quedan dos frenos estructurales. Restringir el número de hojas J mantiene bajo control la complejidad de cada árbol y dificulta el sobreajuste a patrones espurios locales; `min_data_in_leaf`, por último, exige un mínimo de instancias en cada hoja terminal, de forma que el árbol no pueda memorizar ruido estadístico en las zonas menos pobladas del espacio de características (Friedman, 2001).

3.4 LightGBM: arquitectura y optimizaciones

Ke et al. (2017) presentan LightGBM como una implementación de gradient boosting cuya principal aportación son dos algoritmos novedosos que recortan el tiempo de entrenamiento sin renunciar a la precisión:

3.4.1 GOSS *Gradient-based One-Side Sampling*

La intuición de GOSS es que, en cada iteración, las instancias con gradientes grandes son las que realmente empujan el ajuste del modelo. Por eso, el algoritmo retiene el $a\%$ de instancias con gradientes más elevados y, del $(1 - a)\%$ restante, muestrea aleatoriamente solo un $b\%$ (Ke et al., 2017). Para no distorsionar la distribución, las instancias muestreadas de bajo gradiente se ponderan con un factor $(1 - a)/b$:

$$\tilde{\mathcal{L}} = \frac{1}{n} \sum_{i \in \mathcal{A}} l_i + \frac{1 - a}{b} \cdot \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} l_j, \quad (3.11)$$

donde $\mathcal{A}$ es el conjunto de alto gradiente y $\mathcal{B}$ la muestra aleatoria de bajo gradiente.

3.4.2 EFB *Exclusive Feature Bundling*

En conjuntos de datos dispersos (*sparse*) es frecuente que muchas características tomen valores no nulos de forma casi excluyente, es decir, que rara vez estén activas a la vez. EFB explota esta circunstancia: identifica esos grupos de características mutuamente excluyentes y los consolida en un único vector, de modo que el algoritmo de construcción del árbol evalúa un número efectivo de dimensiones bastante menor (Ke et al., 2017).

3.4.3 Crecimiento por hojas (*leaf-wise*)

Mientras que XGBoost construye los árboles nivel a nivel (*level-wise*), LightGBM expande en cada paso la hoja con mayor ganancia de pérdida, sin importar a qué profundidad se

encuentre. El resultado son árboles más profundos con menos hojas para el mismo nivel de error, aunque esta agresividad tiene un precio: en conjuntos de datos pequeños puede favorecer el sobreajuste (Ke et al., 2017). De hecho, si no se limita la profundidad, el modo *leaf-wise* puede producir árboles degenerados o sesgados, con una rama larguísima y el resto del árbol apenas desarrollado. Para mantener este riesgo a raya (LightGBM Developers, 2025) se recurre a dos hiperparámetros que actúan como limitadores complementarios: `max_depth`, que impone un techo absoluto a la profundidad del árbol, y `min_data_in_leaf`, que exige un número mínimo de muestras por hoja y elimina así las microparticiones que memorizan ruido.

3.4.4 Manejo nativo de valores faltantes

LightGBM incorpora de serie la gestión de valores nulos (NaN) durante la construcción del árbol. En cada división candidata, el algoritmo evalúa a qué rama (izquierda o derecha) conviene enviar las instancias con NaN para minimizar la pérdida, y elige la asignación óptima. En la práctica, esto equivale a aprender, nodo a nodo, cuál es la dirección de imputación más informativa según los datos de entrenamiento. Para series temporales con huecos ocasionales en las variables meteorológicas, como ocurre en este trabajo, esta propiedad resulta muy conveniente: ahorra la imputación previa y deja que sea el propio modelo quien aprenda cómo comportarse ante datos faltantes (Ke et al., 2017).

3.4.5 Tratamiento de variables categóricas

Encontrar la partición óptima de una variable categórica en un árbol de decisión es, en principio, un problema combinatorio: con k categorías, el número de particiones binarias posibles asciende a $2^{k-1} - 1$, lo que, como demuestran Breiman et al. (1984), se vuelve intratable en cuanto k crece. Afortunadamente, el resultado clásico de Fisher (1958) muestra que, en regresión con pérdida cuadrática, el problema admite solución exacta en tiempo $O(k \log k)$: basta ordenar las categorías por el valor medio de la variable objetivo y buscar el umbral de corte óptimo sobre esa secuencia ordenada, de modo que la búsqueda pasa de 2^{k-1} a $k - 1$ evaluaciones. LightGBM implementa una generalización de este resultado basada en histogramas de gradientes: ordena las categorías por la ratio gradiente/hessiano acumulado y localiza la partición de menor pérdida en tiempo proporcional a $k \log k$. Todo ello sin recurrir al *One-Hot Encoding*, con menor coste de memoria y mejor precisión predictiva que la codificación binaria explícita (Ke et al., 2017).

3.4.6 El ecosistema del gradient boosting: Random Forest, XGBoost, LightGBM y CatBoost

LightGBM no es, ni mucho menos, la única implementación de *ensembles* de árboles competitiva en problemas tabulares. Situarlo en su ecosistema permite justificar la elección de forma razonada, en lugar de darla por supuesta.

Random Forest. Como contraste metodológico, Random Forest ([Breiman, 2001](#)) combina árboles mediante *bagging*: cada árbol se entrena sobre una remuestra *bootstrap* y las predicciones se agregan por promediado, en lugar de construirse de forma aditiva y secuencial como en el boosting. Esta estrategia reduce la varianza del estimador, pero no corrige el sesgo de forma iterativa; por eso, en problemas de STLF con fuerte componente no lineal, los métodos de boosting suelen llevarle ventaja ([Hastie et al., 2009](#)).

XGBoost. XGBoost ([Chen y Guestrin, 2016](#)) fue el gran responsable de popularizar el gradient boosting regularizado a escala. Aporta un término de regularización explícito sobre la complejidad del árbol, el uso de información de segundo orden (Hessiano) en la función objetivo y un algoritmo *sparsity-aware*. Su crecimiento por niveles (*level-wise*) lo hace robusto, aunque menos eficiente que el crecimiento por hojas cuando los conjuntos de datos son grandes.

LightGBM. Como se ha descrito en las secciones anteriores, [Ke et al. \(2017\)](#) introducen GOSS, EFB y el crecimiento por hojas (*leaf-wise*), tres piezas que en conjunto le otorgan una ventaja sustancial en velocidad de entrenamiento sobre *datasets* de gran volumen, sin merma de precisión.

CatBoost. Por su parte, CatBoost ([Prokhorenkova et al., 2018](#)) aporta dos innovaciones que interesan especialmente a este trabajo. La primera es el *ordered boosting*, que mitiga el sesgo de predicción (*prediction shift*) que aparece cuando se usan los mismos datos para estimar gradientes y para construir el modelo. La segunda es el tratamiento de variables categóricas mediante estadísticos de objetivo ordenados (*ordered target statistics*), que evita la fuga de información propia del *target encoding* convencional; este mecanismo guarda relación directa con el tratamiento que aquí se da a las variables *dia_semana* y *es_festivo* (Sección 4.1.2).

Comparación y justificación de la elección. Los estudios comparativos sistemáticos ([Bentéjac et al., 2021](#); [Grinsztajn et al., 2022](#)) apuntan a que las tres bibliotecas de boosting alcanzan una precisión similar cuando se ajustan con cuidado; las diferencias reales aparecen en la eficiencia y en escenarios concretos. CatBoost brilla cuando abundan las variables categóricas de alta cardinalidad, mientras que LightGBM suele ser el más rápido sobre grandes volúmenes de datos numéricos. Conviene ser precisos, no obstante, sobre dónde pesa aquí ese argumento de eficiencia. Aunque la base de datos de producción acumula más de 87 000 registros históricos, el conjunto que interviene en cada entrenamiento es mucho menor: la ventana móvil de 730 días equivale a unas 17 500 filas horarias, de las que solo la mitad (unos 8 800 registros) ajusta efectivamente los árboles bajo la partición cronológica 50/50 (Sección 4.1.3). A esa escala, la diferencia de velocidad bruta entre las tres bibliotecas en un entrenamiento aislado se mide en segundos. Poca cosa. No justificaría, por sí sola, la elección. Donde la eficiencia sí resulta valiosa es en el número de entrenamientos: la búsqueda de hiperparámetros explora de forma iterativa un espacio de miles de combinaciones (Tabla 4.2); el ciclo de producción reentrena cada semana de forma desatendida; y los episodios de retrabajo fuera del ciclo ordinario obligan a encadenar entrenamientos completos en poco tiempo. Los de comienzos de julio de 2026 sirven de ejemplo: la depuración que

siguió al episodio de datos anómalos del 25 de junio y al incidente técnico posterior (Secciones 4.3.1 y 4.1.3) exigió reentrenar una y otra vez en cuestión de horas, y ahí un ciclo que se completa en segundos abarata cada iteración de la depuración. A este argumento operativo se suman los motivos ajenos a la velocidad: las pocas variables categóricas presentes son de baja cardinalidad (`dia_semana`, `es_festivo`), así que no hacen falta las garantías de CatBoost; el manejo nativo de valores faltantes (Sección 3.4.4) viene muy bien ante los huecos ocasionales de las series meteorológicas; y el binario autónomo de LightGBM en C++ encaja con el despliegue ligero del sistema, que entrena sin depender de un entorno Python.

3.5 Ingeniería de características

3.5.1 Codificación cíclica de variables periódicas

Las variables de calendario, como la hora del día o el día del año, son numéricamente periódicas: la hora 23 y la hora 0 son adyacentes en el tiempo, pero su diferencia aritmética es 23. Si se representan como enteros sin más, se introduce un sesgo artificial que los árboles de decisión, al trabajar con umbrales, no pueden corregir por sí solos. La codificación sinusoidal resuelve el problema transformando cada variable periódica v con periodo P en un par de coordenadas sobre el círculo unitario (Costa et al., 2025; Grierson, 2025):

$$v_{\sin} = \sin\left(\frac{2\pi v}{P}\right), \quad v_{\cos} = \cos\left(\frac{2\pi v}{P}\right). \quad (3.12)$$

De este modo, $v = 0$ y $v = P$ pasan a representarse por el mismo punto $(0, 1)$, y la continuidad circular queda preservada. En este trabajo la transformación se aplica a la hora del día ($P = 24$) y al día del año ($P = 365$, o 366 en años bisiestos), usando la hora fraccional $v = h + m/60$ para conservar la resolución subhoraria.

3.5.2 Retardos temporales (*lags*)

Los retardos son, sencillamente, copias desplazadas en el tiempo de la variable objetivo, que se incorporan al modelo como características adicionales:

$$\text{lag}_k(t) = y_{t-k\Delta t}. \quad (3.13)$$

En este trabajo se emplean dos:

- $\text{lag}_1 = y_{t-1h}$, que recoge la inercia inmediata del sistema.
- $\text{lag}_{24} = y_{t-24h}$, que captura el patrón diario (misma hora del día anterior).

Sobre la ventana de evaluación de este trabajo (`exportacion_demanda_para_metricas.xlsx`, 208 marcas horarias, Sección 4.3.5), la autocorrelación de Pearson entre la demanda y su propio valor retardado k horas es de $\rho_{t,t-1} \approx 0,97$ ($n = 207$ pares) para el retardo inmediato

y de $\rho_{t,t-24} \approx 0,68$ ($n = 184$ pares) para el diario. Merece la pena señalar que el retardo semanal, no incluido en el modelo, presenta una autocorrelación intermedia ($\rho_{t,t-168} \approx 0,91$, si bien con solo $n = 40$ pares dentro de esta ventana breve, cifra a tomar como orientativa), sensiblemente superior a la del diario: la demanda de la misma hora de la semana anterior comparte tipo de día (laborable o fin de semana), mientras que la de hace 24 horas puede cruzar la frontera entre ambos regímenes. Este hecho refuerza la incorporación del retardo $t-168$ h como línea de trabajo futura (Sección 5.2).

3.5.3 Temperatura y efecto de climatización

La relación entre temperatura y demanda eléctrica suele adoptar una forma de «U» asimétrica: la demanda toca fondo en temperaturas templadas y repunta tanto con el frío extremo como con el calor extremo (Valor et al., 2001). La influencia de la temperatura T puede modelarse de forma aproximada como:

$$c(T) = \alpha \cdot \max(0, T_{\text{ref}} - T)^{\beta_1} + \gamma \cdot \max(0, T - T_{\text{confort}})^{\beta_2}, \quad (3.14)$$

donde T_{ref} es la temperatura de referencia para calefacción, T_{confort} el umbral de activación de la refrigeración, y $\alpha, \beta_1, \gamma, \beta_2$ son parámetros del modelo. En cuanto a los datos, se utilizan las temperaturas horarias de las diez ciudades más pobladas de Alemania (Berlín, Hamburgo, Múnich, Colonia, Fráncfort, Stuttgart, Düsseldorf, Leipzig, Dortmund y Essen), obtenidas de la API Open-Meteo (Open-Meteo, 2026a,b) a través del archivo ERA5. Esta representación multi-punto refleja la heterogeneidad geográfica del sistema alemán bastante mejor que una única temperatura promedio nacional: un frente frío puede estar barriando Hamburgo mientras Múnich sigue templada, y ese matiz se pierde por completo en la media.

3.5.4 Detección robusta de valores atípicos: MAD insesgado

La Desviación Absoluta Mediana (*Median Absolute Deviation*, MAD) es un estimador robusto de dispersión, preferible a la desviación típica clásica cuando la serie puede contener valores atípicos extremos. La razón es intuitiva: al construirse sobre la mediana en lugar de la media, un único valor disparatado no puede desplazar arbitrariamente el estadístico. Para poder usarlo como estimador de la desviación típica bajo el supuesto de normalidad, el MAD crudo debe escalarse por una constante de consistencia c :

$$\hat{\sigma} = c \cdot \text{mediana}_i(|x_i - \text{mediana}(x)|). \quad (3.15)$$

El valor que suele citarse, $c \approx 1,4826$, es en realidad la constante asintótica: procede de $1/\Phi^{-1}(0,75)$ y solo es exacta cuando el tamaño de muestra n tiende a infinito.

El matiz importa. Con n finito y pequeño, el MAD crudo subestima sistemáticamente la dispersión real de la muestra; aplicar la constante asintótica en ese régimen infla artificialmente cualquier estadístico z construido sobre él, y eleva la tasa de falsos positivos justo

cuando menos datos hay para decidir. Akinshin (2021) repasa las principales propuestas para corregir este sesgo de muestra finita: los valores tabulados de Croux y Rousseeuw (1992) para $n \leq 9$, junto con la aproximación $b_n = n/(n - 0,8)$ para $n > 9$; el refinamiento de Williams (2011), $b_n = n/(n - 0,801)$; la formulación asintótica de Hayes (2014), que depende de la paridad de n ; y la tabla exhaustiva de Park, Kim y Wang (2020) para $2 \leq n \leq 100$. En este trabajo, sin embargo, no se adopta directamente ninguna de esas fórmulas cerradas: se calibra un factor de corrección propio mediante simulación de Montecarlo, acotado al rango de tamaños muestrales que realmente interesa en este proyecto ($4 \leq n \leq 12$). Los factores resultantes reproducen, con discrepancias del orden de 10^{-3} , los valores exactos de la tabla de Park, Kim y Wang, y validan así la calibración; en la práctica, equivale a adoptar ese enfoque recortado al rango de trabajo. La Sección 4.3.1 detalla esta calibración y su aplicación concreta al detector de anomalías de la demanda eléctrica alemana.

3.6 Predicción autorregresiva a ciegas

Cuando el horizonte de predicción h supera el retardo disponible (en este caso, más de 24 horas), no queda otra opción que recurrir a una estrategia recursiva: los valores reales futuros de las características de retardo, que ya no existen, se sustituyen por las propias predicciones del modelo:

$$\begin{aligned}\hat{y}_{T+1} &= f(y_T, y_{T-23}, \mathbf{x}_{T+1}), \\ \hat{y}_{T+2} &= f(\hat{y}_{T+1}, y_{T-22}, \mathbf{x}_{T+2}), \\ &\vdots \\ \hat{y}_{T+h} &= f(\hat{y}_{T+h-1}, \hat{y}_{T+h-25}, \mathbf{x}_{T+h}) \quad \text{para } h > 24.\end{aligned}\tag{3.16}$$

El error de este esquema crece con h de forma no lineal: un error en $\hat{y}_{T+1}$ se propaga a $\hat{y}_{T+2}$ a través de la característica lag_1 , y acaba alcanzando a $\hat{y}_{T+25}$ por el canal de lag_{24} . El lector puede intuir a dónde lleva esto: a una semana vista, casi todo lo que el modelo «recuerda» de la demanda son invenciones suyas. El análisis empírico de esta propagación del error es, de hecho, uno de los experimentos centrales del presente trabajo. Conviene señalar, no obstante, que las variables exógenas conocidas de antemano (temperatura y calendario) actúan como anclas que limitan la deriva del modelo: aunque el retardo autorregresivo vaya alejándose de la realidad, la señal de temperatura y la hora del día siguen informando al modelo del nivel de demanda esperable, y esa información amortigua la divergencia a largo plazo.

3.7 Revisión de la literatura y posicionamiento metodológico

Adelantemos la conclusión: el estado del arte actual del STLTF se define por la tensión entre la potencia expresiva del aprendizaje profundo y la eficiencia e interpretabilidad del

gradient boosting, tal y como constata la revisión de [Ahmad et al. \(2025\)](#) en el contexto de las redes inteligentes. Todo el recorrido histórico desemboca ahí. La predicción de demanda eléctrica a corto plazo lleva más de cinco décadas siendo un problema abierto de investigación activa; los trabajos seminales de [Gross y Galiana \(1987\)](#) formalizaron el marco del STLF y sus condicionantes operativos. Durante los años ochenta y noventa dominaron los métodos estadísticos paramétricos (modelos ARIMA con componentes estacionales, suavizado exponencial de Holt-Winters), baratos de estimar y fáciles de auditar ([Hong y Fan, 2016](#)). A comienzos de los años 2000, la mayor disponibilidad de datos y de capacidad de cómputo dio protagonismo a las máquinas de vectores de soporte y a las primeras redes neuronales de retroalimentación, a las que siguieron los métodos de *ensemble* de árboles; la publicación de XGBoost ([Chen y Guestrin, 2016](#)) y LightGBM ([Ke et al., 2017](#)) convirtió al gradient boosting en el referente para datos tabulares hacia mediados de la década de 2010. La fase más reciente corresponde a las arquitecturas de aprendizaje profundo (LSTM, GRU, Transformer), que desde finales de esa década han ido ganando protagonismo en la predicción energética. En este apartado se revisan ambas corrientes y se justifica la elección de LightGBM en el contexto concreto de este trabajo.

3.7.1 Gradient boosting: el paradigma tabular dominante para STLF

La capacidad del gradient boosting para capturar relaciones no lineales complejas a partir de covariables heterogéneas (temperatura, retardos, calendario) lo convierte en una herramienta especialmente potente para STLF. [Kostas et al. \(2023\)](#) demuestran que LightGBM, apoyado en una ingeniería explícita de características (variables de calendario, retardos autorregresivos y jerarquías temporales), obtiene resultados competitivos en predicción de carga. En la misma línea, [He et al. \(2024\)](#) validan en un entorno multi-horizonte que añadir características meteorológicas y de calendario a LightGBM produce ganancias consistentes frente a *baselines* autorregresivos puros. A ello se suma la interpretabilidad nativa del modelo: la importancia de características puede extraerse directamente del *ensemble*, de modo que es posible auditar qué señales aportan mayor poder predictivo, una ventaja que [Lemos-Vinasco et al. \(2026\)](#) consideran decisiva en el sector regulado.

Esta competitividad no es anecdótica. En un *benchmark* sistemático sobre 45 conjuntos de datos tabulares, [Grinsztajn et al. \(2022\)](#) constatan que los modelos basados en árboles superan a las redes neuronales profundas en más del 80 % de los casos. Hay una razón estructural detrás: el aprendizaje profundo presupone regularidades suaves en el espacio de características, explotables mediante la propagación del gradiente en capas densas, y esas regularidades sencillamente no existen cuando las variables son de naturaleza radicalmente heterogénea. Un retardo temporal, una temperatura, un indicador binario de festivo y una codificación angular cíclica son señales sin estructura métrica compartida. Los árboles de decisión, en cambio, las tratan de forma nativa mediante particiones independientes por variable, sin normalización conjunta ni invarianzas geométricas de por medio.

3.7.2 Aprendizaje profundo: arquitecturas y comparación crítica

Las redes de memoria a largo y corto plazo (LSTM) marcan la primera incursión del aprendizaje profundo en el dominio temporal. Su mecanismo de celdas con puertas de entrada, olvido y salida les permite retener información selectiva a lo largo de secuencias arbitrariamente largas, y capturar así dependencias de la demanda a múltiples escalas sin necesidad de especificar retardos concretos. [Ullah et al. \(2024\)](#) revisan 47 trabajos de STLF con arquitecturas híbridas CNN-LSTM y constatan que combinar convoluciones (para extraer patrones locales) con LSTM (para las dependencias globales) da resultados competitivos en múltiples horizontes. La variante bidireccional (BiLSTM), que recorre la secuencia en ambos sentidos, y la red de unidades recurrentes cerradas (GRU), más simple y eficiente, son extensiones habituales en el ámbito energético. Con todo, las arquitecturas recurrentes arrastran limitaciones comunes: hay que especificar ventanas de entrada, son sensibles a la inicialización y su coste de entrenamiento pesa en ciclos de re-entrenamiento frecuentes.

Los modelos Transformer, contruidos sobre el mecanismo de atención multi-cabeza, permiten modelar dependencias temporales de largo alcance sin las servidumbres de la propagación secuencial. En predicción multi-horizonte, el *Temporal Fusion Transformer* (TFT) de [Lim et al. \(2021\)](#) es la arquitectura de referencia más citada: combina redes de selección de variables (*Variable Selection Networks*), codificadores LSTM locales y atención dispersa multi-cabeza para producir predicciones interpretables con cuantificación de incertidumbre por cuantiles. Frente a las LSTM puras, su principal aportación metodológica es la capacidad de tratar de forma diferenciada las covariables conocidas a futuro (como las variables de calendario, cuya evolución es completamente determinista) y las variables estocásticas, algo especialmente relevante para el STLF. [Mughees et al. \(2024\)](#) identifican al TFT y sus variantes como el estado del arte en predicción energética multi-horizonte para datasets de gran escala.

Pese a estos avances, la evidencia comparativa matiza la narrativa de superioridad automática del aprendizaje profundo. La propia revisión de [Mughees et al. \(2024\)](#) sobre STLF con aprendizaje profundo concluye que, para horizontes de hasta 48 horas y datasets de tamaño moderado (del orden de años de datos horarios), los métodos de gradient boosting son comparables o superiores en la mayoría de *benchmarks*, con menores costes de entrenamiento y mayor robustez frente al sobreajuste. [Gasparin et al. \(2022\)](#) llegan al mismo resultado en una comparación directa sobre datos reales de carga eléctrica: cuando el conjunto de características está bien construido, LightGBM y XGBoost rinden igual o mejor que las variantes de LSTM. [Ashraf et al. \(2024\)](#) lo confirman también en el ámbito residencial. La explicación teórica la aporta, de nuevo, el hallazgo de [Grinsztajn et al. \(2022\)](#) sobre datos tabulares heterogéneos: el espacio de características del STLF (una mezcla de retardos, temperaturas, indicadores cíclicos y binarios) es exactamente el tipo de problema en que los árboles conservan su ventaja estructural.

Existen también modelos híbridos que combinan gradient boosting y redes recurrentes para aprovechar lo mejor de ambos paradigmas. [Fan et al. \(2021\)](#) proponen una arquitectura

LightGBM + Attention-BiLSTM en la que el *boosting* hace de extractor de características y la red recurrente modela las dependencias residuales de largo alcance. El enfoque mejora algunos *benchmarks*, sí, pero a costa de una complejidad de entrenamiento y ajuste que merma la interpretabilidad y exige datasets mayores para no sobreajustar. En las condiciones del presente trabajo (re-entrenamiento semanal con una ventana de veinticuatro meses de datos horarios y un conjunto de características diseñado expresamente para capturar las principales fuentes de dependencia), el gradient boosting puro ofrece el mejor equilibrio entre capacidad predictiva, eficiencia computacional e interpretabilidad.

3.7.3 Modelos fundacionales de series temporales

La frontera más reciente del estado del arte (2023–2026) la ocupan los modelos fundacionales de series temporales (*Time Series Foundation Models*, TSFM): redes, en su mayoría basadas en Transformers, preentrenadas sobre corpus masivos y heterogéneos de series temporales y capaces de generar predicciones en modo *zero-shot*, es decir, sin entrenamiento específico para el dominio de destino. En esencia, trasladan al ámbito temporal el paradigma de los grandes modelos de lenguaje.

El primero en proponerse fue TimeGPT (Garza et al., 2024), un Transformer codificador-decodificador de acceso cerrado vía API, entrenado sobre más de cien mil millones de observaciones procedentes de dominios muy diversos (energía, *retail*, finanzas, IoT). Poco después llegó Chronos (Ansari et al., 2024), que adaptó la arquitectura de modelado de lenguaje al dominio temporal tokenizando los valores continuos mediante escalado y cuantización sobre un *backbone* tipo T5, con salida probabilística; en su evaluación sobre 42 conjuntos de datos supera a los métodos estadísticos locales (AutoARIMA, Seasonal Naive) e iguala o mejora a modelos profundos ajustados por conjunto como DeepAR, TFT y PatchTST. Su evolución, Chronos-2 (Ansari et al., 2025), añade soporte para covariables y predicción multivariante, y resuelve con ello una de las principales limitaciones de la versión original. En paralelo han surgido TimesFM (Das et al., 2024), un modelo decodificador desarrollado por Google Research; Moirai (Woo et al., 2024), un modelo codificador orientado a la predicción universal sobre frecuencias y tipos de variable heterogéneos; y Lag-Llama (Rasul et al., 2024), un modelo decodificador probabilístico que utiliza retardos como características de entrada.

Posicionamiento crítico respecto al presente trabajo. Pese a su notable capacidad de generalización la evidencia comparativa matiza la superioridad de estos modelos para el STLTF operativo. Hay para empezar un problema de covariables: varios de ellos, en su forma básica, son estrictamente univariantes (Chronos y Lag-Llama, entre otros) y no incorporan de forma nativa las variables exógenas conocidas a futuro, temperatura y calendario, que son justo las señales que más valor aportan en la predicción de demanda (Sección 3.5). Las evaluaciones independientes sobre datos reales tampoco resultan concluyentes: el rendimiento *zero-shot* es competitivo, pero no consistentemente superior al de modelos con ingeniería explícita de características cuando se dispone de señales exógenas fuertes (Ansari et al., 2024), en línea con el hallazgo general de Grinsztajn et al. (2022) sobre la ventaja estructural

de los árboles en datos tabulares heterogéneos. Queda por último el obstáculo práctico: su despliegue exige típicamente aceleración por GPU, frente al coste computacional reducido de LightGBM, y eso casa mal con el principio de bajo coste que vertebra este trabajo. Por todo ello, los modelos fundacionales se contemplan aquí como una línea de comparación futura de alto interés (Sección 5.2), más que como una alternativa que invalide el enfoque adoptado.

3.7.4 Estrategias de predicción multi-horizonte

Cuando el horizonte supera el retardo mínimo disponible, la literatura distingue tres estrategias: la predicción directa (un modelo independiente por horizonte), la recursiva (un único modelo aplicado iterativamente) y la híbrida MIMO (*Multiple Input Multiple Output*) (Fildes y Goodwin, 2024). Taieb y Hyndman (2014) demuestran empíricamente que, cuando las características autorregresivas presentan alta autocorrelación (como ocurre en la demanda eléctrica horaria), la estrategia recursiva con gradient boosting supera a la directa en la mayoría de *benchmarks*; ese resultado es el que fundamenta la adopción de la estrategia recursiva en el presente trabajo.

La evaluación de modelos multi-horizonte exige, además, protocolos de validación robustos. Petrović et al. (2026) muestran que la validación con origen fijo puede llegar a invertir el *ranking* de modelos respecto a la validación con origen móvil (*rolling-origin*); de ahí que el protocolo experimental de este trabajo adopte de forma explícita la evaluación *walk-forward* en lugar de una única partición temporal.

3.7.5 Posicionamiento y contribución del trabajo

El presente trabajo se separa de la literatura existente en tres frentes. Datos, primero: se emplean exclusivamente señales de acceso libre y bajo coste, sin datos de precios, generación ni contratos de balance. Riesgo, después: se cuantifica la propagación del error en predicción autorregresiva a ciegas para horizontes de hasta una semana (168 horas), un aspecto que la evaluación de modelos operativos suele pasar por alto. Y ámbito, por último: se demuestra la suficiencia de esta representación mínima en el contexto concreto del sistema eléctrico alemán, cuya demanda supera los 500 TWh anuales y está expuesta a perturbaciones exógenas significativas que afectan a los datos de entrenamiento, como las documentadas por Khalid et al. (2024) durante el periodo COVID-19.

Los datos de demanda y previsión oficial proceden de la plataforma SMARD de la Bundesnetzagentur (Bundesnetzagentur, 2026; SMARD, 2026), la fuente de referencia para el sistema eléctrico alemán. Las series meteorológicas se obtienen de la API Open-Meteo (Open-Meteo, 2026a,b): cualquiera puede, con estas dos fuentes, reproducir el experimento completo sin pagar un euro. Apostar por proveedores gratuitos no está exento de riesgos operativos (límites de peticiones, cuotas modificables sin previo aviso); su análisis detallado, con evidencia del propio sistema, se aborda en el capítulo de desarrollo (Sección 4.3.2).

Desarrollo del Proyecto y Resultados

4

4.1 Metodología

El planteamiento metodológico de este trabajo descansa sobre dos pilares. Por un lado, el modelo predictivo en sí: gradient boosting con LightGBM, una elección que debe menos a la moda del aprendizaje profundo que a la evidencia empírica disponible, que sitúa a los métodos de árboles por delante de las redes neuronales cuando se trabaja con datos tabulares heterogéneos ([Grinsztajn et al., 2022](#)). Por otro, una metodología de evaluación de *forecasting* pensada para reflejar las condiciones reales en las que el sistema tiene que funcionar. De hecho, toda la arquitectura del pipeline (ingesta continua de datos, re-entrenamiento periódico, evaluación a varios horizontes) está diseñada como la de un sistema en producción, no como la de un experimento de laboratorio que se ejecuta una vez y se archiva.

4.1.1 Visión general del pipeline

El pipeline de predicción se organiza en tres capas funcionales que trabajan de forma coordinada y sin intervención manual, gracias a un conjunto de tareas *cron*.

La primera capa se ocupa de la ingesta y consolidación de datos, y descansa sobre tres tareas *cron* independientes (`conf/crontab.txt`). El script `adquirir\_datos.py` se lanza en los minutos 10 y 40 de cada hora; el segundo disparo funciona como reintento, por si el dato oficial todavía no se hubiera publicado en el minuto 10. En cada ejecución descarga el consumo real desde la API pública de SMARD, consulta las temperaturas de las diez principales ciudades alemanas a través de Open-Meteo, calcula las variables cíclicas y de festividad, e inserta los registros ya procesados en la base de datos SQLite local (`demanda_energia.db`), que actúa como única fuente de verdad del sistema. A esta ingesta horaria se suma un proceso diario, a la 1:00 AM, que reconsolidamos los últimos siete días (`adquirir_datos.py --recent-days 7`): su papel es recoger las revisiones que el proveedor pueda publicar a posteriori, rellenar marcas temporales a las que les falte la demanda real o la previsión de SMARD, y completar los valores nulos (NULL/NaN) que hayan quedado pendientes. Por último, existe una tarea de autocuración profunda que corre todos los domingos y el día 1 de cada mes a las 2:00 AM: repasa hasta diez años de historial en busca de huecos en la demanda de SMARD (`--recent-days 3650`) y, a continuación, completa las temperaturas y previsiones pendientes mediante `-complete`. Un detalle práctico condiciona todo este di-

seño: las plataformas gratuitas como Open-Meteo imponen límites de tráfico moderados (el conocido error 429, *too many requests*), de modo que evitar llamadas innecesarias o repetidas y dejar márgenes de tiempo entre consultas no es una cortesía, sino un requisito para que el pipeline sobreviva.

La segunda capa es el pipeline de re-entrenamiento autónomo, orquestado por `pipeline.sh`. Cada domingo a las 3:00 AM se ejecuta `pipeline.sh -recent-days 730 -val-ratio 50 -train-file train/germany-24-meses.train`, que encadena en un solo comando todo el ciclo: descarga de los últimos 730 días (unos 24 meses), construcción del *dataset* con `construir_modelo.py` y partición cronológica en dos ficheros `.train/.valid` (CSV separados por comas) a través del parámetro `-val-ratio 50`. El 50 % más antiguo, doce meses, acaba en `germany-24-meses_train.train` y el 50 % más reciente, otros doce, en `germany-24-meses_valid.valid`. Hecho esto, el propio `pipeline.sh` lanza el entrenamiento de LightGBM en C++ pasándole ambos ficheros, de manera que `early_stopping_round` (Sección 4.1.3) actúa también en el ciclo de producción y no solo en los experimentos. El proceso termina actualizando los pesos del modelo en `LightGBM_model.txt` y recalculando las predicciones pendientes con `adquirir_datos.py -complete`, lo que en ocasiones introduce pequeños cambios en las curvas de previsión ya publicadas.

La tercera capa es la de explotación en tiempo real. Un servidor web Flask (`panel_control.py`) lee el histórico de SQLite y delega en `predecir_futuro.py`, que ejecuta la inferencia autorregresiva hora a hora sobre el horizonte solicitado (T+24 horas por defecto). El resultado viaja como JSON hasta el panel de control interactivo, donde se visualiza gráficamente. La Figura 4.1 resume el recorrido completo de los datos a través de las tres capas.

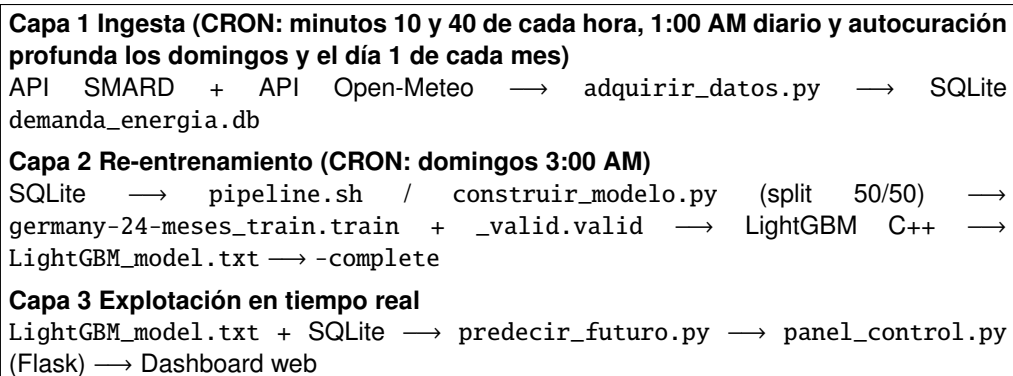


Figura 4.1: Diagrama de flujo del pipeline completo de predicción de la demanda eléctrica.

La calidad del conjunto se audita con `herramientas/prueba_sobreaajuste.py`, una utilidad que segmenta el historial, contrasta las curvas de pérdida de entrenamiento y validación, y detecta divergencias que apunten a sobreajuste o a infraajuste, sugiriendo en cada caso los ajustes necesarios para corregirlos.

4.1.2 Ingeniería de características

El paso central del preprocesamiento consiste en transformar la serie temporal en una matriz de características tabulares. A diferencia de una red LSTM, LightGBM no consume secuencias: cada instante t debe representarse como una fila autocontenida que reúna el pasado reciente (los *lags*), el contexto de calendario y las condiciones meteorológicas asociadas a ese instante.

La codificación cíclica resulta imprescindible para que el modelo no interprete como discontinuidades bruscas transiciones que en realidad son suaves: el paso de la hora 23 a la hora 0, o del 31 de diciembre al 1 de enero. Proyectar las variables temporales sobre el círculo unitario mediante funciones trigonométricas garantiza la continuidad matemática de la representación: $\text{hora_sin} = \sin\left(\frac{2\pi \cdot h}{24}\right)$, $\text{hora_cos} = \cos\left(\frac{2\pi \cdot h}{24}\right)$, y de forma análoga para el día del año, con período 365 o 366 cuando el año es bisiesto.

Las columnas `dia_semana` y `es_festivo` se declaran como variables categóricas dentro de LightGBM. Esto permite al algoritmo construir divisiones de histograma óptimas (*many-to-many splits*) sin recurrir a *One-Hot Encoding*, con una doble ventaja: se ahorra la memoria y el tiempo de cómputo que exigiría expandir el espacio de características en k columnas binarias dispersas, y de paso mejora la calidad de las divisiones del árbol.

En cuanto a la meteorología, la temperatura es el determinante físico principal de la demanda eléctrica: la relación entre ambas dibuja una curva en «U», con mayor consumo tanto en frío intenso (calefacción) como en calor extremo (refrigeración) (Valor et al., 2001). Incorporar diez zonas metropolitanas e industriales repartidas por todo el territorio permite capturar la heterogeneidad geográfica del consumo. Un apunte práctico: como LightGBM trabaja exclusivamente con umbrales de corte sobre histogramas, estas variables no necesitan ninguna transformación previa de escala.

4.1.3 Estrategia de validación

La evaluación opera en dos niveles distintos que conviene no mezclar. El primero es el de cada reentrenamiento: sobre una ventana de datos recientes se aplica una partición cronológica estricta, sin que la etapa de entrenamiento tenga acceso a datos futuros (los detalles se desarrollan en los apartados siguientes). El segundo es el de las métricas globales de las Tablas 4.6 y 4.7, que se calculan exclusivamente sobre el registro prospectivo de la validación operativa en vivo (Sección 4.4.3): las columnas `prediccion_1h` a `prediccion_168h` de la base de datos, en las que cada valor quedó escrito antes de que existiera la demanda real correspondiente, y que el anexo `exportacion_demanda_para_metricas.xlsx` recoge íntegramente.

Es importante no confundir estas predicciones con la columna `prediccion` de la base de datos de producción. Esa columna es otra cosa: el registro operativo acumulado de los sucesivos reentrenamientos semanales, en el que cada fila conserva la salida del modelo concreto que estaba en vigor la semana en que esa fila se procesó por primera vez. Es decir, una mezcla de modelos distintos cuya composición exacta la base de datos ni siquiera

Categoría	Variable	Descripción
Variable objetivo	demanda_target	Demanda eléctrica real en el instante t (MW)
Retardos	demanda_t_1	Demanda en $t-1$ (hora inmediatamente anterior)
autorregresivos	demanda_t_24h	Demanda en $t-24$ (misma hora del día anterior)
Codificación	hora_sin	$\sin(2\pi h/24)$, componente senoidal de la hora
cíclica (hora)	hora_cos	$\cos(2\pi h/24)$, componente cosenoidal de la hora
Codificación	dia_ano_sin	$\sin(2\pi d/365)$, estacionalidad anual (seno)
cíclica (año)	dia_ano_cos	$\cos(2\pi d/365)$, estacionalidad anual (coseno)
Variables de	dia_semana	Día de la semana (1–7, 1 = lunes); categórica
calendario	es_festivo	Festivo nacional: 0 o 1
Temperatura por ciudad (10 ciudades)	temp_berlin	Temperatura en Berlín (°C)
	temp_hamburgo	Temperatura en Hamburgo (°C)
	temp_munich	Temperatura en Múnich (°C)
	temp_colonia	Temperatura en Colonia (°C)
	temp_frankfurt	Temperatura en Fráncfort (°C)
	temp_stuttgart	Temperatura en Stuttgart (°C)
	temp_dusseldorf	Temperatura en Düsseldorf (°C)
	temp_leipzig	Temperatura en Leipzig (°C)
	temp_dortmund	Temperatura en Dortmund (°C)
	temp_essen	Temperatura en Essen (°C)

Tabla 4.1: Resumen completo de características del modelo.

permite verificar, porque no guarda ningún identificador de versión. El error semanal agregado (`error_absoluto`) ofrece al menos una pista indirecta: mantiene un patrón estable y de baja volatilidad hasta finales de 2022, y a partir de enero de 2023 pasa a mostrar saltos bruscos de una semana a otra, algo coherente con los aproximadamente 180 modelos distintos aplicados desde entonces, cada uno únicamente al tramo temporal que le tocó vivir y muchos de ellos anteriores a ajustes posteriores del pipeline. En cualquier caso, esa columna no interviene en el cálculo de la Tabla 4.6 ni de la Tabla 4.7. Las cifras reportadas corresponden a los reentrenamientos semanales del pipeline de julio de 2026 operando en condiciones reales (el del 5 de julio y, para el tramo final de la ventana, el actualmente desplegado del 12 de julio; Sección 4.1.3), y no a un modelo único e inmutable: las cadenas de predicción se lanzaron con las previsiones meteorológicas efectivamente disponibles en cada momento de emisión, no con observaciones ERA5 retrospectivas, y la fila con horizonte $h = 1$ conserva un carácter de *nowcasting* que se discute con calma en la Sección 4.4.2.

4.1.3.1 Ventana de entrenamiento

La primera versión del esquema de validación empleaba doce meses de entrenamiento seguidos de tres de validación. Con el uso quedó claro que este diseño arrastraba un sesgo estacional: validar solo sobre un trimestre concreto (por ejemplo, únicamente meses de invierno) acaba optimizando el modelo para esa estación y no para el ciclo anual completo. La corrección consistió en ampliar la ventana a veinticuatro meses, repartidos a partes iguales entre doce meses de entrenamiento y doce de validación. De este modo ambos bloques cubren un año natural entero y el diagnóstico de sobreajuste deja de depender de la estación sobre la que se evalúa.

4.1.3.2 Partición cronológica para la detección de sobreajuste

Dentro de esa ventana de veinticuatro meses, los datos se dividen en dos bloques cronológicos estrictos:

- Conjunto de entrenamiento: los doce meses más antiguos de la ventana.
- Conjunto de validación: los doce meses más recientes de la ventana.

Durante el entrenamiento se monitoriza el MSE (l_2) en ambos conjuntos, iteración a iteración, hasta un máximo de 2.000 rondas de *boosting*. Si la curva de validación deja de mejorar durante `early_stopping_round = 50` iteraciones consecutivas, LightGBM detiene el proceso por sí solo y conserva los pesos del árbol óptimo. El sobreajuste queda así contenido sin necesidad de fijar de antemano el número de árboles.

Este esquema encierra una disyuntiva que hay que hacer explícita: al reservar los doce meses más recientes para la validación, el modelo exportado nunca llega a entrenar con ellos. LightGBM ajusta los árboles exclusivamente sobre el fichero `.train` (los doce meses más antiguos de la ventana), de modo que el modelo desplegado en julio de 2026 se ajustó con datos que terminan, aproximadamente, en julio de 2025; el año más reciente interviene solo en la decisión de parada. Es el precio de corregir el sesgo estacional del esquema 80/20 anterior: un bloque de validación que cubre el ciclo anual completo se paga con datos de entrenamiento un año más viejos. La decisión se tomó con conocimiento de causa y descansa sobre dos atenuantes. El primero es estructural: el nivel reciente de la demanda entra en el modelo por las características, no por los umbrales: `demanda_t_1` concentra por sí sola el 76,43 % de la ganancia total (Sección 4.4.6), de manera que una deriva pura de nivel se propaga en gran medida a través del retardo en el momento de la inferencia, aunque los umbrales de partición sigan calibrados sobre los cuantiles del año anterior. El segundo es empírico: las métricas de validación en vivo (MAPE del 1,50 % a $h = 1$ y del 3,45 % a $h = 24$, Sección 4.4.3) incorporan ya cualquier coste de esta decisión, que queda así medido y no oculto. Aun con esos atenuantes, el esquema renuncia a la solución canónica del problema (emplear la validación únicamente para seleccionar el número de árboles y reentrenar después sobre la ventana completa), una renuncia que se registra como limitación

explícita (Sección 4.4.13) y cuya corrección se propone como mejora inmediata del pipeline (Sección 5.2).

Esta ventana ampliada es la que se ha usado para entrenar el modelo final (`modelos/LightGBM\_model.txt`) y la que respalda las métricas de este capítulo. Coincide, además, con la que emplea la tarea automática de *cron* que reentrena el modelo cada semana en producción (`conf/crontab.txt`): tras una actualización del pipeline el 29 de junio de 2026, el ciclo semanal pasó de una ventana móvil de 456 días (`-recent-days 456`, unos quince meses) con partición cronológica en torno al 80/20 (`-val-ratio 20`, doce meses de entrenamiento y tres de validación) a una de 730 días (`-recent-days 730`, unos 24 meses) con partición cronológica 50/50 (`-val-ratio 50`) a través de `pipeline.sh`, replicando en producción el mismo esquema de doce más doce meses del apartado anterior. El modelo generado ese mismo 29 de junio de 2026 (1 370 árboles) ya respondía a esta configuración de 730 días; hay que ser honestos con un detalle, no obstante: aquel entrenamiento se lanzó a mano y no a través del mecanismo de *cron*, por lo que no aparece en `logs/cron\_entrenamiento.log`, y las cifras de convergencia citadas en la Sección 4.1.4.4 para ese log corresponden todavía a la configuración intermedia previa a este cambio. A comienzos de julio de 2026, un problema técnico en la máquina de producción obligó a reentrenar el modelo de nuevo (sin pérdida alguna de datos, pues la base de datos conservaba el historial completo), manteniendo esta misma configuración de 730 días y partición 50/50. El ciclo semanal de producción, sin embargo, no se interrumpió por el incidente: `logs/cron\_entrenamiento.log` registra dos reentrenamientos completos bajo esta configuración dentro de la propia ventana de validación en vivo del 4 al 12 de julio de 2026 (Sección 4.4.3): el del domingo 5 de julio (parada temprana en la iteración 1.627, árbol óptimo en la 1.577) y el del domingo 12 de julio (parada temprana en la iteración 1.565, árbol óptimo en la 1.515). Como el pipeline no versiona los modelos (cada ciclo sobrescribe `modelos/LightGBM_model.txt` sin conservar copia de la versión anterior), el modelo del 5 de julio estuvo desplegado, y por tanto generando las predicciones que se escriben en el registro en vivo, desde ese domingo hasta la mañana del 12 de julio; el modelo actualmente desplegado (el del 12 de julio, cuya curva de convergencia se muestra en la Figura 4.2) es responsable únicamente del tramo final de la ventana. Las métricas de las Tablas 4.6 y 4.7 combinan, por tanto, las salidas de dos reentrenamientos semanales consecutivos del mismo pipeline, con idéntica configuración de hiperparámetros (Tabla C.1) y ventanas de entrenamiento casi enteramente solapadas entre sí. Esto no introduce contaminación dentro de muestra en ningún caso (cada predicción se escribe antes de conocerse la demanda real correspondiente, sea cual sea el modelo semanal que la generó), pero seamos precisos: a la ventana la sirve el pipeline operando de forma continua, tal como está diseñado para hacerlo en producción, no un modelo único e inmutable. La Sección 4.4.13 recoge esta circunstancia como limitación explícita.

La divergencia entre ambas curvas es lo que permite diagnosticar el estado del modelo: si el error de validación toca un mínimo y luego repunta mientras el de entrenamiento sigue bajando, el modelo está memorizando el ruido estadístico del conjunto de entrenamiento. Cuando esto ocurre, se accionan las palancas de regularización que se describen en la sec-

ción siguiente.

4.1.3.3 Horizonte de predicción y evaluación de deriva

En producción, el modelo genera previsiones a un horizonte de $T+24$ horas mediante predicción autorregresiva: la estimación de cada hora t se retroalimenta como variable `demanda_t_1` para predecir $t + 1$, y el proceso se repite a lo largo de todo el horizonte. Para cuantificar la degradación que introduce este encadenamiento (*drift*), `predecir_tramo.py` simula el proceso sobre tramos consecutivos del fichero de entrenamiento y calcula el MAE, el error máximo y la correlación de Pearson entre la serie predicha y la real, manteniéndose en todo momento desconectado de los valores reales intermedios.

4.1.4 Configuración del modelo LightGBM

La configuración del modelo vive en el fichero `train.conf` y se articula en los bloques que siguen.

4.1.4.1 Función de pérdida

Se emplea `objective = regression_l2`, esto es, el Error Cuadrático Medio (MSE). La elección frente al MAE (`regression_l1`) responde al contexto operativo: en una red eléctrica, lo prioritario es garantizar el equilibrio en tiempo real entre generación y consumo para evitar apagones. Un error de predicción masivo obligaría a activar de golpe centrales de respaldo muy costosas, con un impacto económico y operativo desproporcionado en comparación con el de un error moderado. Como el MSE penaliza cuadráticamente las desviaciones grandes, empuja al modelo a atacar en primer lugar precisamente esos errores críticos.

4.1.4.2 Complejidad del árbol

El principal control de complejidad en LightGBM es `num_leaves`, que fija el número máximo de hojas por árbol: un valor alto permite capturar patrones más finos, pero a cambio dispara el riesgo de sobreajuste. La práctica habitual consiste en mantenerlo por debajo de $2^{\text{max_depth}}$ para no acabar con árboles degenerados. A su vez, `max_depth` acota la profundidad máxima y frena el crecimiento excesivo en estructuras asimétricas, algo característico del modo *leaf-wise*. Por último, `min_data_in_leaf` impone un número mínimo de muestras en cada hoja terminal: subirlo (500–1 000) impide que el modelo memorice micropatrones espurios cuando las curvas de validación dan señales de sobreajuste, mientras que bajarlo permite afinar más a costa de ese mismo riesgo. El valor finalmente seleccionado y en uso en producción (`conf/train.conf`) es `min_data_in_leaf = 100`, el extremo inferior de la rejilla explorada (Tabla 4.2). La decisión se apoya en dos hechos: el volumen horario del conjunto de entrenamiento es lo bastante grande, y el diagnóstico con `herramientas/prueba_sobreajuste.py` no mostró divergencia relevante entre las curvas

de pérdida de entrenamiento y validación. Con esas condiciones, 100 fue el valor que minimizó el MSE de validación sin activar el criterio de sobreajuste que se describe más adelante en esta sección.

4.1.4.3 Tasa de aprendizaje y submuestreo

La `learning_rate` es el factor de encogimiento (*shrinkage*) que escala la contribución de cada árbol nuevo al *ensemble*: cuanto más baja, más robusto es el modelo resultante, pero más iteraciones necesita para converger. `feature_fraction` restringe la fracción de columnas visibles para cada árbol; con un valor de 0,8, el modelo no puede apoyarse exclusivamente en el retardo $t-1$ en cada partición, lo que le fuerza a explorar también las señales climáticas y cíclicas y, de paso, incrementa la robustez general del conjunto. Por su parte, `bagging_fraction` submuestra aleatoriamente una fracción de las filas en cada iteración, añadiendo diversidad y reduciendo la varianza entre árboles.

4.1.4.4 Parada temprana

`early_stopping_round = 50`: si el error del conjunto de validación no mejora durante 50 iteraciones consecutivas, LightGBM detiene el entrenamiento por sí solo, conserva los pesos del árbol óptimo y descarta los árboles sobreajustados posteriores. Con la configuración de hiperparámetros vigente (`num_leaves = 63`, `min_data_in_leaf = 100`), un ciclo de producción registrado en `logs/cron/_entrenamiento.log` (28 de junio de 2026), ejecutado aún bajo la configuración intermedia del pipeline entonces vigente (`-recent-days 456 -val-ratio 20`, previa a la actualización del 29 de junio descrita en la Sección 4.1.3), detuvo el entrenamiento en la iteración 655, con el árbol óptimo en la 605. La cifra concreta puede variar de una semana a otra según la ventana de datos de cada reentrenamiento. El propio `logs/cron/_entrenamiento.log` registra además dos ciclos bajo la configuración de 730 días/24 meses actualmente en vigor: el del domingo 5 de julio de 2026 (parada temprana en la iteración 1.627, árbol óptimo en la 1.577) y el del domingo 12 de julio de 2026, que generó el modelo desplegado (parada temprana en la iteración 1.565, árbol óptimo en la 1.515, tal como muestra la Figura 4.2); el primero estuvo en producción durante la mayor parte de la ventana de validación en vivo, como se explica en la Sección 4.1.3. Lo relevante del mecanismo es doble: elimina la necesidad de fijar a mano el número total de árboles, y garantiza que el modelo exportado (`LightGBM_model.txt`) corresponde al punto de mínimo error de generalización —siempre, claro está, que el script de entrenamiento reciba también el fichero de validación (Sección 4.1.3). Tanto el entrenamiento de evaluación de este capítulo (ventana de veinticuatro meses) como el ciclo semanal de producción actual (`conf/crontab.txt`, vía `pipeline.sh`) pasan ese fichero, de modo que la parada temprana está activa en ambos casos.

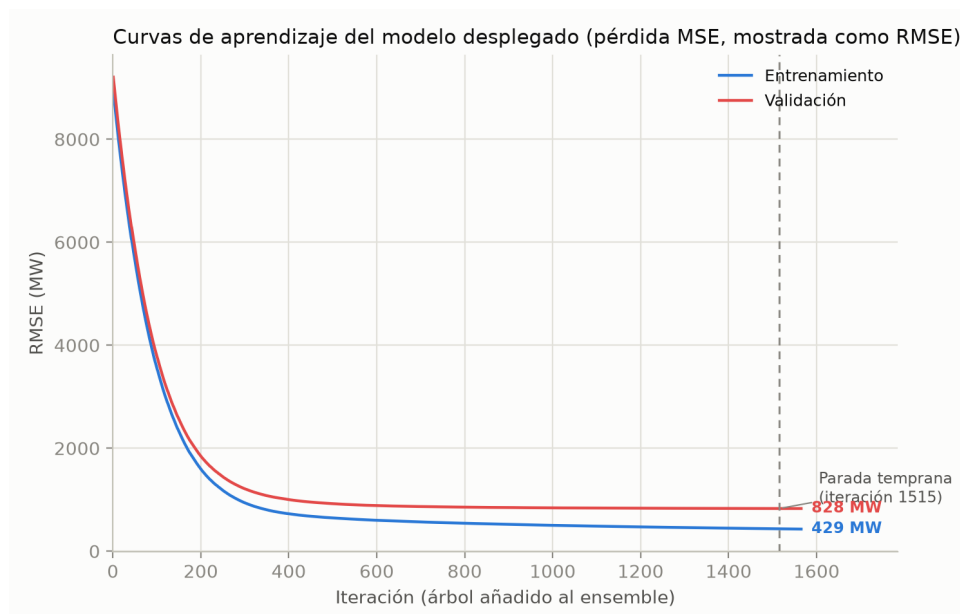


Figura 4.2: Curvas de aprendizaje (RMSE de entrenamiento y de validación, $\sqrt{MSE}$) del ciclo de reentrenamiento del 12 de julio de 2026 que generó el modelo actualmente desplegado, extraídas de `logs/cron/_entrenamiento.log`. La parada temprana detiene el proceso en la iteración 1.565 tras 50 iteraciones sin mejora del error de validación, y conserva el árbol óptimo de la iteración 1.515 como modelo exportado.

4.1.5 Optimización de hiperparámetros

El ajuste de hiperparámetros combina dos mecanismos complementarios: una búsqueda en rejilla manual (*manual grid search*) sobre los parámetros críticos de regularización, y la parada temprana automática de LightGBM para determinar el número óptimo de iteraciones. El enfoque es, por así decirlo, bidireccional: la complejidad estructural del árbol se ajusta de forma explícita, mientras que el número de árboles se optimiza implícitamente y sin coste computacional adicional.

El diagnóstico principal procede de `herramientas/prueba/_sobreajuste.py`, que se para cronológicamente entrenamiento y validación, ejecuta hasta 2.000 iteraciones con evaluación paralela y genera el log de convergencia. El análisis de esas curvas de pérdida es lo que va guiando los ajustes iterativos del espacio de búsqueda.

Parámetro	Rango explorado	Criterio de selección
<code>num_leaves</code>	31, 63, 127, 255	Mínimo MSE de validación
<code>max_depth</code>	-1 (sin límite), 6, 8, 10	Mínimo MSE de validación
<code>min_data_in_leaf</code>	100, 200, 500, 800, 1000	Mínimo MSE de validación
<code>learning_rate</code>	0,01; 0,05; 0,1	Compromiso velocidad/calidad
<code>feature_fraction</code>	0,7; 0,8; 0,9; 1,0	Mínimo MSE de validación
<code>bagging_fraction</code>	0,7; 0,8; 0,9; 1,0	Mínimo MSE de validación

Tabla 4.2: Espacio de búsqueda de hiperparámetros.

Cada combinación de hiperparámetros se entrena con la partición cronológica descrita antes (doce meses de entrenamiento, doce de validación). La métrica de referencia es el MSE del conjunto de validación en el árbol óptimo que determina la parada temprana, y se selecciona la combinación que lo minimiza siempre que la diferencia entre las curvas de pérdida de entrenamiento y validación no supere un umbral de divergencia indicativo de sobreajuste activo.

4.2 Planteamiento del Problema

4.2.1 Modelo de referencia (baseline): naive estacional

Para poder medir la ganancia real que aporta el modelo LightGBM hace falta un punto de comparación honesto, y ese papel lo cumple un modelo de referencia *naive* estacional (*seasonal naive baseline*). Este modelo no aprende absolutamente nada: su predicción para el instante t es, sin más, el valor observado en el mismo instante de un ciclo anterior, sin ajuste alguno por las condiciones actuales.

4.2.1.1 Variantes del baseline

Se consideran dos variantes:

- *Naive* diario (lag-24h): la predicción para la hora t es la demanda registrada exactamente 24 horas antes:

$$\hat{y}_t = y_{t-24}$$

Captura la estacionalidad diaria del consumo, pero es incapaz de distinguir si el día anterior fue excepcional (una temperatura extrema, un festivo imprevisto).

- *Naive* semanal (lag-168h): la predicción para la hora t es la demanda del mismo día y hora de la semana anterior:

$$\hat{y}_t = y_{t-168}$$

Incorpora la dimensión semanal (laborable frente a fin de semana), aunque a cambio arrastra cualquier anomalía que hubiera ocurrido siete días atrás.

4.2.1.2 Justificación del baseline

El *naive* estacional es el punto de referencia estándar en la literatura de predicción de series temporales energéticas, y por una buena razón: un modelo de *machine learning* que no supere consistentemente este umbral no aporta valor operativo alguno, por sofisticado que sea computacionalmente. La mejora relativa sobre el MAE y el MSE del *baseline* es, en consecuencia, la métrica primaria de éxito del sistema.

En el caso concreto de este proyecto, el *naive* semanal (lag-168h) es un rival especialmente exigente, porque la demanda eléctrica exhibe una periodicidad semanal muy fuerte.

Superarlo de forma consistente solo es posible incorporando de verdad información meteorológica, de festivos y de tendencias recientes, señales de las que el *baseline* no dispone.

4.3 Desarrollo del Proyecto

4.3.1 Demanda eléctrica alemana SMARD

La variable objetivo del modelo es la demanda eléctrica horaria de Alemania, que se obtiene a través de la API pública de la plataforma SMARD (*Strommarktdaten*), gestionada por la Agencia Federal de Redes alemana (*Bundesnetzagentur*). La plataforma agrega y publica los datos de todos los gestores de red de transporte (TSOs) del sistema eléctrico alemán, y tiene una ventaja nada menor para un proyecto de este tipo: no exige autenticación ni registro.

Endpoint y estructura de consulta

La API de SMARD organiza el histórico en bloques semanales, identificados por marcas temporales en milisegundos UTC. La integración está implementada en la función `descargar\_smard()` de `adquirir_datos.py` y sigue un esquema en dos pasos: primero se consulta un índice que devuelve la lista de bloques disponibles, y después se descarga individualmente cada bloque de interés.

- Endpoint de índice (`index\_url`): devuelve la lista de bloques semanales disponibles como marcas de tiempo en milisegundos UTC.

`https://www.smard.de/app/chart_data/{filtro}/{region}/index_{resolution}.json`

Devuelve un objeto JSON con la clave `timestamps`, que contiene la lista de marcas temporales en milisegundos (UTC) correspondientes al inicio de cada semana disponible.

- Endpoint de datos (`data_url`): descarga el bloque semanal correspondiente a una marca de tiempo concreta.

`https://www.smard.de/app/chart_data/{filtro}/{region}/
{filtro}_{region}_{resolution}_{ts}.json`

Devuelve un objeto JSON con la clave `series`, una lista de pares `[timestamp_ms, valor_MW]`, uno por hora del bloque semanal; la Figura 4.5 muestra un ejemplo de respuesta para el indicador 410.

Los parámetros fijos de consulta son `region = 'DE'` (Alemania completa) y `resolution = 'hour'` (granularidad horaria). El script descarga dos indicadores a la vez:

- Indicador 410 Consumo real (*Stromverbrauch: Tatsächlicher Last*): el valor medido y consolidado de la demanda eléctrica.
- Indicador 411 Previsión oficial (*Stromverbrauch: Prognostizierte Last*): la estimación que publican los operadores de los sistemas de transmisión antes del cierre del mercado diario, y que este trabajo emplea como línea de referencia competidora en la evaluación del modelo.

La respuesta JSON del *endpoint* de índice contiene un array con 597 marcas de tiempo (una por semana, desde 2015 hasta la semana actual), como muestra la Figura 4.3. El pipeline conserva estas marcas tal cual, en milisegundos UTC, que constituyen la clave canónica de almacenamiento; la conversión al huso horario local de Berlín (Europe/Berlin) solo se aplica en memoria y de forma puntual, cuando hace falta derivar variables que dependen de la hora civil alemana (festivos, codificación cíclica de la hora), incluyendo los cambios estacionales entre CET (UTC+1) y CEST (UTC+2).

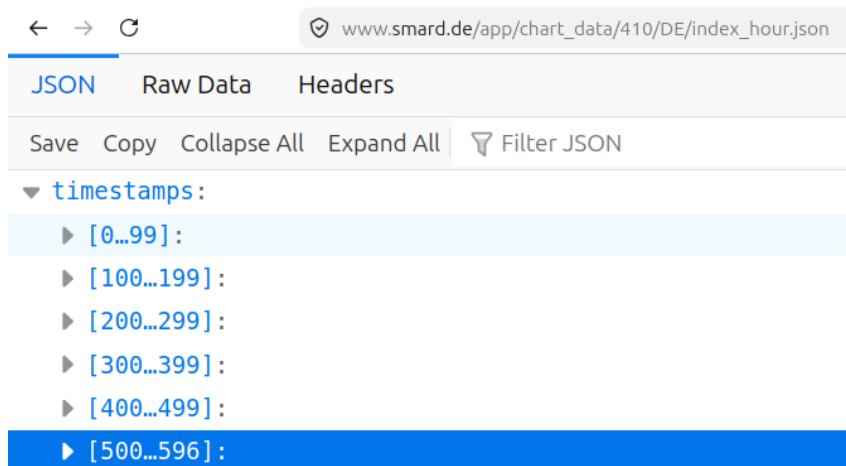


Figura 4.3: Respuesta del *endpoint* de índice (*index_hour.json*) de la API de SMARD. La clave *timestamps* contiene 597 bloques semanales (posiciones 0–596), cada uno identificado por su marca de tiempo de inicio en milisegundos UTC.

Lógica de paginación por bloques semanales

El histórico de SMARD está particionado en ficheros JSON semanales, de unas 168 entradas cada uno. El script convierte las fechas solicitadas a milisegundos UTC, recorre el índice completo y se queda únicamente con los bloques que solapan con el intervalo de interés. Como medida de seguridad, incluye siempre además el bloque semanal inmediatamente anterior al primero identificado: así se absorben los posibles desplazamientos provocados por el horario de verano (Europe/Berlin frente a UTC).

Raw Data	Headers
Copy	Collapse All Expand All Filter JSON
584:	17730108000000
585:	17736156000000
586:	17742204000000
587:	17748216000000
588:	17754264000000
589:	17760312000000
590:	17766360000000
591:	17772408000000
592:	17778456000000
593:	17784504000000
594:	17790552000000
595:	17796600000000
596:	17802648000000

Figura 4.4: Últimas entradas del índice de SMARD (posiciones 584–596). El bloque más reciente (17802648000000) corresponde al domingo 31 de mayo de 2026 a las 22:00 UTC, es decir, al lunes 1 de junio a las 00:00 en hora civil de Berlín (CEST, UTC+2): los bloques semanales del índice están anclados a la medianoche local alemana, no a la UTC, en coherencia con la conversión de husos descrita en el texto. La marca confirma que el historial disponible supera los 11 años.

Cobertura temporal y resolución

Como muestra la Figura 4.4, el índice de SMARD expone más de 11 años de bloques semanales (desde 2015), y la base de datos de producción de este trabajo ha consolidado en la práctica observaciones horarias completas desde el 29 de junio de 2016 hasta la actualidad: el volcado íntegro exportado el 12 de julio de 2026 (*exportacion_demanda.xlsx*, Sección 4.3.5) contiene 88.111 registros que cubren 3.673 días consecutivos (con cobertura horaria prácticamente completa, salvo un puñado de huecos puntuales de mediados de julio de 2026 detallados en la Sección 4.3.5), incluidas las marcas futuras hasta el 19 de julio de 2026 que solo contienen predicciones. Ahora bien, este trabajo restringe deliberadamente el uso de ese histórico (Sección 1.2, “Alcance y limitaciones”): el entrenamiento del modelo emplea una ventana móvil de los últimos 730 días (unos 24 meses), y todas las métricas reportadas en las Tablas 4.6 y 4.7 se calculan exclusivamente sobre la ventana de validación operativa en vivo del 4 al 12 de julio de 2026 (Sección 4.4.3), recogida en el anexo

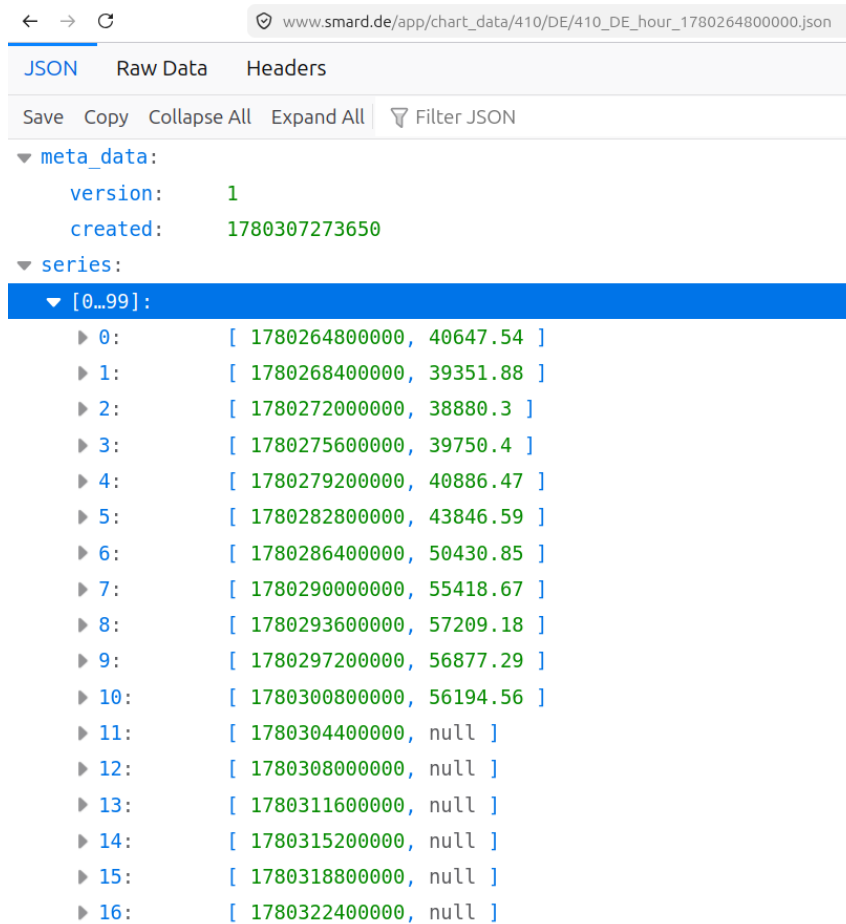


Figura 4.5: Ejemplo de respuesta del endpoint de datos de SMARD para el indicador 410 (consumo real). Cada entrada de series contiene el par `[timestamp_ms, potencia_MW]`. Los valores null corresponden a horas futuras aún no publicadas.

`exportacion_demanda_para_metricas.xlsx`. Insistamos: se trata de una acotación de alcance, no de una limitación de disponibilidad de datos: el histórico anterior a esa ventana móvil de entrenamiento (desde el 29 de junio de 2016 hasta aproximadamente mediados de 2024) existe en la base de datos de producción, pero no se emplea en este trabajo y queda registrado como línea de trabajo futura. La cobertura efectiva puede variar ligeramente en los extremos del período, en función de qué bloques semanales estén disponibles en la API.

Cada registro corresponde a una hora completa de producción/consumo. Las marcas temporales se almacenan en SQLite en formato `YYYY-MM-DD HH:MM`, en UTC: la propia definición de la columna `marca_temporal` en el esquema documenta la convención (*“almacenada en UTC; evita colisiones por DST”*). Es una decisión que se aparta deliberadamente de la hora civil de los mercados energéticos europeos (CET/CEST), y lo hace precisamente para esquivar la ambigüedad que el horario de verano introduce en la clave primaria de la base de datos.

El motivo es fácil de ver. El cambio de horario de verano genera dos horas singulares al año en el huso civil de Berlín: en el paso a CEST (marzo) desaparece la hora local de las

2:00, y en el retorno a CET (octubre) esa misma hora local aparece duplicada. Almacenar `marca_temporal` directamente en hora local de Berlín produciría, por tanto, huecos y colisiones en la clave primaria dos veces al año. Para evitarlo, el pipeline adopta UTC como escala horaria canónica de almacenamiento (continua, sin discontinuidades por DST) y solo recurre a la conversión a `Europe/Berlin`, mediante `pandas/tz`, de forma puntual y en memoria, cuando toca calcular variables de calendario en hora civil (festivos, codificación cíclica de la hora del día).

Limpieza y control de calidad

Una vez descargados y combinados los DataFrames de los indicadores 410 y 411, el script aplica tres pasos de limpieza:

1. Deduplicación: cuando dos bloques semanales consecutivos se solapan, conserva el último valor con `df[~df.index.duplicated(keep='last')]`.
2. Reordenación cronológica: el DataFrame se ordena de forma ascendente por marca temporal, requisito para que los cálculos de retardos sean consistentes.
3. UPSERT idempotente: la inserción en SQLite emplea `INSERT\, \dots\, ONCONFLICT(marca\_temporal)\, DO\, UPDATE`, de modo que los datos provisionales se sobrescriben con los definitivos sin generar filas duplicadas.

Ninguno de estos pasos interpola ni rellena huecos: los valores ausentes se conservan como NaN en esta fase. Y así siguen, sin corrección posterior, en el caso del indicador 411 (`demanda_prevision`), que no cuenta con detector de anomalías propio ni con la jerarquía de imputación que se describe a continuación (su ausencia se registra como NULL en la base de datos y se vigila mediante un contador dedicado en el resumen del pipeline). El indicador 410 (`demanda_real`), en cambio, sí recibe el tratamiento completo de detección y corrección que se detalla en el siguiente apartado.

Detección y corrección de anomalías

La primera versión del detector de anomalías se basaba en un umbral estático sobre el salto horario (`MAX_SALTO_HORARIO_MW`): cada lectura se comparaba únicamente contra su valor en $t-1$. En la práctica, este diseño arrastraba dos fallos estructurales. El primero: si el dato corrupto era precisamente el valor en $t-1$, el umbral quedaba «envenenado» y dejaba de detectar el salto en las horas siguientes, propagando el error en cadena, en una especie de efecto dominó. El segundo: al descartar el registro anómalo sin sustituirlo por nada, el campo se quedaba en NULL, y eso rompía la cadena autorregresiva de retardos (`demanda_t_1`, `demanda_t_24h`) de la que dependen tanto el entrenamiento como la inferencia recursiva. Un caso real ilustra bien el problema: el 25 de junio, SMARD llegó a registrar un valor de demanda de 1,1 billones de MW. Con un umbral fijo mal calibrado y sin mecanismo de imputación,

ese error habría dejado un hueco irrecuperable justo donde más falta hace la continuidad de la serie.

El detector actual abandona el umbral estático y lo sustituye por un criterio estadístico robusto, basado en la mediana y la desviación absoluta mediana (*Median Absolute Deviation*, MAD) calculadas de forma **estacional**: en lugar de comparar cada lectura contra sus vecinas inmediatas, se compara contra el mismo instante del día en las últimas cuatro semanas, tomando de cada semana el propio punto y sus dos horas adyacentes ($t-1$, t , $t+1$). Esto proporciona hasta 12 muestras de referencia por observación. Formalmente, dado un conjunto de muestras históricas $\{x_i\}_{i=1}^n$ ($n \leq 12$):

$$\text{MAD}_{\text{crudo}} = \text{mediana}_i(|x_i - \text{mediana}(x)|), \quad z = \frac{|x_{\text{obs}} - \text{mediana}(x)|}{c(n) \cdot \text{MAD}_{\text{crudo}}}$$

y se marca como anómalo todo registro con $z > 3$, siempre que haya al menos 4 muestras históricas disponibles. La comparación estacional (en lugar de una ventana móvil de las últimas 24 h) resuelve además el que podríamos llamar «efecto lunes»: la subida perfectamente legítima de la demanda el lunes por la mañana, comparada contra la calma del domingo anterior, generaba falsos positivos con un criterio de ventana corta; comparar contra el mismo instante de semanas previas elimina ese sesgo de calendario de raíz.

Queda un caso límite: cuando la dispersión estacional es nula (por ejemplo, si las muestras de referencia coinciden exactamente), el denominador se anularía. Para evitar divisiones por cero, la implementación vectorizada impone un suelo mínimo al MAD escalado: si $c(n) \cdot \text{MAD}_{\text{crudo}}$ es cero, se sustituye por 1 000 MW antes de calcular z . El suelo elimina las indeterminaciones numéricas a costa de una sensibilidad ligeramente menor del detector en esos casos, por lo demás infrecuentes, de dispersión estacional genuinamente nula.

Merece la pena detenerse en el factor de escala $c(n)$, que convierte el MAD crudo en un estimador de la desviación típica y que solo es exacto de forma asintótica: para muestras normales con $n \rightarrow \infty$, $c(n) \rightarrow 1/\Phi^{-1}(0,75) \approx 1,4826$, ya que el MAD converge al percentil 75 de la distribución de $|Z|$. Con n finito y pequeño, los 4 a 12 puntos que se manejan aquí, la mediana de las desviaciones absolutas subestima sistemáticamente la dispersión real, así que aplicar el factor asintótico inflaría artificialmente el estadístico z y elevaría el riesgo de falsos positivos justo cuando menos muestras hay disponibles. La corrección adoptada consiste en calibrar el factor por simulación de Montecarlo (8 millones de repeticiones por tamaño de muestra, sobre una normal estándar) y aplicarlo de forma dinámica según el número real de muestras recuperadas en cada caso, en lugar de asumir siempre $n = 12$ (Akinshin, 2021):

n	4	5	6	7	8	9	10	11	12
$c(n)$	2,018	1,804	1,764	1,687	1,671	1,633	1,625	1,601	1,596

Tabla 4.3: Factor de corrección MAD $\rightarrow$ σ calibrado por simulación, según el número de muestras estacionales disponibles.

Los valores calibrados admiten además una validación externa directa: coinciden, con discrepancias inferiores a 10^{-3} atribuibles al ruido de la propia simulación, con los factores

exactos C_n tabulados por Park, Kim y Wang (2020) que recoge Akinshin (2021) (por ejemplo, 2,0178 frente a 2,0167 para $n = 4$, o 1,5960 frente a 1,5962 para $n = 12$). La Tabla 4.3 equivale, en la práctica, a la tabla del enfoque Park-Kim-Wang recortada al rango $4 \leq n \leq 12$, que es exactamente el que puede producir el detector estacional: 4 semanas $\times$ 3 desplazamientos horarios dan como máximo 12 muestras, y los huecos de la serie pueden reducir ese número.

En el código, la tabla se materializa como un diccionario de constantes (FACTORES_MAD_ESTACIONAL, con claves $n = 4, \dots, 12$) consultado a través de una función de *lookup* (`factor_mad_estacional(n)`) que acota el argumento al intervalo válido $[4, 12]$ antes del acceso, de modo que ningún tamaño muestral fuera de rango pueda provocar un fallo. Este criterio se aplica en dos puntos del pipeline, con la misma lógica ($MAD_n = c(n) \cdot MAD_{\text{crudo}}$) pero implementaciones distintas. Durante la consolidación histórica (`descargar_smard()`) el cálculo es vectorizado sobre todo el DataFrame: las hasta 12 muestras de referencia de cada observación se disponen como columnas desplazadas, se calcula el MAD crudo por filas (con $C = 1$), se cuenta el número efectivo de muestras válidas de cada fila (`notna().sum(axis=1)`) y se multiplica cada MAD crudo por el factor $c(n)$ correspondiente a su n concreto, mapeado fila a fila. El estadístico z resultante se evalúa contra el umbral 3 solo donde hay al menos 4 muestras válidas. Durante la ingesta horaria en modo cron (`obtener_demandas_reales()`), en cambio, la misma operación se realiza de forma escalar sobre la última lectura publicada por SMARD: mediana y MAD crudo de los valores recuperados de las cuatro semanas previas, factor según el recuento real de esas muestras, y comparación del cociente robusto con el mismo umbral.

Cuando un registro se marca como anómalo, el diseño busca no dejarlo nunca en NULL, aplicando una jerarquía de imputación en cascada que prioriza la mejor fuente disponible en cada caso: (1) la previsión oficial de SMARD (`demanda_prevision`) para ese mismo instante; (2) si no está disponible, el valor real de la semana anterior a la misma hora ($t-168$); (3) si tampoco existe, la mediana histórica agrupada por día de la semana y hora; y (4) como último recurso (relevante solo en el arranque de la serie, donde no hay historial suficiente para las etapas anteriores) relleno hacia adelante y hacia atrás (`ffill/bfill`). Una precisión: esta cascada completa de cuatro pasos solo está implementada tal cual en la ruta de consolidación histórica (`descargar_smard()`); la ruta de ingesta horaria en tiempo real (`obtener_demandas_reales()`) replica los pasos (1) y (2), pero el paso (3) queda inoperante por una referencia a una variable no definida en esa función, y no implementa el paso (4). En la práctica, si un registro anómalo detectado en tiempo real no encuentra previsión SMARD ni valor de $t-168h$ disponibles en ese instante, puede quedar temporalmente sin imputar. El registro se marca en todo caso como no consolidado (`smard_consolidado = 0`), de modo que la siguiente pasada de autocuración (que sí ejecuta la cascada completa de cuatro pasos) lo revisa y lo corrige; el efecto neto es que, para esos casos concretos, la imputación completa puede demorarse hasta la siguiente consolidación en lugar de aplicarse en el mismo instante de la detección. Esta discrepancia entre las dos rutas de código se identifica como corrección pendiente en la Sección 4.4.13.

Falta un último eslabón: garantizar la integridad de los bloques semanales descargados. Para ello existe un control de completitud: cada bloque de 168 horas solo se da por definitivo

cuando la base de datos local contiene 168 filas marcadas como `smard_consolidado = 1` en ese rango; si no es así, `descargar_smard()` vuelve a solicitar el bloque en la siguiente ejecución. Este mecanismo, combinado con la operación UPSERT sobre la clave primaria `marca_temporal`, permite reconciliar de forma idempotente tanto los huecos de descarga como los registros reabiertos por el detector de anomalías, sin duplicar filas ni requerir intervención manual.

4.3.2 Datos meteorológicos Open-Meteo/ERA5

De todos los predictores externos posibles, la temperatura del aire es el más relevante para la demanda eléctrica: determina el consumo de calefacción en invierno y el de refrigeración en verano (Valor et al., 2001). Alemania, además, no es un país climáticamente homogéneo: son unos 357 000 km² con gradientes térmicos apreciables entre el norte marítimo y el sur continental. Para capturar esa heterogeneidad se descargan datos de temperatura horaria de diez ciudades representativas.

Criterio de selección de ciudades

Las diez ciudades se escogieron combinando dos criterios: población superior a 500 000 habitantes (lo que garantiza representatividad del consumo residencial y terciario) y una distribución geográfica equilibrada por el territorio alemán (norte, centro, este, oeste y sur). El consumo energético está fuertemente correlacionado con la masa poblacional y con la actividad industrial urbana; cubrir las diez mayores urbes permite además capturar los gradientes climáticos entre el norte (Hamburgo), el sur (Múnich) y el centro (Berlín, Fráncfort) del país. La Tabla 4.4 recoge las ciudades seleccionadas con sus coordenadas, tal y como figuran en el diccionario `CIUDADES_ALEMANAS` del script.

Ciudad	Latitud (°N)	Longitud (°E)
Berlín	52,5200	13,4050
Hamburgo	53,5511	9,9937
Múnich	48,1351	11,5820
Colonia	50,9375	6,9603
Fráncfort	50,1109	8,6821
Stuttgart	48,7758	9,1829
Düsseldorf	51,2277	6,7735
Leipzig	51,3397	12,3731
Dortmund	51,5136	7,4653
Essen	51,4556	7,0116

Tabla 4.4: Ciudades seleccionadas para la extracción de temperatura horaria, por orden de población.

En conjunto, estas diez ciudades representan aproximadamente el 15 % de la población alemana y cubren las principales zonas climáticas del país: la llanura nordalemana (Berlín,

Hamburgo), la cuenca del Rin y el Ruhr (Colonia, Düsseldorf, Dortmund, Essen), el centro (Fráncfort, Leipzig) y el sur (Múnich, Stuttgart).

Variables extraídas

Para cada ciudad y cada hora del período de estudio se extrae una única variable, `temperature_2m`: la temperatura del aire a 2 metros sobre el nivel del suelo, en grados Celsius, con resolución horaria y zona horaria Europe/Berlin. Se almacena en SQLite con un decimal de precisión, en una columna independiente por ciudad (`temp_berlin`, `temp_hamburgo`, ..., `temp_essen`).

¿Por qué no incorporar humedad relativa, velocidad del viento o irradiación solar? Porque [Díaz-Iglesias et al. \(2025\)](#) confirman que la temperatura a 2 metros es el predictor meteorológico dominante en predicción de demanda a corto plazo, y que las variables adicionales no mejoran de forma significativa el error de validación fuera de muestra cuando el modelo ya dispone de retardos temporales de la propia variable objetivo, como es el caso. Esta conclusión se limita, no obstante, al objetivo de reducir el error de predicción general: en la Sección 5.2 se retoma la posible incorporación de radiación, viento y humedad, pero no ya para mejorar el MAE, sino para representar explícitamente el efecto de la generación renovable variable sobre la demanda neta, un mecanismo estructural distinto que los retardos autorregresivos no capturan.

Fuente ERA5 e integración en Open-Meteo

Los datos históricos llegan a través de la *Archive API* de Open-Meteo (`archive-api.open-meteo.com`), que funciona como interfaz de acceso al reanálisis ERA5 del Centro Europeo de Predicción a Plazo Medio (ECMWF). ERA5 es un reanálisis atmosférico global de quinta generación: cubre desde 1940 hasta la actualidad con una resolución espacial de unos 31 km y resolución temporal horaria. Sus variables meteorológicas se obtienen asimilando datos de observación en un modelo de circulación general, lo que garantiza tanto la coherencia física como la ausencia de lagunas en la serie histórica.

El papel de Open-Meteo es el de capa de acceso: descarga las celdas del retículo ERA5 más cercanas a las coordenadas de cada ciudad, interpola los valores al punto exacto y los entrega con marcas de tiempo en hora civil de Berlín, fijada explícitamente mediante el parámetro `timezone=Europe/Berlin`. Para no contradecir la convención de almacenamiento de la Sección 4.3.1, precisemos que esa hora local no es la escala con la que se almacena ni se cruza nada: dado que la clave canónica de la base de datos es UTC, el script convierte los índices temporales recibidos de Europe/Berlin a UTC (mediante `pandas.zoneinfo`, resolviendo los tramos CET/CEST y las horas singulares del cambio de horario) antes de cualquier operación de cruce con la serie de demanda, y solo vuelve a la hora de Berlín en la capa de presentación y al derivar las variables de calendario en hora civil. En otras palabras: el sistema recibe y muestra en hora de Berlín, pero trabaja internamente en UTC. El valor de fijar el parámetro `timezone` de forma explícita está, precisamente, en hacer determinista esa

conversión: deja establecido sin ambigüedad el huso en el que llegan las marcas de la API, de modo que la reexpresión a UTC no dependa de suposiciones sobre el comportamiento por defecto del servicio.

API de Archivo Histórico (ERA5). La función `obtener_temperaturas_historicas()` consume el *endpoint*:

```
https://archive-api.open-meteo.com/v1/archive
?latitude={lat}&longitude={lon}
&start_date={YYYY-MM-DD}&end_date={YYYY-MM-DD}
&hourly=temperature_2m&timezone=Europe%2FBerlin
```

Este servicio expone el reanálisis ERA5 con una latencia mínima de un día (los datos del día en curso no están disponibles), así que el script limita automáticamente la fecha de fin a «ayer» para no lanzar peticiones vacías.

API de Previsión en Tiempo Real. La función `obtener_temperaturas_actuales()` consume el *endpoint*:

```
https://api.open-meteo.com/v1/forecast
?latitude={lat}&longitude={lon}
&current=temperature_2m&timezone=Europe%2FBerlin
```

Devuelve la temperatura medida en el instante presente bajo la clave `current.temperature_2m`.

API de Previsión Horaria (*forecast*). Los dos endpoints anteriores no bastan para alimentar las cadenas de predicción autorregresiva a horizontes superiores a una hora ($h = 6, 12, 24, 48, 72, 168$): el archivo histórico tiene una latencia mínima de un día y la previsión en tiempo real solo da un valor instantáneo, mientras que estas cadenas necesitan temperatura prevista para cada instante futuro del horizonte solicitado. Esa serie la obtiene `predecir_futuro.py`, mediante la función `obtener_temperaturas_pronostico()`, consultando el mismo dominio con parámetros distintos:

```
https://api.open-meteo.com/v1/forecast
?latitude={lat}&longitude={lon}
&hourly=temperature_2m
&start_date={YYYY-MM-DD}&end_date={YYYY-MM-DD}
&timezone=Europe%2FBerlin
```

El parámetro `hourly` (en lugar de `current`) devuelve una serie horaria de previsión (de hasta 16 días según la documentación pública de Open-Meteo) para las diez ciudades en una única petición combinada. Al igual que `obtener_temperaturas_actuales()`, esta función reintenta hasta tres veces (parámetro `OPEN_METEO_API_MAX_REINTENTOS`) ante un fallo, y si

el servicio sigue sin responder, propaga NaN para las ciudades y horas afectadas en lugar de interrumpir la cadena de predicción. Esta es, por tanto, la fuente real de las temperaturas que se citan en la Sección 4.4.2 al describir la evaluación multi-horizonte.

Proceso de descarga y alineación temporal

La descarga histórica se hace ciudad por ciudad (`obtener_temperaturas_historicas()`), con una pausa de 0,5 segundos entre peticiones para respetar los límites de tráfico de la API; esta función concreta no reintenta ante un fallo puntual, de modo que un error HTTP aislado en una ciudad se resuelve en la siguiente pasada de autocuración y no en la misma ejecución. La lógica de reintento (hasta tres intentos, con una espera de 1,0 segundo entre ellos) corresponde en cambio a `obtener_temperaturas_actuales()`, la función que consulta la previsión en tiempo real mediante una única petición combinada para las diez ciudades.

Una vez descargadas, los índices temporales de las diez series (ya reexpresados en UTC conforme a lo descrito más arriba) se alinean con las marcas temporales de la tabla de demanda mediante una operación de cruce (*join*) por clave exacta en UTC. Los pequeños huecos residuales se quedan marcados como NULL, el modelo recibe NaN en su lugar, y el proceso de recuperación seguirá intentando conseguirlos durante una semana. Las temperaturas resultantes se almacenan como diez columnas independientes en la misma tabla SQLite que contiene la demanda, lo que garantiza la coherencia temporal entre la variable objetivo y los predictores meteorológicos.

El problema de lo gratis

Apostar por fuentes de datos de acceso libre no está exento de riesgos operativos. La API Open-Meteo sirve datos meteorológicos en formato JSON de forma gratuita y sin registro previo, pero su plan público impone un límite de peticiones diarias. Cuando ese límite se supera (algo que puede ocurrir durante descargas históricas masivas o en días de alta actividad del sistema *cron*), la API devuelve un error explícito en lugar de los datos solicitados, como se ilustra en la Figura 4.6.

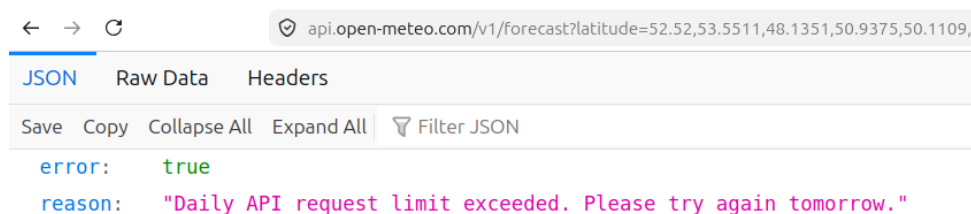


Figura 4.6: Error devuelto por la API Open-Meteo al superar el límite diario de peticiones gratuitas.

En condiciones normales, en cambio, la misma API responde con los datos de temperatura horaria en un JSON estructurado por localización (con los campos `latitude`, `longitude` y

el array `hourly.temperature_2m`), tal y como recoge a modo de ejemplo el fichero `forecast.json` adjunto a la memoria: en concreto, una respuesta de la API de Previsión Horaria descrita más arriba, con la serie `hourly.temperature_2m` de las diez ciudades para las 48 horas siguientes al momento de la petición. Este riesgo de interrupción, inherente a cualquier proveedor gratuito que puede modificar sus cuotas sin previo aviso, constituye una de las limitaciones operativas del sistema, y conviene tenerlo presente antes de extrapolar el enfoque a entornos de producción donde la disponibilidad continua de datos meteorológicos resulte crítica. Las tres capas de resiliencia que se describen a continuación son, precisamente, la respuesta de diseño a este riesgo.

Tolerancia a fallos y continuidad del pipeline

Para que el pipeline sobreviva a los fallos de red o de la propia API, el script implementa tres capas de resiliencia:

1. Control de tasa: pausa obligatoria de 0,5 s entre peticiones a ciudades sucesivas en la descarga histórica, respetando los límites de la API.
2. Reintentos: hasta 3 intentos, con pausa de 1,0 s, si la API devuelve error HTTP o hay un fallo de red, implementados en la consulta de previsión en tiempo real (`obtener\_temperaturas\_actuales()`, una única petición para las diez ciudades); la descarga histórica ciudad por ciudad no reintentará ante un fallo puntual y confía en la siguiente pasada de autocuración para completar el dato.
3. Estrategia de tolerancia a fallos: si la API sigue fallando tras los reintentos, la base de datos registra NULL de forma intencionada; ese valor llega al modelo como NaN, que LightGBM gestiona de forma nativa enviándolo a la rama de imputación aprendida durante el entrenamiento (Sección 3.4.4), de modo que la predicción se emite igualmente en lugar de fallar o exigir una imputación *ad hoc*. El módulo de autocuración (`-complete`) detectará el nulo y lo rellenará al día siguiente con el dato definitivo de ERA5.

4.3.3 Festivos nacionales alemanes

La demanda eléctrica no se comporta igual un martes cualquiera que un festivo nacional: en estos últimos, la demanda industrial y terciaria se desploma mientras la residencial mantiene perfiles muy parecidos a los de un domingo. Incorporar este efecto de forma explícita, como variable predictora, permite al modelo capturar esas caídas sistemáticas sin confundirlas con anomalías de la señal.

Fuente de datos

El calendario de festivos nacionales alemanes procede de la librería Python `holidays` (versión ≥ 0.20), que implementa los calendarios oficiales de más de 100 países y regiones.

Para Alemania se instancia el objeto `holidays.DE(years=[año])`, que carga los festivos de ámbito federal (*Bundesfeiertage*) directamente en memoria, sin necesidad de peticiones HTTP adicionales. Entre ellos figuran:

- Año Nuevo (*Neujahr*) 1 de enero.
- Viernes Santo (*Karfreitag*) y Lunes de Pascua (*Ostermontag*).
- Día del Trabajo (*Tag der Arbeit*) 1 de mayo.
- Día de la Ascensión (*Christi Himmelfahrt*) y Lunes de Pentecostés (*Pfingstmontag*).
- Día de la Unidad Alemana (*Tag der Deutschen Einheit*) 3 de octubre.
- Navidad y San Esteban (*1. y 2. Weihnachtstag*) 25 y 26 de diciembre.

El script inicializa el objeto de forma dinámica, a partir de los años presentes en los datos a procesar:

```
años = set(int(ts[:4]) for ts in pendientes['marca_temporal'])
festivos_de = holidays.DE(years=list(años))
```

Codificación binaria

La variable `es_festivo` se define como un indicador binario de valor entero:

$$\text{es_festivo}(t) = \begin{cases} 1 & \text{si la fecha del instante } t \text{ es festivo nacional} \\ 0 & \text{en caso contrario} \end{cases}$$

El indicador vale lo mismo para las 24 horas de un día festivo, dado que el efecto sobre la demanda persiste durante toda la jornada. La evaluación se limita a comparar la parte de fecha (`dt.date()`) contra el conjunto de festivos:

```
es_festivo = 1 if dt.date() in festivos_de else 0
```

Con esta codificación, LightGBM puede activar ramificaciones específicas de sus árboles para corregir a la baja las predicciones en días festivos, cuyo perfil de consumo se asemeja al de un domingo (descenso pronunciado de la demanda industrial y comercial).

Tratamiento de festivos regionales

Alemania tiene 16 estados federados (*Bundesländer*), y cada uno puede declarar festivos adicionales de ámbito regional: el Día de la Reforma Protestante en algunos estados del este, el Corpus Christi en los estados católicos del sur, etcétera. Estos festivos regionales se han dejado fuera de la variable `es_festivo`, por dos razones:

1. La variable objetivo es un agregado de todo el sistema alemán. El impacto de un festivo que solo afecta a uno o dos estados sobre la demanda total es marginal (inferior al 10 % del efecto de un festivo federal) y puede ser absorbido implícitamente por los retardos temporales y las variables cíclicas del modelo.
2. Incluirlos exigiría ponderar su impacto según la participación de cada estado en el consumo eléctrico total, lo que añadiría complejidad al pipeline sin una mejora demostrada en los indicadores de error de validación.

4.3.4 Análisis exploratorio de datos

Antes de construir el modelo predictivo se llevó a cabo un análisis exploratorio de los datos (EDA, por sus siglas en inglés), con tres objetivos: caracterizar la distribución estadística de la demanda, identificar los patrones temporales relevantes y verificar la relación entre las variables predictoras y la variable objetivo.

Distribución estadística de la demanda

La serie de demanda horaria de Alemania se caracteriza aquí sobre el período enero de 2023–junio de 2026 (30.569 observaciones del histórico de producción), una ventana amplia y reciente que sirve de contexto descriptivo; recuérdese que las métricas de error del modelo, en cambio, se calculan exclusivamente sobre la ventana de validación en vivo de julio de 2026 (Sección 4.4.3). La distribución es unimodal y prácticamente simétrica. La Tabla 4.5 resume los estadísticos descriptivos principales.

Estadístico	Valor (MW)
Media	53 105
Mediana	53 194
Desviación típica	9 185
Mínimo	30 903
Máximo	78 241
Asimetría	+0,11

Tabla 4.5: Estadísticos descriptivos de la demanda eléctrica horaria (MW), calculados sobre las 30.569 observaciones del período descriptivo 1 de enero de 2023–30 de junio de 2026.

La distribución resulta prácticamente simétrica (asimetría +0,11), con media y mediana casi coincidentes. Merece un apunte la combinación concreta que muestran los datos: la mediana queda 89 MW *por encima* de la media, algo que la regla heurística clásica asociaría a asimetría negativa, mientras que el coeficiente de asimetría (tercer momento estandarizado) es ligeramente positivo. No hay contradicción: esa regla es una orientación empírica, no un teorema, y esta distribución la incumple por un mecanismo visible en la propia tabla. Por un lado, las numerosas horas de baja demanda (valles nocturnos, fines de semana y agostos) concentran masa por debajo del centro y tiran de la media ligeramente hacia abajo respecto a la mediana; por otro, la cola derecha se extiende algo más lejos en términos tipificados

que la izquierda (el máximo queda a $+2,7\sigma$ de la media y el mínimo a $-2,4\sigma$), y como el tercer momento pondera cúbicamente los extremos, esos picos invernales raros pero intensos bastan para dejar el coeficiente en positivo. Ambos estadísticos describen, pues, aspectos distintos de una misma forma casi simétrica. En cuanto a la localización de los extremos: los valores mínimos (mínimo absoluto de 30 903 MW) se concentran en las horas nocturnas de festivos y fines de semana, mientras que los máximos (hasta 78 241 MW) corresponden a los picos matinales de los días laborables de invierno.

Perfil diario (curva de carga media por hora)

La curva de carga media horaria dibuja una «M» con dos máximos diferenciados, forma característica de los sistemas eléctricos europeos. Entre las 02:00 y las 03:00 h (hora de Berlín) la demanda toca su mínimo nocturno, en torno a 42 700 MW de media. A partir de las 06:00 h escala con rapidez hasta el máximo matinal, hacia las 10:00–11:00 h ($\approx 60\,000$ MW), empujada por el arranque de la actividad industrial y terciaria. Tras un suave descenso al mediodía, la demanda recupera un segundo máximo, algo más moderado, hacia las 18:00 h ($\approx 58\,600$ MW), cuando el regreso a los hogares se solapa con la actividad industrial tardía; desde las 21:00 h cae de forma sostenida hasta el siguiente valle nocturno.

Perfil semanal

En el perfil semanal se distinguen tres grupos de días con diferencias estadísticamente significativas. Los laborables (lunes a viernes) concentran la demanda media más alta, con los dos picos diarios muy marcados y un nivel base elevado durante las horas centrales. Los sábados, el pico matinal se desplaza aproximadamente una hora hacia la derecha y pierde amplitud, con un nivel medio un 10–15 % inferior al de los laborables. Los domingos y festivos, por último, la curva se aplanan y presenta un único máximo suave entre las 11:00 y las 14:00 h, con un nivel medio que puede quedar hasta un 20–25 % por debajo del de los laborables de invierno.

Esta diferenciación es la que justifica incluir `dia_semana` (día de la semana codificado como entero de 1 a 7) y `es_festivo` como predictores explícitos del modelo.

Perfil anual: estacionalidad y tendencias

La demanda exhibe además una marcada estacionalidad anual: los valores más altos aparecen en diciembre, enero y febrero (calefacción eléctrica, menos horas de luz, mayor actividad industrial en el primer trimestre), y los más bajos en julio y agosto (menor actividad industrial, vacaciones, ausencia de calefacción).

Merece la pena señalar, como contexto documentado por fuentes gubernamentales y de prensa, que en el otoño de 2022 (septiembre–noviembre) la demanda alemana fue significativamente inferior a lo esperado, como consecuencia de las campañas de ahorro energético lanzadas ante la crisis de suministro de gas natural. Ese período precede tanto a la ventana

caracterizada en este EDA (enero de 2023 en adelante, Tabla 4.5) como a la ventana de validación en vivo sobre la que se evalúa el modelo (4–12 de julio de 2026, Sección 4.4.13), así que no se observa directamente en los datos aquí analizados. Se menciona únicamente como ejemplo del tipo de ruptura estructural exógena que ningún predictor basado solo en variables temporales cíclicas podría capturar si llegara a ocurrir dentro de la ventana evaluada.

Correlación temperatura-demanda: forma en U

La relación entre la temperatura media ponderada de las diez ciudades y la demanda describe una curva en «U», típica de los sistemas energéticos con penetración mixta de calefacción eléctrica y aire acondicionado. Para temperaturas bajas (por debajo de 5 °C), la demanda crece al bajar la temperatura, con una pendiente aproximada de $-500 \text{ MW/}^\circ\text{C}$, empujada por la calefacción eléctrica y el calentamiento de agua sanitaria. En la zona de confort (15–20 °C) la demanda toca su mínimo estructural, ya que apenas hay necesidad de acondicionamiento térmico. Por encima de 25 °C la demanda vuelve a subir, aunque con una pendiente menor que en el extremo frío ($\approx +200 \text{ MW/}^\circ\text{C}$), reflejo del uso creciente de sistemas de refrigeración comerciales e industriales; la penetración del aire acondicionado residencial en Alemania es todavía moderada en comparación con los países mediterráneos, y esa es la razón de la asimetría de la curva.

Esta relación no lineal justifica introducir la temperatura como predictor directo, sin transformación previa: los árboles de decisión capturan de forma natural las relaciones no monótonas, dividiendo en múltiples umbrales a lo largo del rango de la variable.

Detección de *outliers*

Los valores atípicos se identifican con el método del rango intercuartílico (IQR), aplicado a la distribución condicional de la demanda por franja horaria y tipo de día (laborable, fin de semana, festivo). Se consideran *outliers* los registros cuya demanda supera los límites:

$$[Q_1 - 3 \cdot \text{IQR}, Q_3 + 3 \cdot \text{IQR}]$$

calculados de forma independiente para cada combinación de hora y tipo de día. Este enfoque condicional evita etiquetar como anómalos valores que, siendo extremos en la distribución marginal, resultan perfectamente coherentes en su contexto: una demanda de 80 000 MW a las 10:00 h de un lunes de enero no es un *outlier*, aunque supere el percentil 99 de la distribución global.

Los casos que sobreviven a este criterio corresponden principalmente a dos situaciones: (1) interrupciones puntuales en la telemetría de los operadores de sistemas de transmisión, que generan valores nulos, y (2) los días de Navidad y Año Nuevo, cuando la demanda cae por debajo del límite inferior de las horas laborables habituales.

4.3.5 Descripción de los conjuntos de datos anexos

4.3.5.1 `exportacion_demanda.xlsx`: volcado íntegro de la base de datos de producción

El fichero `exportacion_demanda.xlsx`, generado mediante el script `exportar_excel.py` el 12 de julio de 2026, contiene el volcado completo de la base de datos de producción: 88.111 registros horarios que cubren 3.673 días consecutivos, desde las 22:00 UTC del 29 de junio de 2016 hasta las 15:00 UTC del 19 de julio de 2026, y cada registro consta de 34 columnas. La cobertura horaria es prácticamente completa, con dos salvedades honestas: los dos días de frontera son parciales por construcción, ya que el volcado empieza y termina a una hora concreta del día y no a medianoche (2 horas el 29 de junio de 2016, 16 horas el 19 de julio de 2026); y, dentro del intervalo, quedan 11 marcas horarias puntuales sin dato, concentradas entre el 16 y el 18 de julio de 2026, pendientes de que el proceso de autocuración (Sección 4.3.1) las complete en una ejecución posterior. En ningún caso falta un día completo. Las filas correspondientes a marcas temporales posteriores al momento de la exportación (hasta el 19 de julio de 2026, límite del horizonte de predicción semanal) contienen únicamente la marca temporal y las predicciones, puesto que su demanda real todavía no ha sido publicada por SMARD.

Este volcado íntegro documenta la totalidad del sistema (histórico de entrenamiento, registro operativo y predicciones prospectivas), y de él se extrae el análisis exploratorio del período 2023–2026. Las métricas de error de las Tablas 4.6 y 4.7, en cambio, se calculan exclusivamente sobre el segundo anexo, que se describe a continuación.

Las columnas se agrupan en cinco bloques temáticos:

Bloque de demanda y predicción operativa.

- **marca_temporal**: sello de tiempo en formato YYYY-MM-DD HH:MM, resolución horaria, escala UTC.
- **demanda_real**: potencia eléctrica media registrada en el período, en MW.
- **demanda_prevision**: previsión oficial publicada por SMARD para ese mismo intervalo horario.
- **prediccion**: registro operativo acumulado de las salidas del modelo propio (Sección 4.1.3).
- **error_absoluto** y **error_porc**: diferencia absoluta y porcentual entre `prediccion` y `demanda_real`.

Bloque de predicciones multi-horizonte en vivo. Las columnas `prediccion_1h`, `prediccion_6h`, `prediccion_12h`, `prediccion_24h`, `prediccion_48h`, `prediccion_72h` y `prediccion_168h` almacenan, para cada hora objetivo, la predicción que el pipeline emitió con esa antelación exacta. A diferencia de la columna `prediccion` y de las evaluaciones retrospectivas, estos valores se escriben *antes* de que la demanda real correspondiente exista, por lo

que constituyen pronósticos genuinamente prospectivos. Están pobladas desde el 4 de julio de 2026 a las 00:00 UTC, fecha de inicio de la validación operativa en vivo que se describe en la Sección 4.4.3; las filas anteriores a esa fecha se eliminaron deliberadamente de estas columnas al reiniciarse el registro tras el reentrenamiento de julio.

Bloque de variables de entrada al modelo.

- **demanda_t_1** y **demanda_t_24h**: retardos autorregresivos de 1 y 24 horas.
- **hora_sin** y **hora_cos**: codificación cíclica de la hora del día.
- **día_ano_sin** y **día_ano_cos**: codificación cíclica del día del año.
- **día_semana**: día de la semana codificado como entero de 1 a 7 (1 = lunes, 7 = domingo).
- **es_festivo**: indicador binario de festivo nacional alemán.

Bloque de temperaturas por ciudad. Temperatura horaria en grados Celsius de las diez ciudades seleccionadas (Tabla 4.4): **temp_berlin**, **temp_hamburgo**, **temp_munich**, **temp_colonia**, **temp_frankfurt**, **temp_stuttgart**, **temp_dusseldorf**, **temp_leipzig**, **temp_dortmund** y **temp_essen**.

Bloque de metadatos.

- **creado_el**: instante de creación del registro en la base de datos.
- **smard_consolidado** y **meteo_consolidado**: indicadores binarios que señalan si los datos de demanda y de meteorología de esa fila están ya consolidados (definitivos) o pendientes de revisión por los procesos de autocuración (Sección 4.3.1).

4.3.5.2 exportacion_demanda_para_metricas.xlsx: ventana de validación operativa

Como segundo anexo se incluye **exportacion_demanda_para_metricas.xlsx**, un subconjunto del volcado anterior restringido a la ventana del 4 de julio de 2026 a las 00:00 UTC al 12 de julio de 2026 a las 15:00 UTC: el tramo en el que las columnas de predicción multi-horizonte en vivo coexisten con demanda real ya publicada (208 marcas horarias). **Todas las métricas de error reportadas en esta memoria** (Tablas 4.6 y 4.7) se calculan exclusivamente sobre este fichero, de modo que cualquier cifra puede reproducirse, columna a columna, a partir de él.

4.3.5.3 Justificación del diseño del conjunto de datos

Estas variables responden a los principales factores que gobiernan la demanda eléctrica a escala horaria. Los *lags* de demanda aportan la memoria a corto y medio plazo; la codificación cíclica de la hora y del día del año evita discontinuidades artificiales en variables de

naturaleza periódica; las temperaturas de múltiples ciudades permiten capturar los efectos de calefacción y refrigeración de forma geográficamente distribuida; y el indicador de festivo recoge las caídas sistemáticas de consumo de los días no laborables.

4.4 Resultados

4.4.1 Métricas globales: MAE, RMSE y MAPE sobre la ventana de validación en vivo

El período de evaluación abarca del 4 de julio de 2026 a las 00:00 UTC al 12 de julio de 2026 a las 15:00 UTC: 208 marcas horarias consecutivas con demanda real ya publicada por SMARD, recogidas en el anexo `exportacion_demanda_para_metricas.xlsx`. Todas las métricas de esta sección (y las de la evaluación por horizontes de la Tabla 4.7) se calculan exclusivamente sobre este fichero, aplicando de forma directa las fórmulas que se muestran a continuación sobre las columnas correspondientes. Las predicciones del modelo propio son las del registro prospectivo en vivo (`prediccion_1h` a `prediccion_168h`, Sección 4.4.3): cada valor quedó escrito en la base de datos antes de que existiera la demanda real que pretende predecir, con las previsiones meteorológicas realmente disponibles en el momento de emisión.

No todos los sistemas juegan con el mismo n : el número de observaciones válidas difiere entre sistemas y horizontes por dos motivos que merecen quedar explícitos desde el principio. El primero es estructural: para cada horizonte h , una predicción solo puede existir para marcas objetivo posteriores en h horas al instante en que arrancó la escritura de esa columna en la base de datos; por eso n decrece con el horizonte, hasta las 41 observaciones disponibles a $h = 168$. Un matiz: ese instante de arranque efectivo, según los registros de la base de datos, se adelantó unas horas a la medianoche del 4 de julio aquí tomada como referencia general de la ventana. De ahí que, por ejemplo, $n = 41$ a $h = 168$ y no el valor algo menor ($n = 40$) que resultaría de aplicar la medianoche del 4 de julio como frontera estricta. El lector que quiera reproducir estos n a partir de las marcas de tiempo exactas de arranque y fin de cada columna, en vez de a partir de la referencia general de la ventana, deberá tener en cuenta este pequeño desplazamiento. El segundo motivo son los 13 huecos puntuales del registro que se analizan en la Sección 4.4.3 (a los que se añaden, específicamente para $h = 168$, dos huecos adicionales no incluidos en esa lista). La previsión de SMARD y los *baselines naive*, en cambio, están disponibles para las 208 marcas de la ventana (con la excepción del *naive* semanal, cuyo retardo de 168 horas solo queda dentro de la ventana para las últimas 40 marcas, aplicando aquí sí de forma estricta la medianoche del 4 de julio como frontera, al calcularse sobre el anexo congelado y no sobre el registro prospectivo).

4.4.1.1 Métricas del modelo propio (evaluación a un paso, $h = 1$)

Error Absoluto Medio (MAE). El MAE promedia los errores absolutos sin penalizar especialmente los valores atípicos:

$$\text{MAE} = \frac{1}{195} \sum_{i=1}^{195} |y_{\text{pred},i} - y_{\text{real},i}| \approx 717,13$$

Una precisión sobre los decimales: las cifras de las fórmulas y tablas de esta sección se reportan con dos decimales por trazabilidad exacta con el anexo de cálculo, pero su precisión estadística real es muy inferior. Con $n = 195$ observaciones, la desviación típica de los errores absolutos que implica el par MAE/RMSE ($\sqrt{\text{RMSE}^2 - \text{MAE}^2} \approx 694 \text{ MW}$) sitúa el error estándar del MAE en torno a los 50 MW, de modo que los decimales carecen de significado estadístico. Por ese motivo, cuando el texto y el resumen citan estas cifras de forma aislada las redondean al MW, y los decimales se conservan únicamente en fórmulas, tablas y cálculos derivados de ellas.

Raíz del Error Cuadrático Medio (RMSE). El RMSE eleva los errores al cuadrado antes de promediarlos, con lo que castiga con especial dureza los fallos puntuales de gran magnitud. Si un modelo comete errores pequeños de forma sistemática pero incurre en dos o tres predicciones catastróficas, su RMSE se disparará respecto al MAE; es, por tanto, un indicador sensible a las *aberraciones* puntuales:

$$\text{RMSE} = \sqrt{\frac{1}{195} \sum_{i=1}^{195} (y_{\text{pred},i} - y_{\text{real},i})^2} \approx 997,86$$

Error Porcentual Absoluto Medio (MAPE). El MAPE expresa el error en términos relativos respecto al valor real, lo que facilita la interpretación con independencia de la escala de la variable:

$$\text{MAPE} = \frac{100\%}{195} \sum_{i=1}^{195} \left| \frac{y_{\text{pred},i} - y_{\text{real},i}}{y_{\text{real},i}} \right| \approx 1,50\%$$

4.4.1.2 Métricas de la previsión oficial SMARD

Las mismas métricas se calculan tomando como referencia las previsiones publicadas por SMARD (columna *demanda_prevision*), disponibles para las 208 marcas de la ventana:

$$\text{MAE}_{\text{SMARD}} = \frac{1}{208} \sum_i |y_{\text{SMARD},i} - y_{\text{real},i}| \approx 1893,08$$

$$\text{RMSE}_{\text{SMARD}} = \sqrt{\frac{1}{208} \sum_i (y_{\text{SMARD},i} - y_{\text{real},i})^2} \approx 2382,23$$

$$\text{MAPE}_{\text{SMARD}} = \frac{100\%}{208} \sum_i \left| \frac{y_{\text{SMARD},i} - y_{\text{real},i}}{y_{\text{real},i}} \right| \approx 4,06\%$$

Un aviso para la lectura del resto del capítulo: esta cifra del 4,06 % describe a SMARD sobre las 208 marcas completas de la ventana. Cuando más adelante la comparación con el modelo propio a $h = 24$ se restringe a las 175 marcas en las que ambas previsiones están disponibles, el MAPE de SMARD sobre ese subconjunto común baja al 4,03 %; esa, y no la de 208 marcas, es la cifra homologable para la comparación uno a uno, y es la que se emplea en el resumen y en las conclusiones. Ambos valores coexisten en el texto porque miden cosas distintas, no por inconsistencia.

4.4.1.3 Métricas de los *baselines* naive

Los dos *baselines naive* (Sección 4.2.1) se evalúan con las mismas fórmulas, sustituyendo y_{pred} por la predicción *naive* correspondiente ($\hat{y}_t = y_{t-24}$ para el diario, $\hat{y}_t = y_{t-168}$ para el semanal), sin componente aprendido alguno:

$$\text{MAE}_{\text{naive24h}} \approx 4174,31 \quad \text{RMSE}_{\text{naive24h}} \approx 5929,28 \quad \text{MAPE}_{\text{naive24h}} \approx 8,66\% \quad (n = 208)$$

$$\text{MAE}_{\text{naive168h}} \approx 1354,15 \quad \text{RMSE}_{\text{naive168h}} \approx 1614,08 \quad \text{MAPE}_{\text{naive168h}} \approx 3,15\% \quad (n = 40)$$

El *naive* diario queda claramente por detrás de todos los sistemas. El *naive* semanal, en cambio, exige una lectura muy cuidadosa: su retardo de 168 horas solo cae dentro de la ventana para las últimas 40 marcas (del 11 de julio en adelante), que además son horas de fin de semana predichas a partir del fin de semana anterior (exactamente el escenario más favorable posible para un *naive* estacional semanal). Su llamativo 3,15 % no es, por tanto, generalizable, pero sí resulta informativo: confirma la potencia de la periodicidad semanal de la demanda y refuerza la incorporación del retardo $t-168$ h como línea de trabajo futura (Sección 5.2).

4.4.1.4 Comparación de resultados

La Tabla 4.6 resume las tres métricas para todos los sistemas, indicando explícitamente el régimen de información de cada fila. En todos los casos, valores más bajos indican mejor ajuste.

Cada sistema se presenta con sus propias métricas de error, calculadas sobre la misma ventana temporal. A diferencia de versiones anteriores de esta evaluación, la fila $h = 24$ del modelo propio y la previsión de SMARD operan ahora en regímenes de información genuinamente comparables: ambas se emiten con una antelación del orden de un día respecto a la hora objetivo. Restringiendo la comparación a las 175 marcas en las que ambas están

Sistema (régimen)	n	MAE (MW)	RMSE (MW)	MAPE (%)
Modelo propio, $h = 1$ (<i>nowcasting</i>)	195	717,13	997,86	1,50
Modelo propio, $h = 24$ (<i>day-ahead</i>)	175	1733,44	2268,53	3,45
Previsión SMARD (<i>day-ahead</i>) [†]	208	1893,08	2382,23	4,06
Naive diario (lag-24h)	208	4174,31	5929,28	8,66
Naive semanal (lag-168h)*	40	1354,15	1614,08	3,15

Tabla 4.6: Métricas de error sobre la ventana de validación operativa en vivo (4 de julio de 2026, 00:00 UTC – 12 de julio de 2026, 15:00 UTC; 208 marcas horarias con demanda real). Todas las cifras se calculan exclusivamente sobre el anexo *exportacion_demanda_para_metricas.xlsx*. Las predicciones del modelo propio son prospectivas: quedaron registradas antes de existir la demanda real correspondiente.

[†] Calculada sobre las 208 marcas completas; en la comparación uno a uno con el modelo propio a $h = 24$, restringida a las 175 marcas comunes, el MAPE de SMARD es del 4,03 % (véase el texto). * El naive semanal solo dispone de retardo dentro de la ventana para las últimas 40 marcas (horas de fin de semana), un subconjunto que lo favorece sistemáticamente; esta restricción proviene de que el anexo congelado solo incluye los retardos $t-1$ y $t-24h$ que emplea el modelo, no $t-168h$, y no de una carencia real de datos (Sección 4.4.13); véase el texto.

disponibles, el modelo propio obtiene un MAPE del 3,45 % frente al 4,03 % de SMARD sobre esas mismas marcas. La diferencia, favorable pero modesta, no se somete aún a contraste estadístico formal: con una ventana de una semana, un test de Diebold-Mariano carecería de potencia (Sección 4.4.11).

4.4.2 Análisis por horizonte de predicción

Detección del sesgo de evaluación y corrección metodológica

El MAPE del modelo propio a un paso ($h = 1$, 1,50 % en la Tabla 4.6) corresponde a un esquema en el que la variable *demanda_t_1* recibe el valor real observado en $t-1$. Ese número, sin más contexto, resulta engañoso, y por eso exige una explicación metodológica antes de interpretarse.

Bajo esa condición, la métrica mide *nowcasting*: el modelo no anticipa la demanda futura, sino que la estima con la información del instante inmediatamente anterior. Con una autocorrelación en ese retardo de $\rho_{t,t-1} \approx 0,97$, el error resultante es bajo por construcción, sea cual sea la calidad del modelo. Una evaluación en estas condiciones no es homologable a un pronóstico *day-ahead*, que no dispone del valor real de ninguna hora posterior a la de publicación.

Para caracterizar el comportamiento del sistema en un régimen de información equivalente al de un pronóstico a 24 horas, el modelo tiene que evaluarse en modo autorregresivo en cadena durante 24 pasos, sin corrección con valores reales intermedios. Esa es la evaluación que permite responder al Objetivo 3 del trabajo.

Evaluación multi-horizonte: protocolo experimental

El modelo se evalúa en modo autorregresivo puro para horizontes $h \in \{1, 6, 12, 24, 48, 72, 168\}$ horas, directamente sobre el registro prospectivo en vivo (Sección 4.4.3). En cada emisión, la cadena de predicción arranca de los últimos valores reales disponibles en ese momento y se genera sin corrección: $\hat{y}_{T+k}$ se retroalimenta como demanda_t_1 para predecir $\hat{y}_{T+k+1}$. Las exógenas deterministas (variables de calendario y festivos) se calculan de forma exacta para cada instante futuro; las temperaturas proceden de las previsiones meteorológicas de Open-Meteo realmente disponibles en el momento de emisión (obtenidas mediante la API de Previsión Horaria descrita en la Sección 4.3.2, no de observaciones retrospectivas), de modo que su incertidumbre queda incorporada de forma natural en el error medido.

El protocolo resultante es *walk-forward* en sentido estricto y operativo: cada hora, el sistema en producción emite y persiste una nueva cadena de predicción, y la columna `prediccion_hh` acumula, para cada marca objetivo, el valor emitido exactamente h horas antes. Los errores se agregan después por horizonte sobre la ventana de validación. Es la variante de origen móvil que recomiendan Petrović et al. (2026) para evitar que el resultado dependa del punto de corte de una partición temporal única, llevada aquí a su forma más exigente: la de un sistema prediciendo en vivo.

Horizonte h (h)	n	MAE (MW)	RMSE (MW)	MAPE (%)
1	195	717,1	997,9	1,50
6	191	1 386,6	1 769,6	2,83
12	185	1 479,1	2 000,8	3,00
24	175	1 733,4	2 268,5	3,45
48	152	2 196,3	2 538,6	4,34
72	135	2 030,7	2 294,2	4,10
168	41	2 274,1	2 710,5	5,31

Tabla 4.7: Evolución del MAE, RMSE y MAPE en función del horizonte de predicción h , calculada exclusivamente sobre el registro prospectivo en vivo de la ventana de validación (anexo `exportacion_demanda_para_metricas.xlsx`; 4–12 de julio de 2026). El tamaño muestral n decrece con el horizonte porque una predicción a h horas solo puede existir para marcas posteriores en h horas al arranque efectivo de escritura de esa columna (ligeramente anterior, según los registros de la base de datos, a la medianoche del 4 de julio tomada como referencia general de la ventana; véase el texto), además de por los huecos puntuales de la Sección 4.4.3 (y, para $h = 168$, dos huecos adicionales no incluidos en esa lista). Todos los valores son medidos en condiciones operativas reales, no estimados ni retrospectivos.

El horizonte $h = 24$ reproduce el mismo régimen de información que un pronóstico *day-ahead*, y a ese horizonte el modelo obtiene un MAPE del 3,45 %. La cifra, sensiblemente más alta que la de la evaluación a un paso ($h = 1$, MAPE 1,50 %), ilustra cómo crece el error a medida que el régimen de evaluación se aleja del *nowcasting* y se aproxima a un pronóstico real, sin corrección con valores intermedios.

Hay un segundo hallazgo en la Tabla 4.7: el error crece de forma marcadamente sublineal, con el grueso del deterioro concentrado en las primeras 24 horas. Entre $h = 1$ y $h = 24$

Evolución del error según el horizonte de predicción (ventana de validación en vivo)

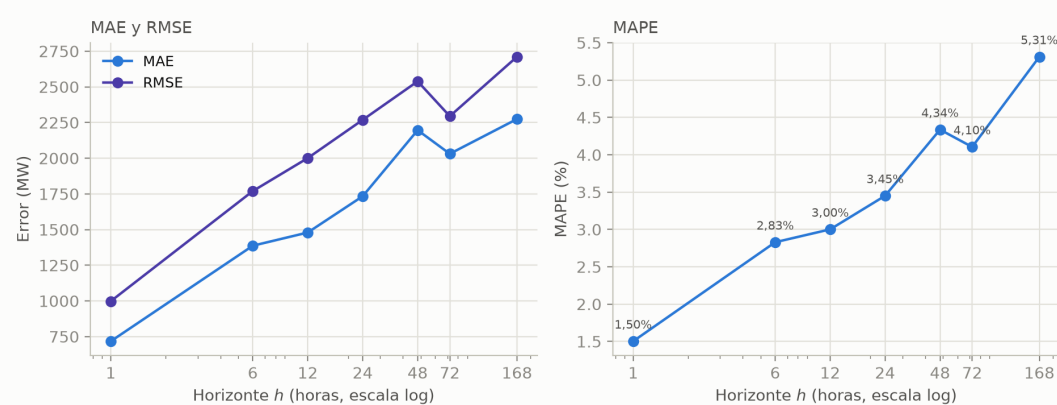


Figura 4.7: Evolución del MAE, el RMSE (izquierda, en MW) y el MAPE (derecha, en %) en función del horizonte de predicción h (escala logarítmica), a partir de los mismos datos de la Tabla 4.7. El crecimiento es marcadamente sub-lineal: el grueso del deterioro se concentra entre $h = 1$ y $h = 24$, seguido de una meseta entre $h = 24$ y $h = 72$, con la ligera no monotonía entre $h = 48$ y $h = 72$ visible en ambos paneles.

el MAPE pasa de 1,50 % a 3,45 % (se multiplica por $\approx 2,3$ mientras el horizonte se multiplica por 24), y a partir de ahí la curva se aplana: 4,34 % a $h = 48$ y 4,10 % a $h = 72$. La ligera no monotonía entre ambos horizontes (el error a tres días vista resulta algo menor que a dos) no tiene interpretación estructural: con 152 y 135 observaciones respectivamente, es ruido muestral de una ventana breve. La explicación del aplanamiento está en las variables exógenas (temperatura prevista y calendario): actúan como un ancla dominante que estabiliza la predicción una vez que la contribución relativa del retardo autorregresivo se diluye frente a esas señales, en línea con lo observado por Taieb y Hyndman (2014). El horizonte semanal exige más cautela: el MAPE de 5,31 % a $h = 168$ se apoya en solo 41 observaciones (concentradas, además, en horas de fin de semana) y debe tomarse como indicativo. Con esas reservas, el cuadro general es consistente: los horizontes de 1 a 24 horas son los más fiables ($\text{MAPE} \leq 3,45\%$), el sistema conserva una precisión operativamente razonable ($\text{MAPE} \approx 4\text{--}4,3\%$) a dos y tres días vista, y no se observa la degradación acelerada a partir de $h = 48$ que se había hipotetizado antes de disponer de mediciones.

4.4.3 Validación operativa en vivo (julio de 2026)

Un problema técnico en la máquina de producción obligó a reentrenar el modelo a comienzos de julio de 2026, sin pérdida alguna de datos: la base de datos conservaba el historial completo. El incidente se aprovechó para instrumentar el pipeline con un registro de predicciones *prospectivas*: desde el 4 de julio de 2026 a las 00:00 UTC, cada ejecución del sistema escribe en la base de datos, además de la predicción operativa habitual, las predicciones emitidas con antelaciones de 1, 6, 12, 24, 48, 72 y 168 horas, en columnas dedicadas (prediccion_1h a prediccion_168h, Sección 4.3.5).

Este protocolo es metodológicamente más exigente que la evaluación retrospectiva empleada en versiones preliminares del trabajo, que aplicaba un único modelo hacia atrás sobre el historial: aquí, cada valor queda escrito en la base de datos *antes* de que exista la demanda real que pretende predecir, empleando las previsiones meteorológicas realmente disponibles en el momento de emisión, no observaciones ERA5 retrospectivas, y sin posibilidad alguna de contaminación dentro de muestra. Se trata de una validación estrictamente fuera de muestra en condiciones operativas reales, y por ese motivo todas las métricas reportadas en este trabajo (Tablas 4.6 y 4.7) se calculan exclusivamente sobre este registro.

El fichero anexo `exportacion_demanda_para_metricas.xlsx` recoge la ventana de validación: del 4 de julio de 2026 a las 00:00 UTC al 12 de julio de 2026 a las 15:00 UTC, esto es, 208 marcas horarias con demanda real ya publicada.

El precio de este rigor es el tamaño muestral: la ventana cubre 8,6 días de un único mes de verano, sin variación estacional ni festivos federales, y con un n que decrece con el horizonte (Tabla 4.7). Las métricas resultantes están libres de los sesgos retrospectivos, pero su precisión estadística es limitada; esta brevedad se registra como la principal limitación del trabajo (Sección 4.4.13), y la extensión de la ventana (que crece de forma automática mientras el pipeline permanece en operación) queda como línea de trabajo futura (Sección 5.2).

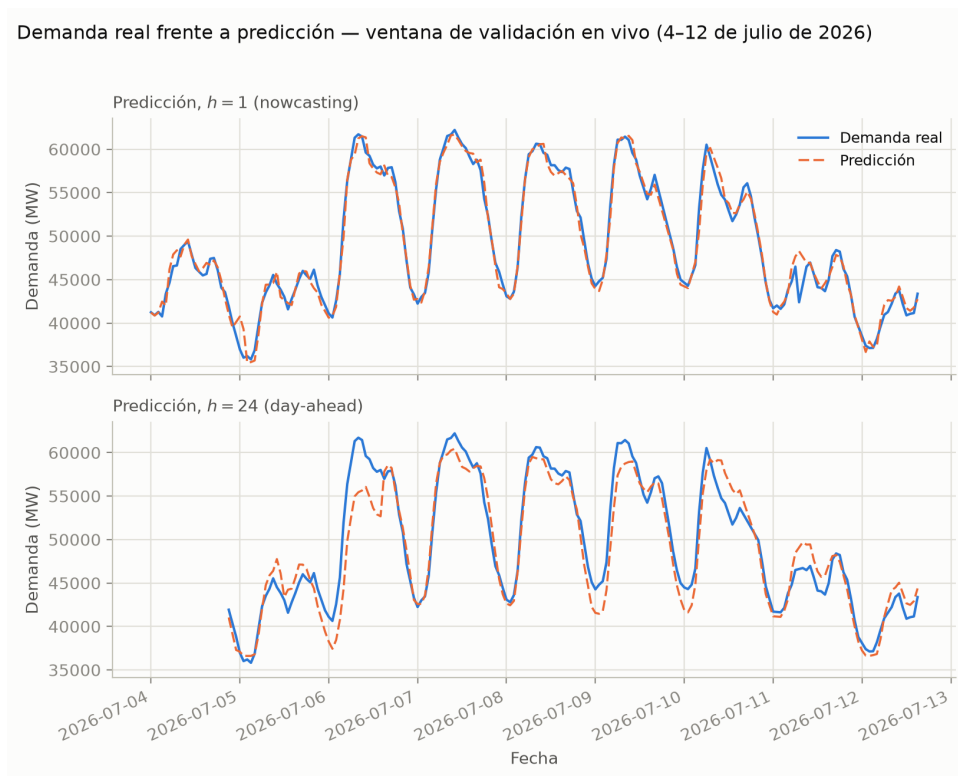


Figura 4.8: Demanda real frente a predicción sobre la ventana de validación en vivo (4–12 de julio de 2026), a los dos horizontes que enmarcan el régimen de nowcasting y el de day-ahead. A $h = 1$ la predicción se superpone casi por completo a la demanda real; a $h = 24$ se aprecia un desfase visible en los picos y valles, consistente con la asimetría de MAPE entre ambos horizontes (1,50 % frente a 3,45 %, Sección 4.4.2).

La Figura 4.9 muestra la misma comparación (demanda real frente a predicción a $h = 1$) tal como la presenta en producción el panel de control interactivo (`panel_control.py`, Sección 4.1.2), en una captura tomada el 12 de julio de 2026 a las 21:42 (hora local de Berlín). Añade dos elementos que la Figura 4.8 no recoge: la previsión oficial de SMARD sobre la misma ventana, y una *trayectoria autorregresiva* que se proyecta hacia delante desde el instante de la captura (12 de julio, en torno a las 20:00) hasta completar una semana. Ese tramo final (del 13 al 15 de julio, en violeta) es una proyección pura hacia el futuro respecto al momento en que se tomó la captura: en esas fechas todavía no existía demanda real publicada por SMARD con la que contrastarla, por lo que el panel la representa sin la serie roja de referencia. Se trata de una capacidad operativa del sistema (mostrar hacia dónde evoluciona la cadena autorregresiva antes de que la realidad la confirme o la desmienta), no de un resultado evaluado: a diferencia de las Tablas 4.6 y 4.7, ese tramo no aporta ninguna métrica de error y no debe confundirse con ellas.

El encabezado del panel, en el instante de la captura, reporta además $N = 156/168$, $MAE = 707,1$ MW, $RMSE = 975,1$ MW y una correlación de Pearson $R = 0,9908$ para el horizonte $h = 1$. Estas cifras no son las métricas oficiales de este trabajo: el panel las calcula en el navegador (JavaScript del lado cliente, no el servidor Flask) sobre la ventana temporal que esté seleccionada en cada momento —de 24 horas por defecto, ampliable a los últimos siete días mediante el botón de zoom «1 Sem.», que es evidentemente la vista activa en esta captura concreta ($N = 156/168$)—, mientras que las Tablas 4.6 y 4.7 se congelan sobre el anexo `exportacion_demanda_para_metricas.xlsx` (4 de julio 00:00 a 12 de julio 15:00 UTC, $n = 195$ válidos a $h = 1$). La ligera diferencia entre el 707,1 MW del panel y el 717,13 MW de la Tabla 4.6 se debe a ese desplazamiento de ventana y del instante de cálculo, no a un cambio de modelo ni de metodología; las cifras que sustentan las conclusiones de este trabajo son, en todo momento, las de la exportación congelada, no las que el panel muestra en vivo en un instante dado.

Integridad del registro en vivo: huecos detectados y análisis de causas

La inspección de la ventana de validación reveló que los horizontes cortos presentan 13 huecos puntuales y dispersos, no un corte continuo, en las marcas 04-jul 23:00; 05-jul 23:00; 09-jul 13:00, 17:00, 18:00, 19:00 y 20:00; 10-jul 00:00, 02:00 y 08:00; 11-jul 08:00 y 23:00; y 12-jul 00:00 (todas en UTC). El proceso de adquisición *no* se interrumpió en ningún momento: `logs/cron\_bd.log` registra 2.246 entradas de ejecución durante el período, de las que solo 4 no llegaron a completar su bloque. Para que la cifra no despiste, hay que conciliarla con el diseño del planificador: los dos disparos horarios del adquirente suman del orden de 414 lanzamientos en los 8,6 días de la ventana, de modo que las 2.246 entradas son el recuento de bloques de ejecución que el log anota, no disparos del *cron*: cada lanzamiento del adquirente registra por separado sus distintas tareas internas, y al mismo log escriben también el resto de procesos programados (reconsolidación diaria, autocuración, ciclo de predicción en vivo). La cifra relevante para la continuidad del servicio es la segunda: solo 4

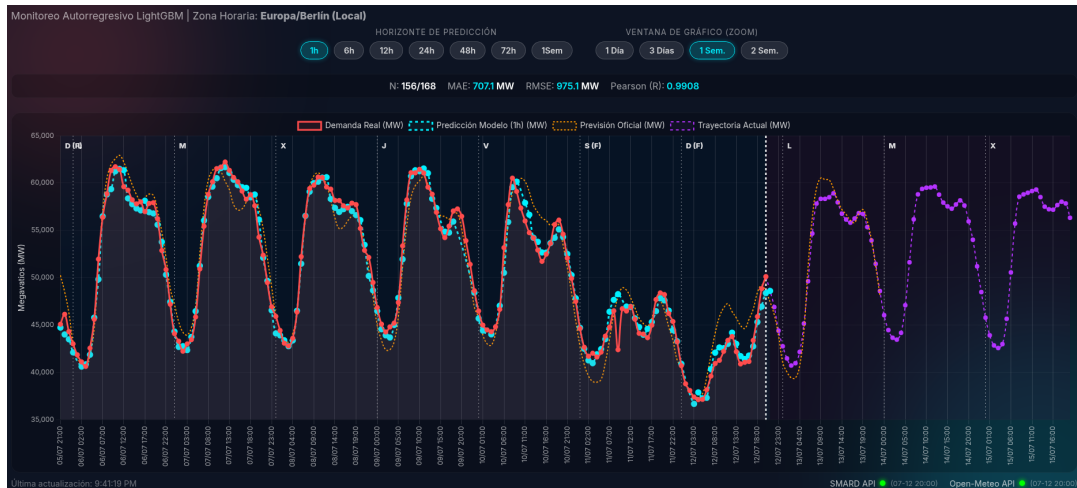


Figura 4.9: Captura del panel de control en producción (`panel_control.py`, 12 de julio de 2026, 21:42 hora de Berlín): demanda real, predicción del modelo a $h = 1$, previsión oficial de SMARD y trayectoria autorregresiva proyectada hacia el 13–15 de julio. El tramo violeta es una proyección hacia el futuro respecto al instante de la captura y no tiene todavía demanda real con la que compararse; las métricas del encabezado ($N = 156/168$, MAE, RMSE, Pearson R) corresponden a la ventana móvil de siete días del panel en ese instante, no a la ventana congelada de las Tablas 4.6 y 4.7.

de esos bloques quedaron incompletos. El análisis forense del log permitió atribuir los huecos a dos causas de naturaleza distinta.

Una de ellas es retrasos de publicación del *feed* de SMARD combinados con el diseño de disparo único del registro en vivo (10 de los 13 huecos). El registro de `prediccion_1h` funciona como un *slot* de una sola oportunidad: en cada ejecución, el sistema consulta a SMARD cuál es la última hora con demanda real publicada, y solo en esa ejecución concreta escribe la predicción a una hora vista para la marca siguiente. Si una hora nunca llega a ser «la última publicada» en ningún *tick* (porque SMARD la publicó con retraso, en bloque junto a horas posteriores), su predicción queda huérfana de forma permanente: los procesos de *backfill* diario y semanal rellenan los datos crudos, pero no reemiten predicciones. La reconstrucción de la secuencia de «últimas horas conocidas» del 9 de julio ilustra el mecanismo: el *cron* vio las 10:00, 11:00, saltó la 12:00, vio 13:00–15:00, saltó las 16:00–19:00 y reapareció directamente en las 20:00; las horas saltadas dejaron sin `prediccion_1h` a sus objetivos correspondientes (13:00 y 17:00–20:00). El mismo patrón explica los huecos del 10 y el 11 de julio. La causa raíz (la latencia de publicación del proveedor) es externa al sistema; el diseño de disparo único, en cambio, es una decisión propia mejorable, y su corrección (reemisión de predicciones huérfanas en las ejecuciones posteriores) se registra como línea de trabajo futura.

Otra es fallos silenciosos de escritura por concurrencia en SQLite (los 3 huecos restantes: 04-jul 23:00, 05-jul 23:00 y 12-jul 00:00). En estos tres casos el log muestra que la ejecución predecesora se completó y reportó éxito («Predicciones a múltiples horizontes guardadas correctamente»), pero el valor no consta en la base de datos, sin ningún error de predicción

registrado. La hipótesis más plausible es un fallo silencioso de escritura: el propio log confirma al menos un caso explícito de `database is locked` en `adquirir_datos.py`, el error característico de SQLite cuando varios procesos acceden simultáneamente al mismo fichero. El escenario es estructural: sobre la misma base de datos concurren la adquisición horaria (minutos 10 y 40), el proceso de reconsolidación diario, y el panel Flask leyendo para servir las gráficas; SQLite, un motor embebido en fichero, no está diseñado para alta concurrencia y aborta la conexión que llega en segundo lugar. La corrección aplicada consistió en añadir un margen de tolerancia a todas las conexiones del proyecto (nueve *scripts* de Python), sustituyendo `sqlite3.connect(RUTA_BD)` por `sqlite3.connect(RUTA_BD, timeout=30.0)`: el proceso que encuentra la base de datos bloqueada ya no aborta de inmediato, sino que espera en cola hasta 30 segundos a que el primero termine. Dado que las operaciones del sistema tardan fracciones de segundo, esta espera elimina en la práctica el fallo por colisión, aunque, al tratarse de un fallo intermitente y no reproducible a voluntad, su erradicación completa solo podrá confirmarse con la observación continuada del registro en vivo.

A ambas causas se suma, como factor de fondo ya descrito en la Sección 4.3.2, el límite de peticiones de Open-Meteo: el log registra numerosos errores HTTP 429 («Open-Meteo inaccesible tras 3 intentos») en la ruta de predicción en vivo; en estos casos el código dispone de un mecanismo de reserva que sustituye la temperatura ausente por NaN, que LightGBM gestiona de forma nativa (Sección 3.4.4), y normalmente la predicción se emite igualmente, por lo que esta causa degrada puntualmente la calidad de los insumos pero no explica los huecos.

Sumados, los 13 huecos suponen en torno al 6 % de las marcas de la ventana para el horizonte más afectado, con predominio claro de la causa externa (retrasos del proveedor) sobre la interna (1 % de anomalía por concurrencia). El episodio deja dos lecciones operativas que se incorporan a las limitaciones y al trabajo futuro: los sistemas que dependen de *feeds* gratuitos deben diseñar sus registros de predicción con capacidad de reemisión, y la persistencia embebida en SQLite, suficiente para este prototipo, marca el punto en el que un despliegue de producción exigente debería migrar a un motor cliente-servidor.

4.4.4 Análisis por franja horaria y día de semana

Desagregando el error del modelo a un paso ($h = 1$) por hora del día sobre la ventana de validación en vivo, no se observa un patrón claro de concentración en las rampas matinal o vespertina: el MAPE por hora oscila entre el 0,90 % (08:00 h) y el 3,28 % (07:00 h) sin una tendencia sistemática hacia las horas de mayor variabilidad estructural de la demanda. Con solo 7–9 observaciones por hora (una única muestra por día de la semana a lo largo de 8,6 días), la variabilidad muestral es demasiado alta para extraer de aquí ninguna conclusión robusta sobre qué franjas horarias concentran el error; confirmar o descartar ese patrón exige repetir el desglose cuando la ventana de validación acumule varias semanas.

Por tipo de día, la ventana solo permite comparar laborables ($n = 112$, MAPE del 1,23 %) frente a fin de semana ($n = 83$, MAPE del 1,86 %): en este tramo concreto el error resulta, de

hecho, algo mayor en fin de semana, justo lo contrario de lo que cabría esperar por la mayor regularidad del perfil de sábados y domingos, y no debe generalizarse dado que la muestra cubre un único ciclo semanal. El resultado, con todo, no debe dejarse como simple curiosidad: hay al menos tres mecanismos compatibles con él, verificables en cuanto la ventana acumule varios ciclos semanales. El primero es aritmético: el MAPE normaliza por el nivel de demanda, y ese nivel es estructuralmente un 10–25 % más bajo en fin de semana (Sección 4.3.4), de modo que un mismo error absoluto en MW pesa más en términos porcentuales un sábado que un martes; repetir la comparación en MAE por tipo de día es el diagnóstico que separaría este efecto de denominador de una diferencia real de precisión. El segundo es muestral: el mayor error individual de toda la ventana (5 871 MW, sábado 11 de julio a las 07:00 h) cae precisamente dentro del subconjunto de fin de semana, que con $n = 83$ es muy sensible a una única desviación grande. El tercero es estructural, y el más informativo: para todas las horas de un sábado, el retardo diario `demanda_t_24h` apunta a un viernes laborable (y para las de un lunes, a un domingo), es decir, cruza la frontera entre regímenes de demanda, exactamente la circunstancia por la que la autocorrelación diaria ($\rho_{t,t-24} \approx 0,68$) resulta muy inferior a la semanal ($\rho_{t,t-168} \approx 0,91$) sobre esta misma ventana. La divergencia entre ambos perfiles alcanza su máximo en la rampa matinal (presente el viernes, ausente o desplazada el sábado; Sección 4.3.4), que es justamente la franja del error máximo observado. Si esta tercera hipótesis es la dominante, al extenderse la ventana el error de fin de semana debería concentrarse de forma sistemática en las mañanas de sábado y lunes, y la incorporación del retardo $t-168$ h, que nunca cruza esa frontera, debería reducirlo de forma medible, lo que aporta una vía de contraste adicional a esa línea de trabajo ya priorizada (Sección 5.2). Es necesario además dejar constancia de un límite estructural de esta ventana: ningún festivo nacional cae dentro del período evaluado (`es_festivo` vale 0 en las 208 marcas), por lo que el comportamiento del modelo en festivos (incluida la hipótesis de que la variable `es_festivo` le permite activar ramas específicas del árbol para esos días) no puede contrastarse empíricamente en este trabajo; queda pendiente de validación en cuanto el registro en vivo cubra al menos un festivo federal.

4.4.5 Análisis por estación del año

La ventana de validación en vivo (4–12 de julio de 2026) cubre un único régimen estacional, pleno verano, por lo que las métricas de error de las Tablas 4.6 y 4.7 no permiten, por ahora, contrastar empíricamente el comportamiento del modelo en invierno ni en las estaciones de transición. Lo que sigue es, por tanto, una expectativa razonada a partir del diseño del modelo y de la relación temperatura-demanda en forma de «U» caracterizada en el análisis exploratorio (Sección 4.3.4), no un resultado medido: cabría esperar que en invierno (diciembre–febrero), con una demanda más alta y un perfil diario más pronunciado, la elevada autocorrelación de los retardos y el efecto más lineal y dominante de la calefacción favorezcan la capacidad predictiva; en verano, en cambio, el efecto de la refrigeración (menos lineal y más sensible a umbrales de temperatura) podría introducir algo más de incer-

tidumbre relativa; y en las estaciones de transición (primavera y otoño), la superposición de calefacción residual y refrigeración incipiente podría reducir la regularidad del perfil y elevar el MAPE. Confirmar o refutar esta expectativa exige extender la ventana de validación en vivo hasta cubrir un ciclo anual completo, tal como se recoge en la Sección 4.4.13 como línea de trabajo prioritaria.

Cabe recordar, en todo caso, que tanto la ventana de validación en vivo como el período caracterizado en el EDA (Tabla 4.5) son posteriores al otoño de 2022, cuando la crisis de suministro de gas natural provocó una reducción estructural documentada de la demanda industrial y residencial alemana; el episodio se cita aquí solo como ejemplo de ruptura estructural exógena (el tipo de evento que ningún predictor basado en datos históricos podría capturar sin información adicional sobre el contexto económico y político), no como comportamiento observado del modelo, pues queda fuera de ambas ventanas.

4.4.6 Importancia de variables (*feature importance*)

LightGBM calcula de forma nativa la importancia de cada variable como la ganancia media acumulada en la reducción de la función de pérdida MSE en todas las divisiones del *ensemble* donde interviene esa variable. El indicador se extrae directamente del modelo entrenado y permite auditar algo esencial: que las señales que el modelo utiliza son físicamente coherentes con el comportamiento esperado de la demanda.

Variable	Categoría	Ganancia (%)
demanda_t_1	Retardo ($t-1$ h)	76,43
demanda_t_24h	Retardo ($t-24$ h)	11,13
hora_cos	Cíclica (hora)	3,73
dia_semana	Calendario	3,71
hora_sin	Cíclica (hora)	3,08
dia_ano_cos	Cíclica (año)	1,05
dia_ano_sin	Cíclica (año)	0,17
temp_berlin	Temperatura	0,15
temp_frankfurt	Temperatura	0,09
temp_munich	Temperatura	0,09
es_festivo	Calendario	0,08
temp_hamburgo	Temperatura	0,07
temp_leipzig	Temperatura	0,06
temp_stuttgart	Temperatura	0,05
temp_colonia	Temperatura	0,03
temp_dusseldorf	Temperatura	0,03
temp_essen	Temperatura	0,03
temp_dortmund	Temperatura	0,03

Tabla 4.8: Importancia de cada variable predictora, medida como ganancia relativa (*gain*) acumulada en el *ensemble* LightGBM, extraída directamente del modelo actualmente desplegado (*modelos/LightGBM_model.txt*, 1.515 árboles). Las 18 variables suman el 100 % de la ganancia total.

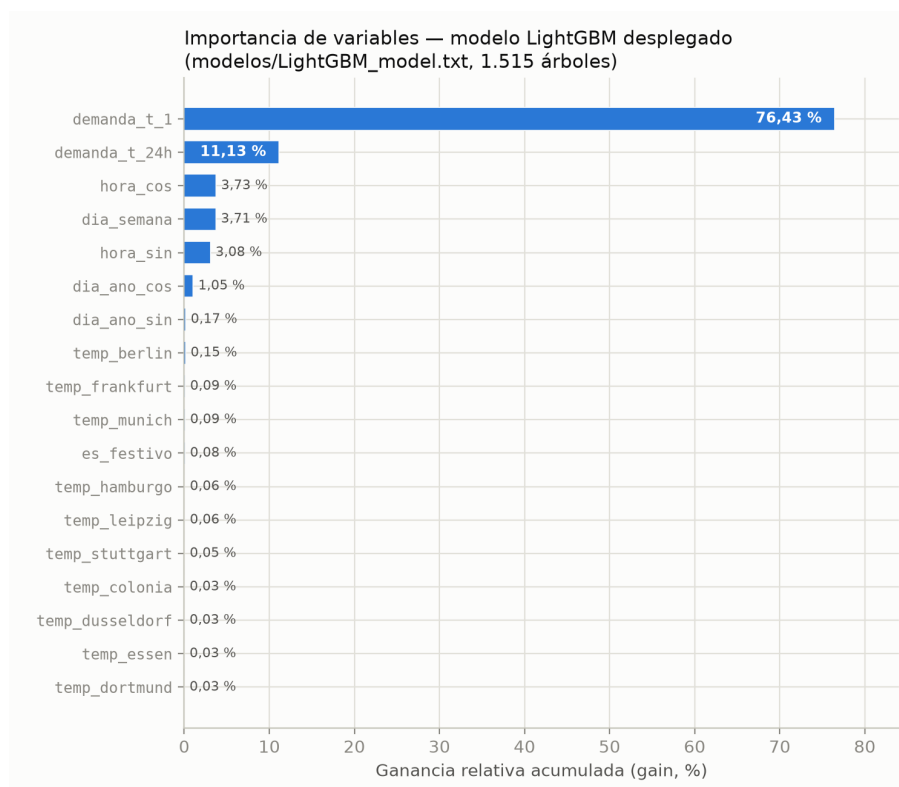


Figura 4.10: Representación gráfica de la Tabla 4.8: ganancia relativa (gain) de cada variable en el ensemble LightGBM desplegado, ordenada de mayor a menor. La escala lineal deja patente la asimetría de la jerarquía: el retardo inmediato domina con claridad, el retardo de 24 h es el único otro predictor con una contribución de dos dígitos, y el resto forma una cola larga de aportaciones modestas.

Los resultados (Tabla 4.8) revelan una jerarquía de importancia coherente con la estructura de autocorrelación de la serie, aunque menos concentrada en un único predictor de lo que un vistazo superficial a esa autocorrelación haría suponer. El retardo inmediato (*demanda_t_1*) acumula el 76,43 % de la ganancia total del *ensemble*, a bastante distancia del resto, algo consistente con la autocorrelación $\rho_{t,t-1} \approx 0,97$: el valor de la hora anterior es, con diferencia, el predictor con mayor poder informativo para la hora siguiente. El retardo de 24 horas (*demanda_t_24h*) ocupa el segundo puesto con un 11,13 %, lejos de ser una contribución marginal, lo que indica que una parte sustancial de la periodicidad diaria de la demanda no queda ya absorbida por el retardo inmediato ni por las componentes cíclicas de la hora, y que el modelo recurre de forma notable a este segundo retardo para capturarla, en línea con $\rho_{t,t-24} \approx 0,68$.

El tercer, cuarto y quinto puesto se reparten de forma ajustada entre *hora_cos* (3,73 %), *dia_semana* (3,71 %) y *hora_sin* (3,08 %), que capturan la estructura intradiaria y semanal de la demanda que los dos retardos no explican por completo. *dia_ano_cos* (1,05 %) se sitúa claramente por encima del resto de variables de calendario y meteorológicas, señal de que la estacionalidad anual aporta información adicional relevante incluso disponiendo de los retardos.

Las diez temperaturas urbanas, pese a su papel conceptual en el modelado del efecto de climatización, ocupan colectivamente una posición secundaria en la jerarquía: sumadas alcanzan un 0,63 % de la ganancia total, con Berlín a la cabeza (0,15 %), seguida de Fráncfort y Múnich (0,09 % cada una). `dia_ano_sin` (0,17 %) y el indicador de festivos `es_festivo` (0,08 %) muestran ganancias del mismo orden modesto. Ojo, sin embargo, con la interpretación: la ganancia por *gain* refleja la reducción de error que aporta cada variable *dado que las demás ya están disponibles* en el árbol. Que temperaturas y festivos aparezcan con ganancias pequeñas no las hace prescindibles: buena parte de la información que aportarían por separado ya la capturan indirectamente los retardos autorregresivos y las componentes cíclicas, sobre todo en un conjunto de datos donde estos alcanzan una autocorrelación tan alta. Su papel se vuelve más relevante justo donde el patrón autorregresivo no basta para anticipar un cambio de régimen (festivos, olas de frío o calor extremo), como se discute en la Sección 4.4.7.

La jerarquía casa con la física del problema: los retardos y la hora del día dominan por la elevada autocorrelación e intradiaridad de la serie, y temperatura y calendario aportan correcciones puntuales pero necesarias en los regímenes que los primeros no cubren. Un modelo que reparte así su ganancia difícilmente está aprendiendo artefactos espurios del conjunto de entrenamiento. Esta trazabilidad es, además, una ventaja diferencial que [Lemos-Vinasco et al. \(2026\)](#) señalan frente a las arquitecturas de aprendizaje profundo, y en la práctica un requisito para la adopción del sistema en entornos regulados.

4.4.7 Análisis de casos extremos

Ninguno de los tres tipos de episodio que se discuten a continuación (festivos de alta intensidad, olas de temperatura extrema y perturbaciones estructurales) se ha producido dentro de la ventana de validación en vivo (4–12 de julio de 2026: ningún festivo federal, y una temperatura media de las diez ciudades que osciló entre 13,5 y 30,8 °C, sin ningún episodio documentado como ola de calor o de frío). Lo que sigue describe, por tanto, el comportamiento *esperado* del modelo a partir de su diseño y de los patrones que muestra la jerarquía de importancia de variables (Sección 4.4.6), no un resultado verificado sobre la ventana de métricas; contrastarlo empíricamente en cuanto el registro en vivo cubra alguno de estos episodios queda como línea de trabajo prioritaria (Sección 4.4.13).

Los períodos de Navidad y Año Nuevo (24 de diciembre – 2 de enero) presentan perfiles de demanda atípicos: la demanda industrial cae a mínimos históricos mientras la residencial se mantiene en niveles comparables a los de un domingo. Cabría esperar que la variable `es_festivo` permita al modelo capturar la caída sistemática, aunque la profundidad exacta de esa caída puede variar de un año a otro según cómo se comporte la demanda industrial, y esa expectativa está aún pendiente de contrastación en vivo.

La Semana Santa alemana (Viernes Santo y Lunes de Pascua como festivos federales) genera un patrón de cuatro días con demanda reducida que, por diseño, el modelo debería poder anticipar gracias a la combinación de la codificación de festivos y la codificación cí-

clica del día del año; de nuevo, se trata de una expectativa de diseño y no de un resultado observado en la ventana evaluada.

Las olas de frío y de calor extremo, por último, activarían los segmentos de alta pendiente de la curva temperatura-demanda (Sección 4.3.4). En esos episodios cabría esperar que la temperatura de las diez ciudades pase a ser el predictor más informativo, obligando al modelo a extrapolar fuera del rango más frecuente de su historial de entrenamiento; la ventana de julio de 2026, con temperaturas moderadas de verano, no somete al modelo a esa prueba.

4.4.8 Interpretación de los resultados

El nivel de error alcanzado en la evaluación a un paso ($h = 1$) refleja, ante todo, la elevada autocorrelación de la serie de demanda eléctrica horaria. Un predictor trivial que se limitara a repetir el valor del instante anterior (*naive lag-1*) obtiene, sobre la misma ventana de validación, un MAE de 1 456,66 MW (MAPE del 2,98 %). El modelo LightGBM, que dispone además del retardo de 24 horas, la codificación cíclica de la hora y las señales de temperatura y calendario, reduce ese error en un 51 % (MAE de 717,13 MW): el componente predictivo aporta valor más allá de la inercia pura de la serie. Como se detalla en la Sección 4.4.6, el desglose de la ganancia del *ensemble* indica que la mayor parte de esa mejora procede del retardo de 24 horas y de la codificación horaria, y solo una fracción marginal de la temperatura y el calendario, por lo que esta reducción no debe leerse como evidencia de que estas dos últimas familias de variables sean, por sí solas, las responsables principales de la ganancia frente al *naive*.

Fijémonos ahora en la relación entre el RMSE (997,86 MW) y el MAE (717,13 MW) de la evaluación a un paso: el cociente es de $\approx 1,39$. Para una distribución de errores puramente gaussiana, el cociente teórico sería $\sqrt{\pi/2} \approx 1,25$; el valor observado indica que la distribución de errores tiene colas algo más pesadas de lo que predeciría un modelo de ruido gaussiano. En términos prácticos: el modelo comete errores moderados de forma consistente (eso lo recoge el MAE), pero incurre en un número reducido de desviaciones de gran magnitud que elevan el RMSE de forma desproporcionada. Dentro de esta ventana, el error individual más alto (5 871 MW) se produce, de hecho, a primera hora de la mañana de un sábado (11 de julio, 07:00 h) y no en una rampa de un día laborable, en línea con el MAPE algo más alto observado en fin de semana que en laborables (Sección 4.4.4); no responde, por tanto, a la concentración en rampas matinales/vespertinas o en episodios meteorológicos extremos que cabría anticipar por diseño (Sección 4.4.7), sino a una desviación puntual que, con solo 195 observaciones, pesa de forma desproporcionada sobre el RMSE. Su localización precisa, eso sí, es coherente con el mecanismo de cruce de regímenes propuesto en la Sección 4.4.4: a las 07:00 de un sábado, el retardo diario `demanda_t_24h` apunta a la rampa matinal de un viernes laborable, el punto de máxima divergencia entre los perfiles de ambos tipos de día.

La crisis de suministro de gas natural del otoño de 2022 (septiembre–noviembre), documentada por fuentes gubernamentales y de prensa como causa de una reducción sostenida de la demanda industrial alemana, ilustra bien el tipo de ruptura estructural exógena que

ningún predictor basado únicamente en datos históricos puede anticipar: de haber ocurrido dentro de la ventana evaluada, un modelo entrenado con patrones de demanda previos a la crisis habría sobreestimado sistemáticamente la demanda en ese tramo, elevando el MAPE del período por encima del valor global. Como el episodio precede en varios años a la ventana de validación en vivo de este trabajo (4–12 de julio de 2026), no se dispone de una medición directa de ese sesgo sobre el modelo aquí entrenado; se menciona como riesgo documentado, inherente a cualquier predictor histórico que carezca de variables que capturen el nivel de actividad económica o industrial en tiempo real, y cuya corrección pasaría por incorporar indicadores socioeconómicos externos, tal como se propone en las líneas de trabajo futuro.

Quedan así perfilados los escenarios en los que cabría esperar que el modelo falle con mayor probabilidad: (1) desviaciones puntuales de causa no identificada, del tipo de la observada el 11 de julio, cuya sistemática no puede establecerse a partir de una sola ventana de 8,6 días; (2) episodios meteorológicos extremos fuera del historial de entrenamiento, y (3) perturbaciones estructurales de la demanda que modifican el nivel base sin que ninguna señal del modelo las anuncie. Ninguno de los dos últimos se ha observado todavía dentro de la ventana evaluada (Sección 4.4.7), por lo que se plantean como riesgos anticipados por diseño, no como fallos verificados.

4.4.9 Suficiencia de las señales de bajo coste

El Objetivo 2 del trabajo planteaba una pregunta concreta: ¿bastan las señales de acceso público y bajo coste (retardos autorregresivos, temperaturas de diez ciudades y variables de calendario) para obtener un predictor competitivo?

La respuesta corta es que sí. La larga exige matices en el horizonte semanal, aunque en el rango operativo principal, de $h = 1$ a $h = 72$, la afirmación se sostiene. A horizonte corto ($h = 1$), el MAPE del 1,50 % queda en el entorno del rango 0,5–1,5 % que Ribeiro y dos Santos Coelho (2023) reportan para modelos STLF de estado del arte en sistemas eléctricos nacionales, ahora medido en vivo y no de forma retrospectiva. A 24 horas (el equivalente a un pronóstico *day-ahead*), la Tabla 4.7 muestra un MAPE medido del 3,45 %, muy por debajo del 8,66 % del *naive* diario (lag-24h, Tabla 4.6) y por debajo también del 4,03 % de la previsión oficial de SMARD sobre las mismas 175 marcas en las que ambas previsiones están disponibles, lo que sugiere que las señales de temperatura, festivos y codificación cíclica aportan valor real más allá de la inercia de la serie. A dos y tres días vista el modelo sigue siendo operativamente útil (MAPE del 4,34 % y el 4,10 %), con una acumulación de error autorregresivo mucho más contenida de lo previsto. El horizonte semanal, en cambio, exige cautela: el MAPE de 5,31 % a $h = 168$ descansa en solo 41 observaciones de fin de semana, y en ese mismo subconjunto el *naive* semanal (lag-168h) obtiene un 3,15 % (el escenario fin de semana contra fin de semana es su terreno más favorable), lo que sugiere que a horizontes largos el modelo aún no explota toda la periodicidad semanal de la serie y refuerza la incorporación del retardo $t-168$ h como línea de trabajo prioritaria.

En cualquier caso, para construir un predictor operativamente útil no hace falta disponer

de datos de mercado (precios, contratos de balance) ni de información privada de los TSO. El conjunto de señales seleccionado captura los tres determinantes fundamentales de la demanda horaria: la inercia a corto plazo (retardos), el efecto térmico (temperaturas) y la estructura temporal (calendario y festivos).

Posicionamiento en la literatura

La Tabla 4.9 contextualiza los resultados obtenidos frente a trabajos representativos de STLTF basados en gradient boosting publicados en los últimos años. La columna «Señales» refleja el tipo de información disponible en cada caso: los trabajos de la literatura suelen apoyarse en datos meteorológicos completos, históricos más largos o información de mercado; el presente trabajo opera exclusivamente con señales de acceso libre.

Referencia	Sistema	h	Señales	MAPE (%)
Ribeiro y dos Santos Coelho (2023)	Varios nacionales	1 h	Múltiples	0,5–1,5
Kostas et al. (2023)	Europa	1–24 h	Cal. + retardos	< 3,0
He et al. (2024)	Multi-escenario	1–24 h	Meteo. + cal.	1,5–3,5
Fan et al. (2021)	China	1–24 h	LightGBM + BiLSTM	< 2,0
Ashraf et al. (2024)	Residencial	1 h	Ensemble de árboles	< 2,5
Presente trabajo	Alemania	1 h (now-casting)	Solo señales públicas	1,50
Presente trabajo	Alemania	24 h	Solo señales públicas	3,45

Tabla 4.9: Comparación con resultados representativos de STLTF con gradient boosting en sistemas nacionales. h = horizonte de predicción. Los valores de la literatura son los reportados o inferidos en la revisión de cada fuente. La fila a $h = 1$ del presente trabajo corresponde a un esquema de nowcasting (el modelo dispone de la demanda real en $t - 1$, Sección 4.4.2); no se ha verificado si las cifras de la literatura a ese mismo horizonte nominal comparten ese régimen de información, por lo que esa fila concreta debe compararse con cautela adicional.

El MAPE de 1,50 % a horizonte $h = 1$ se sitúa en el entorno del rango reportado por Ribeiro y dos Santos Coelho (2023) para modelos de estado del arte en sistemas nacionales, si bien la comparación directa a ese horizonte está condicionada por el efecto de autocorrelación discutido en la Sección 4.4.2. A horizonte comparable ($h = 24$), el MAPE medido de 3,45 % queda ligeramente por encima del límite de < 3,0 % reportado por Kostas et al. (2023), aunque dentro del rango más amplio de He et al. (2024) (1,5–3,5 %); con dos matices adicionales que merecen destacarse: aquí el resultado se obtiene con un conjunto de señales de coste nulo, y se mide en vivo (con la incertidumbre meteorológica real incluida), mientras que buena parte de las cifras de la literatura proceden de evaluaciones retrospectivas.

4.4.10 Comportamiento autorregresivo y papel de las variables exógenas

Cuando el modelo opera en modo de predicción en cadena ($h > 1$), el error se propaga de forma acumulativa: un error en $\hat{y}_{T+1}$ se convierte en un error en la variable `demanda_t_1` de la predicción $\hat{y}_{T+2}$, y así sucesivamente. Este mecanismo de propagación es el aspecto metodológico central del Objetivo 3.

Las variables exógenas deterministas (temperatura y calendario) actúan como anclas que limitan la deriva del modelo a horizontes largos. Aunque el retardo autorregresivo se vaya alejando progresivamente de la realidad, la señal de temperatura horaria y las variables cíclicas de hora y día del año siguen aportando información precisa sobre el nivel esperado de demanda. Esta propiedad estabilizadora resulta especialmente valiosa en los horizontes de 24–168 horas, donde la demanda revierte de forma natural hacia su patrón estacional y la temperatura (disponible a través de las previsiones meteorológicas de Open-Meteo) sigue siendo una señal fiable (Taieb y Hyndman, 2014).

El perfil de degradación medido en vivo (Tabla 4.7) confirma que el crecimiento del error es sub-lineal, con el grueso del deterioro concentrado en las primeras 24 horas (MAPE del 1,50 % a $h = 1$ frente al 3,45 % a $h = 24$) y una meseta posterior en torno al 4–4,3 % a dos y tres días vista, sin rastro de la aceleración del *drift* a partir de $h = 48$ –72 que se había anticipado inicialmente. El horizonte semanal (5,31 % a $h = 168$, con solo 41 observaciones) apunta a una degradación adicional moderada que habrá que confirmar cuando la ventana de validación acumule más datos. Como herramienta complementaria de diagnóstico local, `predecir_tramo.py` simula el proceso autorregresivo sobre tramos concretos y calcula el MAE, el error máximo y la correlación de Pearson de cada cadena, lo que permite inspeccionar el efecto de fase que las métricas agregadas diluyen (Sección 4.4.13).

4.4.11 Diferencias metodológicas con la previsión oficial de SMARD

SMARD es el predictor de referencia del sistema eléctrico alemán, y su previsión es la que los agentes del mercado utilizan para sus decisiones de programación; por eso resulta pertinente situar el régimen de evaluación del modelo propio frente al suyo. El paso al protocolo prospectivo en vivo mejora sustancialmente la comparabilidad respecto a versiones anteriores de esta evaluación: la fila $h = 24$ del modelo propio se emite ahora con 24 horas reales de antelación (un régimen de información equiparable, incluso algo más exigente, que el de la previsión *day-ahead* de SMARD, publicada con unas 12–24 horas de antelación). Sobre las 175 marcas comunes de la ventana, el modelo obtiene un MAPE del 3,45 % frente al 4,03 % de SMARD.

Aun así, tres cautelas impiden proclamar superioridad:

1. **Potencia estadística.** La ventana cubre una única semana de verano. Un contraste formal de Diebold-Mariano sobre 175 observaciones horarias fuertemente autocorrelacionadas carecería de potencia y de representatividad estacional; el contraste queda pendiente de que la ventana de validación acumule varios meses (Sección 5.2).

2. **Objetivo de optimización distinto.** La previsión de SMARD está orientada a la gestión del mercado eléctrico y puede incorporar consideraciones de cobertura de reservas que introduzcan un sesgo conservador al alza, comportamiento por lo demás coherente con la gestión del riesgo de apagones. Sus errores no miden, por tanto, exactamente lo mismo que los del modelo propio.
3. **Riesgo de circularidad con la propia fuente de comparación.** El paso 1 de la cascada de imputación (Sección 4.3.1) sustituye los valores de `demanda_real` marcados como anómalos por `demanda_prevision`, que es exactamente la misma serie usada aquí como benchmark competidor. Si alguna de las 175 marcas comunes de esta comparación hubiera sido imputada de ese modo, SMARD acertaría trivialmente contra sí mismo en esa marca, sesgando el resultado a su favor. Como se detalla en la Sección 4.4.13, el esquema actual no permite confirmar ni descartar si esto ha ocurrido dentro de la ventana evaluada.

Con esas tres reservas, la comparación en vivo constituye un indicio favorable, pero no puede calificarse de metodológicamente limpio sin reservas: por primera vez en este trabajo, ambos sistemas se miden prediciendo el mismo futuro desconocido, aunque la posible circularidad señalada arriba queda sin cuantificar.

4.4.12 Síntesis: resultados frente a objetivos

La Tabla 4.10 recoge de forma explícita en qué grado se ha alcanzado cada uno de los tres objetivos del trabajo, a la luz de los resultados obtenidos.

El tercer objetivo merece un comentario aparte. El Objetivo 3 planteaba el análisis de la predicción autorregresiva a ciegas, y la experimentación deparó dos sorpresas: la magnitud de la asimetría entre $h = 1$ (MAPE 1,50 %) y $h = 24$ (MAPE 3,45 %) resultó mayor de lo anticipado, pero la degradación posterior se aplanaba notablemente entre $h = 24$ y $h = 72$ (MAPE del 4,10–4,34 %). Ambos hechos, juntos, constituyen el hallazgo metodológico de mayor valor del trabajo. El resultado pone sobre la mesa un riesgo sistemático en la evaluación de modelos STLTF operativos (la confusión entre *nowcasting* y *forecasting*, que los trabajos citados omiten con frecuencia) y deja establecido un protocolo de evaluación multi-horizonte reproducible, aplicable a cualquier sistema de predicción de demanda.

Estos resultados dejan el trabajo donde pretendía situarse: inteligencia artificial aplicada al forecasting energético. LightGBM, respaldado por la evidencia empírica sobre datos tabulares heterogéneos, ofrece una alternativa de bajo coste a los métodos estadísticos clásicos; la metodología de evaluación multi-horizonte aporta, de propina, un protocolo reproducible que cuantifica los límites y la asimetría de una evaluación que la literatura operativa suele pasar por alto.

Obj.	Enunciado	Evidencia empírica
1	Pipeline de predicción con datos públicos	Sistema operativo en producción: ingesta horaria vía SMARD + Open-Meteo, re-entrenamiento semanal, panel web interactivo y, desde el 4 de julio de 2026, registro prospectivo de predicciones a siete horizontes (Sección 4.4.3); convergencia estable del modelo LightGBM sin sobreajuste (parada temprana, Sección 4.1.4.4) y jerarquía de importancia coherente con la física del problema.
2	Suficiencia de señales de bajo coste	MAE $\approx$ 717 MW, MAPE = 1,50 % a $h = 1$; MAPE = 3,45 % a $h = 24$, medidos en vivo usando solo señales gratuitas, dentro del rango del estado del arte y por debajo de la previsión oficial de SMARD (4,03 %) sobre las mismas 175 marcas comunes.
3	Análisis de la predicción autorregresiva a ciegas	Degradación sub-lineal medida en vivo, con el grueso del deterioro en las primeras 24 horas (MAPE del 1,50 % a $h = 1$ al 3,45 % a $h = 24$) y meseta en torno al 4–4,3 % a $h = 48$ –72; las variables exógenas amortiguan la deriva autorregresiva más de lo anticipado; cuantificación empírica de la asimetría $h = 1$ vs $h = 24$ como hallazgo metodológico propio.

Tabla 4.10: Correspondencia entre los objetivos del trabajo y los resultados que los demuestran.

4.4.13 Limitaciones

Este trabajo presenta una serie de limitaciones que condicionan tanto la interpretación de los resultados como sus posibilidades de generalización:

1. **Ámbito geográfico único.** El modelo se ha entrenado y validado exclusivamente sobre el sistema eléctrico alemán. Trasladar el enfoque a otros países dependerá de que existan datos públicos equivalentes y de las particularidades de cada perfil de demanda (penetración del aire acondicionado, estructura industrial, mezcla energética).
2. **Ausencia de variables socioeconómicas.** El modelo no incorpora indicadores económicos capaces de explicar variaciones estructurales de la demanda (actividad industrial, precio de la energía, subsidios). Esto limita su capacidad de anticipar cambios de régimen como la reducción de demanda documentada en el otoño de 2022 a raíz de la crisis energética. El episodio es anterior a la ventana de validación en vivo evaluada en este trabajo (4–12 de julio de 2026) y se cita solo como ejemplo del tipo de perturbación que este enfoque no podría anticipar si se repitiera dentro del período evaluado.

3. **Horizonte máximo fiable.** En modo autorregresivo, la acumulación de error en los retardos va restando utilidad a las predicciones a medida que crece h . Contra lo que se anticipaba inicialmente, la evaluación empírica completa (Tabla 4.7) muestra una degradación sub-lineal, no acelerada: el MAPE pasa del 3,45 % a $h = 24$ a una meseta del 4,34 % a $h = 48$ y el 4,10 % a $h = 72$, y solo alcanza el 5,31 % a $h = 168$ (una semana). Aun así, el error ya supera el umbral del 4 % a partir de $h = 48$, y la cifra a $h = 168$ se apoya en solo 41 observaciones concentradas en fin de semana, por lo que debe tomarse como indicativa y no como una cota firme; queda como línea de trabajo futura extender la evaluación más allá de una semana y con mayor tamaño muestral para determinar en qué horizonte se produce, finalmente, la saturación o el repunte del error. Esta cautela es puramente estadística (tamaño muestral); el siguiente punto añade una cautela de naturaleza distinta (estructural, no de muestreo) sobre esos mismos horizontes.

4. **Desfase temporal (efecto de fase) en la predicción autorregresiva a nivel local.**

Las métricas agregadas de la Tabla 4.7, al estar promediadas sobre todo el conjunto de test, diluyen un patrón sistemático que sí aflora al inspeccionar tramos individuales con `prededir_tramo.py`: en modo autorregresivo puro, la predicción reproduce la forma de la demanda real desplazada aproximadamente el mismo número de horas que el horizonte h , en lugar de anticipar los puntos de giro. El efecto es casi imperceptible a $h = 1$ (Pearson $\approx 0,96$ sobre tramos de ejemplo), pero se hace evidente a $h = 6$ y $h = 12$: la cadena autorregresiva sitúa el pico vespertino o el valle nocturno varias horas después de su ocurrencia real, el MAE local en tramos de ejemplo alcanza entre 5 100 y 5 700 MW (frente a los 1 387–1 479 MW agregados de la Tabla 4.7 para esos mismos horizontes) y la correlación de Pearson cae hasta $\approx 0,63$. Esta comparación, eso sí, no aísla un único factor (el nivel de agregación): las cifras locales de `prededir_tramo.py` se calculan *in-sample*, sobre tramos del mismo fichero de entrenamiento (`train/germany-24-meses_train.train`) que ajustó los árboles del modelo, mientras que las cifras agregadas de la Tabla 4.7 proceden del registro prospectivo en vivo, estrictamente fuera de muestra (Sección 4.1.3). La brecha entre ambas cifras combina, por tanto, dos efectos que este trabajo no ha separado: la dilución estadística del promedio y el cambio de régimen de evaluación (*in-sample* frente a prospectivo); el efecto de fase descrito a continuación es real y se ha verificado también de forma cualitativa sobre el registro en vivo, pero su magnitud exacta fuera de muestra no debe leerse directamente de estas cifras de `prededir_tramo.py`. El error se concentra en las rampas de mayor pendiente (subida matinal, bajada vespertina), donde puede llegar a los 8 000–15 000 MW, mientras que las mesetas de mediodía o de valle nocturno apenas se ven penalizadas. El mecanismo es directamente atribuible a la jerarquía de importancia de la Sección 4.4.6 (Tabla 4.8): `demanda_t_1` concentra por sí sola el 76,43 % de la ganancia total del modelo (*gain*, calculada sobre `modelos/LightGBM_model.txt`), de manera que la cadena autorregresiva se apoya sobre todo en proyectar hacia de-

lante el patrón reciente en lugar de aprender a anticipar los cambios de tendencia. En consecuencia, las predicciones a $h \geq 6$ h no deberían emplearse para decisiones que dependan de la sincronización temporal exacta de picos y valles (por ejemplo, la programación de arranque de generación de punta); su uso recomendado se limita a estimar el nivel medio esperado de demanda en el horizonte considerado. La estrategia de modelos directos por horizonte, ya identificada como línea de trabajo futura (Sección 5), es la vía propuesta para corregir este efecto de fase.

5. **Retardos limitados.** El modelo emplea únicamente dos retardos ($t-1$ y $t-24h$). Incorporar retardos adicionales (por ejemplo, $t-48h$ y $t-168h$) podría mejorar la captura de la estacionalidad semanal, especialmente en la predicción a horizontes superiores a 24 horas.
6. **Evaluación sobre una única ventana, y solo una fracción mínima del histórico disponible.** Las métricas se calculan sobre una ventana de validación en vivo de 8,6 días, dentro de un histórico de producción que arranca en 2016. Ese histórico completo no se aprovecha en su totalidad: el entrenamiento del modelo solo emplea los últimos 24 meses (ventana móvil de 730 días, es decir, aproximadamente desde mediados de 2024), y el análisis exploratorio se limita, como contexto descriptivo, al período enero de 2023–junio de 2026 (Tabla 4.5); el resto del histórico (desde el 29 de junio de 2016 hasta mediados de 2024, aproximadamente) no interviene en este trabajo. Una validación complementaria con múltiples ventanas (por ejemplo, reproduciendo el protocolo prospectivo sobre otros tramos del año, o contrastando el comportamiento del modelo antes y después de la crisis energética de 2022) proporcionaría estimaciones de error más robustas.
7. **Dependencia de la calidad de la previsión meteorológica.** Las cadenas de predicción en vivo se alimentan de las previsiones de Open-Meteo disponibles en el momento de emisión, de modo que el error meteorológico queda incorporado, correctamente, en las métricas medidas. Ahora bien, ese error no se ha descompuesto: no se ha cuantificado qué fracción del error de demanda a horizontes largos es atribuible al error de la previsión de temperatura, un análisis que ayudaría a priorizar mejoras (por ejemplo, si conviene más añadir retardos o mejorar los insumos meteorológicos).
8. **Brevedad de la ventana de validación en vivo.** Es la principal limitación del trabajo en su forma actual. El registro prospectivo multi-horizonte (Sección 4.4.3), fuente única de todas las métricas reportadas, cubre 8,6 días de julio de 2026: un solo tramo del ciclo semanal, un único régimen estacional (verano), ningún festivo federal, y un tamaño muestral que decrece con el horizonte hasta las 41 observaciones a $h = 168$ (concentradas en horas de fin de semana). Las métricas son metodológicamente limpias (estrictamente fuera de muestra y en condiciones operativas reales) pero estadísticamente poco precisas, y su representatividad estacional es nula fuera del verano; el llamativo resultado del *naive* semanal a $h = 168$ (Tabla 4.6) ilustra bien cuánto puede distorsio-

nar un subconjunto tan pequeño. Ese subconjunto de 40 marcas, hay que precisarlo, no responde a una limitación de disponibilidad de datos: la base de datos de producción conserva histórico horario continuo desde 2016, por lo que el retardo de 168 horas existe, con demanda real registrada, para las 208 marcas de la ventana. La restricción a 40 proviene de que las métricas de las Tablas 4.6 y 4.7 se calculan exclusivamente sobre el anexo `exportacion_demanda_para_metricas.xlsx` (Sección 4.4.3), que solo incluye como columnas de retardo $t-1$ y $t-24h$, las que emplea el modelo, y no $t-168h$. En consecuencia, el MAPE del 3,15 % del *naive* semanal no debe leerse como el resultado de una evaluación exhaustiva de ese *baseline*: no se ha comprobado si mantendría su ventaja frente al modelo (5,31 % a $h = 168$) al calcularse sobre las 208 marcas disponibles en la base de datos completa, en lugar de sobre el subconjunto de fin de semana favorable al que lo restringe el anexo congelado. La ventana seguirá extendiéndose de forma automática mientras el pipeline permanezca en operación, y su consolidación hasta cubrir al menos un ciclo anual es la línea de trabajo futura de mayor prioridad.

9. **Huecos en el registro prospectivo y fragilidad de la persistencia embebida.** La ventana de validación en vivo presenta 13 huecos puntuales en los horizontes cortos (Sección 4.4.3): diez atribuibles a retrasos de publicación del *feed* de SMARD combinados con el diseño de disparo único del registro, que no reemite predicciones huérfanas, y tres compatibles con fallos silenciosos de escritura por concurrencia en SQLite (`database is locked`). El primer factor exige un mecanismo de reemisión; el segundo se ha mitigado con `timeout=30.0` en todas las conexiones, aunque un despliegue de producción exigente debería considerar la migración a un motor de base de datos cliente-servidor.
10. **Cambio de modelo dentro de la ventana de validación en vivo.** El ciclo de reentrenamiento semanal (domingos, 3:00 AM) no se detiene por el hecho de que la ventana de evaluación esté abierta: dentro del propio periodo evaluado (4–12 de julio de 2026) se ejecutaron dos reentrenamientos completos bajo la configuración de producción vigente: el del 5 de julio (árbol óptimo en la iteración 1.577) y el del 12 de julio (árbol óptimo en la 1.515, el modelo actualmente desplegado). Las cifras de las Tablas 4.6 y 4.7 combinan, así, las salidas de dos modelos semanales consecutivos y no de uno único e inmutable (Sección 4.1.3). Ambos modelos comparten pipeline, hiperparámetros y una ventana de entrenamiento casi enteramente solapada, y en ningún caso hay contaminación dentro de muestra (cada predicción se sigue escribiendo antes de conocer la demanda real correspondiente), pero la mezcla introduce una fuente adicional, y previsiblemente pequeña, de variabilidad que no se ha cuantificado por separado. Evitarla por completo exigiría, o bien congelar el modelo durante toda la ventana de evaluación (a costa de dejar de reentrenar semanalmente, contradiciendo el diseño operativo del sistema), o bien registrar el identificador de versión del modelo en cada predicción para poder desagregar el análisis por modelo; esto último se incorpora como

mejora inmediata del pipeline.

11. **Ausencia de refit final: el modelo desplegado nunca entrena con los últimos doce meses.** La partición cronológica 50/50 (Sección 4.1.3) implica que los árboles se ajustan únicamente sobre el bloque más antiguo de la ventana: el modelo que predice julio de 2026 se entrenó con datos que terminan hacia julio de 2025, y el año más reciente interviene solo en la parada temprana. No existe un paso de reentrenamiento final sobre los veinticuatro meses completos con el número de árboles ya seleccionado. Si el nivel o la forma de la demanda derivaron durante 2025–2026, los umbrales de partición de los árboles están calibrados sobre los cuantiles del régimen anterior. El efecto queda amortiguado por la vía de las características, pues el nivel reciente entra en inferencia a través de los retardos (`demanda_t_1` concentra el 76,43 % de la ganancia total, Sección 4.4.6), y acotado empíricamente por las métricas en vivo, pero no se ha cuantificado por separado. La corrección es de bajo coste (una segunda invocación de LightGBM con `num_iterations` fijado en el árbol óptimo y sin fichero de validación) y se recoge como mejora inmediata en la Sección 5.2.
12. **Cascada de imputación incompleta en la ruta de ingesta en tiempo real.** La jerarquía de imputación de cuatro pasos que corrige los valores marcados como anómalos (Sección 4.3.1) está implementada de forma completa en la rutina de consolidación histórica, pero en la rutina que corre en cada ejecución horaria de cron el tercer paso (mediana histórica por día de la semana y hora) no llega a ejecutarse y no existe el cuarto (`ffill/bfill`). El registro afectado queda marcado como no consolidado y se corrige en la siguiente pasada de autocuración, por lo que el impacto práctico se limita a un posible retraso en la imputación (nunca a una pérdida permanente del dato), pero conviene documentarlo como una discrepancia real entre el diseño descrito y el código actualmente en producción, y corregirla es una mejora inmediata de bajo coste.
13. **Ausencia de trazabilidad de la imputación y riesgo de circularidad no cuantificable.** Tal como se señala en la Sección 4.4.11, el paso 1 de la cascada de imputación sustituye `demanda_real` anómala por `demanda_prevision` (la misma serie que se usa como competidor de referencia en la comparación con SMARD), lo que introduce un riesgo de circularidad: cualquier marca imputada de este modo dentro de una ventana de comparación favorecería artificialmente a SMARD en esa marca concreta. El esquema de `datos_alemania` no incluye ninguna columna de auditoría (por ejemplo, un indicador de fuente o de imputación) que registre de forma permanente si un valor fue sustituido por este mecanismo; las columnas `smard_consolidado` y `meteo_consolidado` solo indican si una fila está pendiente de revisión en el momento actual, no si en algún momento pasó por la cascada, y una imputación puede quedar posteriormente sobrescrita por una revisión definitiva de SMARD sin dejar rastro. En consecuencia, no es posible reportar a día de hoy, ni para esta ventana ni retrospectivamente, cuántas de las marcas evaluadas fueron efectivamente imputadas de este modo; instrumentar esa

trazabilidad se recoge como mejora del pipeline en la Sección [5.2](#).

Conclusiones y Líneas de Trabajo Futuro

5

5.1 Conclusiones

En este trabajo se ha construido y evaluado un sistema completo para predecir la demanda eléctrica horaria de Alemania, usando LightGBM y únicamente datos públicos. Antes de cerrar, conviene repasar qué se ha conseguido respecto a los tres objetivos que se plantearon al principio.

Objetivo 1. Pipeline de predicción con datos públicos. El pipeline desarrollado funciona solo. Un conjunto de tareas *cron* se encarga de todo: cada hora descarga los datos de SMARD y de Open-Meteo/ERA5 y los va acumulando en una base de datos SQLite; una vez por semana re-entrena el modelo LightGBM con los últimos 24 meses de observaciones; y las predicciones resultantes se consultan desde un panel web interactivo. Lo interesante de esta arquitectura es que con datos gratuitos se puede montar un predictor operativo de principio a fin, desde la ingesta hasta la explotación, sin depender de fuentes privadas ni de mercado. Desde el 4 de julio de 2026 el pipeline registra en vivo las predicciones emitidas a siete horizontes distintos (de 1 a 168 horas) antes de que exista la demanda real correspondiente, lo que inaugura una validación estrictamente prospectiva del sistema en condiciones operativas reales (Sección 4.4.3). Ese registro sacó a la luz dos debilidades operativas del prototipo: los fallos de escritura por concurrencia en SQLite, ya corregidos mediante un margen de espera en las conexiones, y los huecos causados por los retrasos de publicación del proveedor, cuya corrección (un mecanismo de reemisión de predicciones huérfanas) queda registrada como línea de trabajo futura (Sección 5.2). En cuanto al modelo en sí, la configuración elegida (pérdida MSE, crecimiento *leaf-wise* y parada temprana sobre una validación cronológica) ha convergido de forma estable y sin señales de sobreajuste. También ha resultado clave la codificación cíclica de las variables periódicas: sin ella, los árboles tratarían la hora o el día del año como enteros ordinales y aparecerían discontinuidades artificiales (por ejemplo, entre las 23 h y las 0 h). Los hiperparámetros finales, a la vista de las curvas de pérdida de entrenamiento y validación, consiguen un equilibrio razonable entre capacidad expresiva y regularización.

Objetivo 2. Suficiencia del conjunto de señales de bajo coste. Con solo dos retardos autorregresivos ($t-1$ y $t-24$ h), las temperaturas horarias de diez ciudades alemanas y unas cuantas variables de calendario codificadas cíclicamente, el modelo alcanza, medido en vivo

sobre la ventana de validación prospectiva de julio de 2026, un MAE de unos 717 MW y un MAPE del 1,50 % en la evaluación a un paso ($h = 1$, modo *nowcasting*). No es poca cosa para un conjunto de señales tan austero. El resultado queda en el entorno del rango que [Ribeiro y dos Santos Coelho \(2023\)](#) sitúan para los mejores modelos STLF en sistemas eléctricos nacionales, así que parece razonable concluir que las señales gratuitas y de bajo coste dan para construir un predictor de calidad más que aceptable. El análisis de importancia por *gain* (Sección 4.4.6) apunta en la misma dirección: dentro de ese conjunto, los retardos y la codificación horaria concentran la mayor parte de la ganancia; la temperatura y el calendario quedan en un papel secundario, aunque no prescindible. Eso sí, este trabajo no aísla mediante ablación cuánto perdería el modelo si se retirase cada familia de variables por separado. La evaluación multi-horizonte (Tabla 4.7) completa el cuadro: a 24 horas vista (el equivalente a un pronóstico *day-ahead*) el MAPE es del 3,45 %, por debajo del 4,03 % de la previsión oficial de SMARD sobre las mismas 175 marcas en las que ambas previsiones están disponibles; se mantiene en el 4–4,3 % a dos y tres días vista, y sube hasta un 5,31 % a $h = 168$, cifra esta última que se apoya en solo 41 observaciones y debe tomarse como indicativa.

Objetivo 3. Análisis de la predicción autorregresiva a ciegas. Al evaluar sistemáticamente por horizonte quedó claro que el MAPE del 1,50 % del modo paso a paso tiene truco: es un esquema de *nowcasting* en el que el modelo conoce el valor real de la demanda en $t-1$. Como la serie está fuertemente autocorrelada ($\rho_{t,t-1} \approx 0,97$), ese error tan bajo no debería sorprender, pero desde luego no puede compararse de tú a tú con la previsión *day-ahead* de SMARD ($h = 24$). Detectar esta asimetría metodológica es uno de los resultados más valiosos del trabajo: pone sobre la mesa lo engañosas que pueden ser las evaluaciones a un solo paso cuando se quiere medir la capacidad predictiva real de un sistema STLF en operación. Por otro lado, se ha observado que las variables exógenas deterministas (temperatura y calendario) hacen de ancla y frenan la deriva del modelo cuando predice de forma autorregresiva encadenada, algo coherente con lo que [Taieb y Hyndman \(2014\)](#) describen para las estrategias recursivas de gradient boosting. El paso al protocolo prospectivo en vivo resolvió la principal objeción que cabía hacer a la comparación con SMARD: la fila $h = 24$ del modelo se emite ahora con 24 horas reales de antelación, un régimen de información equiparable (incluso algo más exigente) al de la previsión *day-ahead* oficial, y sobre las mismas marcas el MAPE sale favorable (3,45 % frente a 4,03 %). Aun así, la comparación debe seguir tomándose como un indicio y no como una prueba de superioridad: la ventana cubre una única semana de verano, y un test de Diebold-Mariano sobre tan pocas observaciones autocorrelacionadas carecería de potencia. El contraste formal queda pendiente de que la ventana de validación acumule varios meses.

5.2 Líneas de trabajo futuro

Quedan abiertas varias líneas de trabajo que darían continuidad natural a lo hecho hasta ahora:

1. Evaluación multi-horizonte más allá de una semana y contraste estadístico formal. La evaluación *walk-forward* de la Sección 4.4.2 ya cubre $h \in \{1, 6, 12, 24, 48, 72, 168\}$ horas de forma medida (no estimada). Faltaría estirla más allá de $h = 168$ para ver en qué punto el error satura o repunta, y aplicar el test de Diebold-Mariano (Sección 3) para confirmar si la diferencia frente a SMARD es estadísticamente significativa en cada horizonte.
2. Incorporación de retardos adicionales. Añadir el retardo $t-168$ h (la misma hora de la semana anterior) para capturar mejor la estacionalidad semanal, que cobra importancia sobre todo cuando se predice a más de 24 horas.
3. Extensión a otros sistemas eléctricos. Probar la misma arquitectura con el sistema eléctrico español (datos de Red Eléctrica de España). Serviría para comprobar hasta qué punto el enfoque es transferible y para comparar el rendimiento entre sistemas con perfiles de demanda y mezclas energéticas distintos.
4. Incorporación de variables de generación renovable. Añadir la producción solar y eólica prevista como exógenas, de modo que el modelo pueda recoger el efecto de la generación variable sobre la demanda neta del sistema.
5. Comparación con arquitecturas de aprendizaje profundo. Poner a competir el LightGBM con redes LSTM, modelos Transformer (por ejemplo, Temporal Fusion Transformer) y modelos fundacionales de series temporales, usando el mismo conjunto de datos y el mismo protocolo *walk-forward* para que la comparación sea limpia.
6. Predicción probabilística. Extender el sistema para que devuelva intervalos de predicción, ya sea mediante regresión por cuantiles (*quantile regression*) o por conjuntos (*ensemble*). Con ello se podría cuantificar la incertidumbre operativa de las previsiones, algo muy útil para la gestión de reservas.
7. Modelos directos frente a recursivos. Comparar la estrategia recursiva adoptada con la alternativa directa (un LightGBM independiente por horizonte) bajo el mismo protocolo de evaluación, siguiendo la metodología de Taieb y Hyndman (2014).
8. Calendario avanzado. Sustituir la librería `holidays.DE`, que solo recoge festivos federales, por un calendario diferenciado por estado federado (*Bundesland*) que pondere cada festivo regional según el peso del estado en el consumo eléctrico total. También merecería la pena añadir como variables binarias el calendario escolar alemán (*Schulferien*), los puentes (*Brückentage*) y los eventos multitudinarios como el Karneval o el Oktoberfest.
9. Meteorología avanzada. Ir más allá de `temperature_2m`: la radiación solar de onda

corta (*shortwave_radiation*) ayudaría a capturar el efecto fotovoltaico y la «curva de pato» de la demanda neta, y la velocidad del viento, la humedad relativa y la nubosidad completarían el cuadro meteorológico.

10. Factores socioeconómicos. Incorporar el precio de la electricidad en el mercado *day-ahead* (EPEX SPOT / ENTSO-E) como exógena, para modelar cómo responde la demanda industrial a las señales de precio (*demand response*), junto con índices de producción industrial que permitan anticipar cambios de nivel en la demanda base.
11. Trazabilidad de la imputación. Añadir al esquema de `datos_alemania` una columna de auditoría (por ejemplo, `fuentes_demanda_real`) que registre de forma permanente si un valor fue observado directamente o sustituido por alguno de los pasos de la cascada de imputación (Sección 4.3.1), y con qué paso concretamente. Esto permitiría, entre otras cosas, excluir o señalar de forma explícita las marcas imputadas con la previsión de SMARD al comparar el modelo contra ese mismo predictor (Sección 4.4.11); desaparecería así el riesgo de circularidad descrito en la Sección 4.4.13.
12. Variables de inercia térmica. Añadir medias móviles de temperatura a 48 h como predictores. La idea es reflejar la inercia térmica de los edificios y que el modelo tarde menos en reaccionar ante olas de frío o de calor sostenidas.
13. Consolidación de la validación operativa en vivo. Extender la ventana de registro prospectivo multi-horizonte iniciada el 4 de julio de 2026 (Sección 4.4.3), fuente única de las métricas de este trabajo, hasta cubrir al menos un ciclo anual completo, de modo que las métricas ganen potencia estadística y representatividad estacional, y el contraste de Diebold-Mariano frente a SMARD pueda aplicarse con garantías. En paralelo, robustecer el propio registro: dotar al *slot* de predicción a una hora vista de un mecanismo de reemisión que recupere las predicciones huérfanas causadas por los retrasos de publicación de SMARD, y evaluar la migración de la persistencia de SQLite a un motor cliente-servidor (por ejemplo, PostgreSQL) que elimine de raíz los fallos de escritura por concurrencia.
14. Refit final sobre la ventana completa y selección *rolling-origin* del número de árboles. Modificar `pipeline.sh` para que, tras la parada temprana, se reentrene el modelo sobre los veinticuatro meses completos con `num_iterations` fijado en el árbol óptimo de la fase de validación y sin fichero de validación, de modo que el modelo desplegado incorpore también el año más reciente y desaparezca el desfase de un año en los datos de entrenamiento que impone la partición 50/50 actual (Sección 4.4.13). El cambio es de coste mínimo (una segunda invocación del binario de LightGBM por ciclo semanal), con la única salvedad de que asume que el número óptimo de árboles se conserva, aproximadamente, al duplicar el volumen de datos de entrenamiento. Como

refinamiento posterior, sustituir la partición única por un esquema de validación con origen móvil (*rolling-origin*) para la selección del número de árboles, en línea con [Petrović et al. \(2026\)](#), lo que además desligaría el diagnóstico de sobreajuste de la elección concreta del punto de corte y haría plenamente operativa la consideración *walk-forward* ya recogida en el estado del arte.

Conjuntos de datos anexos



Se entregan junto a la memoria dos ficheros de datos:

- `exportacion_demanda.xlsx`: volcado íntegro de la base de datos de producción, generado con `exportar_excel.py` el 12 de julio de 2026. Contiene 88.111 registros horarios que cubren 3.673 días consecutivos (del 29 de junio de 2016 al 19 de julio de 2026, con cobertura horaria prácticamente completa —los detalles de los huecos residuales se recogen en la Sección 4.3.5—), con 34 columnas por registro: demanda real, previsión oficial SMARD, predicción operativa del modelo propio y sus errores, las predicciones prospectivas a siete horizontes (`prediccion_1h` a `prediccion_168h`, pobladas desde el 4 de julio de 2026), las variables de entrada al modelo (retardos, codificaciones cíclicas, día de la semana y festivos), las temperaturas horarias de las diez ciudades seleccionadas y los metadatos de consolidación (`creado_el`, `smard_consolidado`, `meteo_consolidado`). Las filas posteriores al momento de exportación contienen únicamente marca temporal y predicciones. Este fichero documenta el sistema completo y sustenta el análisis exploratorio; las métricas de error, en cambio, no se calculan sobre él.
- `exportacion_demanda_para_metricas.xlsx`: subconjunto restringido a la ventana de validación operativa en vivo, del 4 de julio de 2026 (00:00 UTC) al 12 de julio de 2026 (15:00 UTC), con 208 marcas horarias. **Todas las métricas de error reportadas en la memoria** (Tablas 4.6 y 4.7) se calculan exclusivamente sobre este fichero.

La descripción detallada de la estructura de columnas y su justificación se encuentra en la Sección 4.3.5.

Código fuente



El código fuente del proyecto está organizado en los siguientes módulos principales:

- `adquirir_datos.py` — Ingesta horaria de datos SMARD y Open-Meteo/ERA5 en SQLite.
- `construir_modelo.py` — Extracción y formato del conjunto de entrenamiento.
- `pipeline.sh` — Orquestador del ciclo de re-entrenamiento semanal en producción (descarga, construcción del *dataset*, entrenamiento LightGBM y recálculo de predicciones pendientes).
- `entrenar.sh` — Lanzamiento manual del entrenamiento LightGBM en C++ a partir de ficheros `.train/.valid` ya contruidos (usado en la inicialización del entorno, no en el ciclo automático semanal).
- `predecir_futuro.py` — Cálculo de la inferencia autorregresiva sobre el horizonte solicitado; la persistencia de las predicciones multi-horizonte en la base de datos la realiza `adquirir_datos.py` en su modo *cron*.
- `panel_control.py` — Servidor Flask con panel web interactivo.
- `herramientas/exportar_excel.py` — Exportación de la base de datos de producción a los ficheros anexos `.xlsx` (Apéndice [A](#)).
- `herramientas/prueba_sobreajuste.py` — Diagnóstico de sobreajuste mediante curvas de pérdida.
- `predecir_tramo.py` — Evaluación de la degradación autorregresiva por horizonte.

Hiperparámetros finales del modelo



La configuración final del modelo LightGBM, almacenada en `conf/train.conf` y vigente en producción, fue seleccionada mediante búsqueda en rejilla sobre los rangos detallados en la Tabla 4.2. Los valores finales resultantes del proceso de optimización son los que minimizan el MSE de validación sin divergencia apreciable entre las curvas de entrenamiento y validación, tal y como se describe en la Sección 4. La Tabla C.1 transcribe directamente los parámetros relevantes de `conf/train.conf`.

Parámetro	Valor final	Descripción
<code>objective</code>	regression (L2)	Función de pérdida: Error Cuadrático Medio (Sección 4)
<code>num_leaves</code>	63	Número máximo de hojas por árbol
<code>max_depth</code>	-1 (sin límite) ¹	No se restringe explícitamente; el control de complejidad recae en <code>num_leaves</code> y <code>min_data_in_leaf</code>
<code>min_data_in_leaf</code>	100	Número mínimo de muestras por hoja terminal
<code>min_sum_hessian_in_leaf</code>	5,0 ²	Suma mínima de hessianos por hoja (regularización adicional)
<code>learning_rate</code>	0,01	Factor de encogimiento (<i>shrinkage</i>)
<code>feature_fraction</code>	0,8	Fracción de columnas muestreadas por árbol
<code>bagging_fraction</code>	0,8	Fracción de filas submuestreadas por iteración
<code>bagging_freq</code>	5	Periodicidad (en iteraciones) del submuestreo de filas
<code>max_bin</code>	255	Número de <i>bins</i> para la discretización de variables continuas
<code>num_iterations</code>	2 000 (máximo)	Tope de árboles; el número real lo determina la parada temprana (Sección 4.1.4.4)
<code>early_stopping_round</code>	50	Iteraciones sin mejora en validación antes de detener el entrenamiento

Tabla C.1: Hiperparámetros finales del modelo LightGBM, transcritos directamente de `conf/train.conf`.

Bibliografía

- Ahmad, T., Chen, H., Huang, R., Kaleem, M., y Shair, J. (2025). Load forecasting in the era of smart grids: Opportunities and advanced machine learning models. *Energy Reports*, 13.
- Akinshin, A. (2021). Unbiased median absolute deviation. aakinshin.net. El propio autor advierte en el artículo que está desactualizado y remite a un tratamiento posterior en arXiv:2207.12005.
- Ansari, A. F., Shchur, O., Küken, J., Auer, A., Han, B., Mercado, P., Rangapuram, S. S., Shen, H., Stella, L., Zhang, X., Goswami, M., Kapoor, S., Maddix, D. C., Guerron, P., Hu, T., Yin, J., Erickson, N., Mutalik Desai, P., Wang, H., Rangwala, H., Karypis, G., Wang, Y., y Bohlke-Schneider, M. (2025). Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*.
- Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S. S., Pineda Arango, S., Kapoor, S., Zschiesner, J., Maddix, D. C., Wang, H., Mahoney, M. W., Torkkola, K., Wilson, A. G., Bohlke-Schneider, M., y Wang, Y. (2024). Chronos: Learning the language of time series. *Transactions on Machine Learning Research*. arXiv:2403.07815.
- Ashraf, I., Songfeng, L., Shafiq, M., Rashid, M., y Nawaz, S. A. (2024). Advancing ensemble learning techniques for residential building electricity consumption forecasting: Insight from explainable artificial intelligence. *PLoS ONE*, 19(11):e0312441.
- Aziz, A. y Than Oo, A. (2020). COVID-19 implications on electric grid operation. *IEEE Smart Grid Bulletin*. Available: <https://smartgrid.ieee.org/bulletins/may-2020/covid-19-implications-on-electric-grid-operation>. Accessed: 2026-07-01.
- Bentéjac, C., Csörgő, A., y Martínez-Muñoz, G. (2021). A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54(3):1937–1967.
- Box, G. E. P. y Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, CA.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., y Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA.
- Bundesnetzagentur (2026). Bundesnetzagentur publishes 2025 electricity market data.
- Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.
- Costa, G., Neto, P., y Barroso, N. (2025). Cyclical temporal encoding and hybrid deep ensembles for multistep energy forecasting. *arXiv preprint arXiv:2512.03656*.
- Das, A., Kong, W., Sen, R., y Zhou, Y. (2024). A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. arXiv:2310.10688.
- De Livera, A. M., Hyndman, R. J., y Snyder, R. D. (2011). Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association*, 106(496):1513–1527.
- Díaz-Iglesias, A., Belaunzaran, X., y Florez-Tapia, A. M. (2025). Short-term power demand forecasting for diverse consumer types to enhance grid planning and synchronisation.
- Energuides (2026). Why does the electricity grid have to stay in balance? Last updated on: 10 April 2026.
- Fan, G., Wang, L., Liu, C., y Wang, J. (2021). A two-stage multistep-ahead electricity load forecasting scheme based on LightGBM and attention-BiLSTM. *Sensors*, 21(22):7697.

- Federal Ministry for Economic Affairs and Climate Action (BMWK) (2025). An electricity market for the energy transition. <https://www.bundeswirtschaftsministerium.de/Redaktion/EN/Dossier/electricity-market-of-the-future.html>. Accessed: 2026-05-31.
- Fildes, R. y Goodwin, P. (2024). Multi-step time series forecasting: Recursive vs direct and hybrid strategies. *Let's Data Science*.
- Fisher, W. D. (1958). On grouping for maximum homogeneity. *Journal of the American Statistical Association*, 53(284):789–798.
- Fraunhofer Institute for Solar Energy Systems ISE (2026). Electricity grid frequency in germany. Data for 22 June 2026, 21:00 (CET).
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.
- Garza, A., Challu, C., y Mergenthaler-Canseco, M. (2024). TimeGPT-1. *arXiv preprint arXiv:2310.03589*.
- Gasparin, A., Lukovic, S., y Alippi, C. (2022). Short-term electricity load forecasting with machine learning. *Information*, 13(2):57.
- Grierson, M. (2025). Is your model time-blind? the case for cyclical feature encoding. *Towards Data Science (Medium)*.
- Grinsztajn, L., Oyallon, E., y Varoquaux, G. (2022). Why tree-based models still outperform deep learning on tabular data. In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520.
- Gross, G. y Galiana, F. D. (1987). Short-term load forecasting. *Proceedings of the IEEE*, 75(12):1558–1573.
- Haben, S., Voß, M., y Holderbaum, W. (2023). Time series forecasting: Core concepts and definitions. In *Core Concepts and Methods in Load Forecasting*, pages 55–66. Springer.
- Hastie, T., Tibshirani, R., y Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, NY, 2 edition.
- He, F., Xu, K., y Zhang, W. (2024). A short-term multi-step electricity load forecasting approach incorporating meteorological and calendar features. In *Proceedings of SPIE*, volume 14136, page 1413608.
- Hedengren, J. y APMonitor Development Team (2024). Autoregressive models with exogenous inputs (ARX). <https://apmonitor.com/dde/index.php/Main/AutoRegressive>.
- Holt, C. C. (2004). Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting*, 20(1):5–10. Reimpresion del original de 1957.
- Hong, T. y Fan, S. (2016). Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting*, 32(3):914–938.
- Hosamo, H. y Mazzetto, S. (2025). Performance evaluation of machine learning models for predicting energy consumption and occupant dissatisfaction in buildings. *Buildings*, 15(1).
- Hyndman, R. J. y Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition.
- Hyndman, R. J., Koehler, A. B., Ord, J. K., y Snyder, R. D. (2008). *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Series in Statistics. Springer, Berlin, Heidelberg.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., y Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Khalid, R., Javaid, N., Al-zahrani, F. A., y Abuhasel, K. A. (2024). Model-based analysis to identify the impact of factors affecting electricity gaps during COVID-19: A case study in Germany. *PLoS ONE*, 19(7):e0306637.
- Kostas, K., Spiliotis, E., y Assimakopoulos, V. (2023). Electricity load and peak forecasting: Feature engineering, probabilistic LightGBM and temporal hierarchies. In *Workshop on Advanced Analytics and Learning on Temporal Data (AALTD), ECML-PKDD 2023*.
- Kundur, P. (1994). *Power System Stability and Control*. McGraw-Hill, New York.
- Lemos-Vinasco, J., Bjørk, J., y Møller, J. K. (2026). Exploring the interpretability of forecasting models for energy balancing market. *arXiv preprint arXiv:2602.00049*.
- LightGBM Developers (2025). Parameters tuning — LightGBM documentation. <https://lightgbm.readthedocs.io/en/stable/Parameters-Tuning.html>. Consultado el 17 de junio de 2026.

- Lim, B., Arık, S. Ö., Loeff, N., y Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.
- Mughees, N., Ali, M., Mughees, A., y Mughees, A. (2024). Short-term electricity-load forecasting by deep learning: A comprehensive survey. *arXiv preprint arXiv:2408.16202*.
- Open-Meteo (2026a). DWD ICON API.
- Open-Meteo (2026b). Weather forecast API.
- Petrović, A., Radovanović, S., y Delibasic, B. (2026). Rolling-origin validation reverses model rankings in multi-step PM10 forecasting: XGBoost, SARIMA, and persistence. *arXiv preprint arXiv:2603.20315*.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., y Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.
- Ramanathan, R., Engle, R., Granger, C. W. J., Vahid-Araghi, F., y Brace, C. (1997). Short-run forecasts of electricity loads and peaks. *International Journal of Forecasting*, 13(2):161–174.
- Rasul, K., Ashok, A., Williams, A. R., Ghonia, H., Bhagwatkar, R., Khorasani, A., Darvishi Bayazi, M. J., Adamopoulos, G., Riachi, R., Hassen, N., Biloš, M., Garg, S., Schneider, A., Chapados, N., Drouin, A., Zantedeschi, V., Nevmyvka, Y., y Rish, I. (2024). Lag-Llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*.
- Red Eléctrica de España (2026). La transición energética en España. https://www.ree.es/sites/default/files/11_PUBLICACIONES/Documentos/Transicion_Energetica.pdf. Accedido el 31 de mayo de 2026.
- Ribeiro, M. H. D. M. y dos Santos Coelho, L. (2023). An overview of short-term load forecasting for electricity systems operational planning: Machine learning methods and the Brazilian experience. *Energies*, 16(21):7444.
- SMARD (2026). Forecast data. Recuperado el 2 de mayo de 2026 de <https://www.smard.de/page/en/wiki-article/5884/206318/forecast-data>.
- Stock, J. H. y Watson, M. W. (2020). *Introduction to Econometrics*. Pearson Education Limited, Harlow, United Kingdom, global 4th edition.
- Taieb, S. B. y Hyndman, R. J. (2014). Boosting multi-step autoregressive forecasts. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*.
- Ullah, K., Ahsan, M., Hasanat, S. M., Haris, M., You-saf, H., Raza, S. F., Tandon, R., Abid, S., y Ullah, Z. (2024). Short-term load forecasting: A comprehensive review and simulation study with cnn-lstm hybrids approach. *IEEE Access*, 12:111858–111881.
- Valor, E., Meneu, V., y Caselles, V. (2001). Daily air temperature and electricity load in Spain. *Journal of Applied Meteorology*, 40(8):1413–1421.
- Winters, P. R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3):324–342.
- Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., y Sahoo, D. (2024). Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. *arXiv:2402.02592*.